

Monash University
Faculty of Engineering
Master of Professional Engineering
Civil Engineering Specialization
CIV5178 – Advanced Water Treatment

Wastewater Treatment Plant Design

Presented by:
Kevin González – 35477350
Jhon Solórzano – 33997470
Julian Mejia – 34774564

Presented to:
Unit coordinator and chief examiner: Brandon Winfrey

May 15, 2025

EXECUTIVE SUMMARY

The purpose of this report is to present the design of a new wastewater treatment plant (WWTP) for a rapidly developing northwest area in Melbourne. The plant is designed to address current and future community needs over a 20-year operational lifespan, ensuring compliance with local guidelines and environmental standards. The design encompasses preliminary, primary, and secondary treatment processes, along with an environmental impact assessment.

The methodology involved population and flow rate estimations based on current and projected data, incorporating domestic, commercial, and infiltration contributions. Key design parameters were derived from established guidelines, including the AMF L8 Sewerage Planning and Design Principles and Melbourne Water datasets. The preliminary treatment includes coarse screens and vortex grit chambers, while primary treatment utilizes circular clarifiers for sedimentation. Secondary treatment employs a Complete-Mix Activated Sludge (CMAS) process to achieve high pollutant removal efficiency. The environmental impact was assessed using the Streeter-Phelps model to ensure dissolved oxygen levels in the receiving waterway remain above regulatory thresholds. Biosolids management includes a plan for handling and treating sludge produced from the treatment processes, including thickening, anaerobic digestion, dewatering, and ultimate disposal or reuse. Findings indicate the plant can handle an average flow of 74,706 m³/day and a peak flow of 131,450 m³/day, with projected increases to 108,324 m³/day and 193,226 m³/day, respectively. The preliminary treatment is designed with four 1.1m wide screens and an equalization basin (adjusted to approx. 21000 m³). The primary treatment, using three circular clarifiers, is designed to achieve approximately 28.7% BOD and 50% TSS removal. The secondary CMAS system is designed for 93.2% BOD removal, targeting a final effluent BOD of 18 mg/L. The environmental assessment confirmed compliance with a minimum dissolved oxygen level of 5.95 mg/L in the receiving waterway. Biosolids management will handle around 48 ton of sludge daily through digestion and dewatering.

In conclusion, the proposed WWTP design is robust, scalable, and environmentally compliant, capable of meeting current demands while accommodating future growth. The inclusion of redundancy and adaptive operational strategies ensure reliability and resilience under varying conditions.

AI USAGE ACKNOWLEDGMENT

We used ChatGPT (model o3 and mini o4) to assist us to build graphs, tables and functions in the python code. Besides, it assists us to create .bib references for this report.

CONTENTS

1	Introduction and Conceptual Design Description	1
1.1	Population estimations	1
1.2	Current flowrates estimation	1
1.3	Minimum current flow rates	2
1.4	Maximum current flow rates	2
1.5	Population Growth and Future Design Considerations	2
1.6	Influent Pollutant Load Estimation	3
2	Preliminary Treatment Design	4
2.1	Facility types	4
2.2	Screens design	4
2.2.1	Blockage Scenario	5
2.2.2	Operational considerations	6
2.3	Equalization basin	7
2.4	Protection against failure	7
3	Primary treatment	7
3.1	Facility Types	7
3.2	Primary Sedimentation	8
3.2.1	Pollutant removal	8
3.3	Design description	9
3.3.1	Weir Loading Rate (WLR) Assessment	10
3.3.2	V-Notch and Launder Design	11
3.3.3	Launder Flow Conditions	12
3.3.4	Inlet design	13
3.4	Protection and future considerations	13
4	Secondary treatment	14
4.1	Facility type	14
4.2	Pollutant removal	14
4.3	Design description	15
4.3.1	Sludge Characteristics and Return Flow	15
4.3.2	Aeration Tank Sizing	15
4.3.3	Organic Loading Rate	15
4.3.4	Sludge Age and Stability	16
4.3.5	Aeration Tank Configuration	16
4.3.6	Aeration requirements	17
4.3.7	Design Summary	18
4.4	Protection against failure	18
5	Environmental Impact Assessment	18
5.1	Impact on receiving waterway	18
5.2	Biosolids management	21
5.3	Carbon emissions	21

6 Summary	21
Appendix 1 Peaking Factors	1
Appendix 2 Area and Population per suburb	2
Appendix 3 Preliminary treatment, screens section	13
Appendix 4 Temperature multiplier for detention time	14
Appendix 5 Mass balance technique for equalization volume	15
Appendix 6 Equalization Off-line diagram	17
Appendix 7 Inlet sections views	18
Appendix 8 Streeter-Phelps Model	19
Appendix 9 Design criteria for primary sedimentation basins	20
Appendix 10 Operational characteristics of activated sludge process	21
Appendix 11 Python scripts	23
Appendix 12 Typical design parameters for activates sludge processes	58

LIST OF FIGURES

1	Wastewater treatment plan conceptual diagram, by Authors.	22
2	Daily fluctuation of peaking factors at each hour of the day for flow rates on an average day in a similar wastewater catchment.	1
3	Boxplots of Biochemical Oxygen Demand (BOD) from Melbourne Water ETP dataset . . .	12
4	Preliminary treatment, screens section.	13
5	Temperature multiplier for detention time [Metcalf and Eddy, 2014].	14
6	Mass balance diagram for current conditions.	15
7	Mass balance diagram for projected conditions.	16
8	Off-line equalization basin –adapted from Fig 3-20, Davis,2010.	17
9	Feed well with energy dissipating inlet (EDI), plan view [Metcalf and Eddy, 2014].	18
10	Inlet profile view [Metcalf and Eddy, 2014].	18
11	Dissolved oxygen deficit. Vertical axes being the deficit, with zero representation saturation.	19

LIST OF TABLES

1	Understudy landscape area estimation	2
2	Current population and flow rates	3
3	Projected population and flow rates	3
4	Design input values for screen and channel	4
5	Screen and channel design results	5

6	Blockage scenario - current average flow	6
7	Blockage scenario - projected peak flow	6
8	Operational resilience and contingency measures [Davis, 2010]	8
9	Standard parameters for detention time [Metcalf and Eddy, 2014].	9
10	Input values for clarifier sizing	10
11	Final clarifier design parameters	11
12	V-notch and launder design summary	13
13	Design input values for secondary treatment	18
14	Key results for secondary treatment design	19
15	Input values for Streeter-Phelps DO sag model	20
16	Environmental impact model results	20
17	Environmental impact model results	22
18	Melbourne suburbs with area and population	2
19	Typical design criteria for primary sedimentation basins [Davis, 2010]	20
20	Operational Characteristics of Activated Sludge Process [Metcalf and Eddy, 2014]	22
21	Typical design parameters for activated sludge processes [Metcalf and Eddy, 2014]	59

1 INTRODUCTION AND CONCEPTUAL DESIGN DESCRIPTION

Population and economical growth among other factors require the subsequently development of cities and the general infrastructure that supports them. As a cities expand, the electrical grid, road systems, drinking water and Wastewater recollection and treatment, among others, need to increase their capabilities as well. In the recent years, a northwest area in Melbourne has been rapidly developing and therefore, requires the construction of a new Wastewater treatment plant that can address the current and future needs of the community. In the present report, the authors present the design of the abovementioned plant following local guidelines.

The designed is conformed by a *Preliminary Treatment - Section 2* where the screening is designed to remove coarse material on the flow stream to protect downstream process equipment and increase its reliability [Metcalf and Eddy, 2014]. The next step is the *Primary Sedimentation - Section 3*, designed to remove the settled solids and floating material. This process can reach removals ratios of 50% to 70% of the total suspended solids (TSS) and 25% to 40% of the BOD. It is followed by the *Secondary Treatment - Section 4*, in which sludge and activation systems are used to remove the soluble, colloidal and particulate suspended organic substances by coagulating them through the microorganisms (sludge).

Finally, the environmental impact of the incorporation of the treated wastewater into the waterway is assessed against the local guidelines applicable to determine whether it complies with its requirements or if further treatment shall be implemented.

1.1 Population estimations

To determine the wastewater flow rates and mass loads for development areas that the plant should be capable of process, where actual flow and load measurements are not available, the population equivalent can be used. This is a standardization of the wastewater production and characteristics based on the amount of people that would produce the same values. For example, in the *AMF L8 Sewerage Planning and Design Principles* which is the standard for wastewater treatment plants, developed by *Yarra Valley Water* which covers the area under design in the present report, on its *Table 3-4* equivalent population factors are presented for different development types for both residential and commercial use. For this report, based on previous surveys and analysis, it is known that the equivalent population from domestic contributions (PE_{domestic}) is equal to 340000. Similarly, for commercial contributions, the equivalent population is known to comply with the following calculation: $PE_{\text{commercial}} = \lceil \sqrt{10} \times 9000 \rceil = 28461$.

1.2 Current flowrates estimation

According to the *Melbourne Water - Customer Outcomes Performance Report* the inflow rate percapita can be estimated as 162 L/PE/day [Melbourne Water, 2024a]. Therefore, the $Q_{\text{current avg, domestic}}$ can be calculated as $\frac{Q_{\text{domestic per capita}} \times PE_{\text{domestic}}}{1000} = 55080.0 \text{ m}^3/\text{day}$. Similarly, based on previous surveys it got determine that the commercial contribution for this area is 280 L/PE/day. Therefore, the $Q_{\text{current avg, commercial}}$ can be calculated as $\frac{Q_{\text{commercial per capita}} \times PE_{\text{commercial}}}{1000} = 7969.08 \text{ m}^3/\text{day}$.

The infiltration/inflow flow rate, when no data is available, can be estimated based on either the total length of the sewer system, or based on the area to be covered. For the last one, the ratio can

vary from 0.2 up to 28 m³/ha/day [Metcalf and Eddy, 2014]. In this report, the authors have chosen the lower limit as the system will be new and therefore the damage on the pipelines will be minimum.

As no data of the covered area is available, the authors have estimated it based on the average population per area ratio of Melbourne suburbs. The Melbourne population density per suburb data is available in the *Australian Bureau of Statistics* web page, see Appendix 2 for the raw data. From it, the Population average per suburb ($P_{average}$) and the average landscape area ($A_{average}$ km²) were calculated. With the $P_{average} / A_{average}$ ratio, multiplied by the $PE_{domestic}$ the estimated area of the understudy suburb ($A_{estimated}$) is obtained. The $PE_{commercial}$ is not included as it does not represent actual population. In the Table 1 the obtained results for the infiltration rate estimation. In the Table 2 the current flow rates obtained are summarized.

Table 1: Understudy landscape area estimation

$P_{average}$	$A_{average}$ (km ²)	$A_{average} / P_{average}$	$A_{estimated}$ (km ²)	Ratio (m ³ /ha/day)	$Q_{infiltration}$ (m ³ /day)
11200.0	18.65	0.0017	566.17	0.2	11323.41

1.3 Minimum current flow rates

The minimum flow rates can be estimated using the minimum peaking factor which from the Figure 2, in Appendix 1, is equal to 0.3. The current flow rates shown in the Table 2 are multiplied by this factor, and results are presented in Table 3.

1.4 Maximum current flow rates

The maximum flow rates can be estimated using the maximum peaking factor which from the Figure 2 is equal to 1.9. The current flow rates shown in the Table 2 are multiplied by this factor, and results are presented in Table 3.

1.5 Population Growth and Future Design Considerations

To account for future demand on the wastewater treatment plant, the design incorporates a projected population growth. Based on previous analysis, it was determined that the annual growth rate can be estimated as: $G_{rate} = 1 + 10^{0.015} = 2.04\%$. The current population is therefore projected during the life span of the plant. The life span is commonly adopted as 20 years as it balances the need to consider future growth and technological advancements while ensuring economic feasibility [Water Environment Federation, 2017]. The projected population can be calculated based on (Eq. 1).

$$PE_{projected} = \left[PE \times \left(1 + \frac{\text{Growth Percentage}}{100} \right)^{20} \right] \quad (\text{Eq. 1})$$

With the per capita flow rates mentioned in the Section 1.2, following the same procedure there mentioned the flow rates for the projected population can be calculated as well. Additionally, the minimum and maximum projected flow rates can be calculated following the peak minimum factor and

peak maximum factor mentioned in Sections 1.3 and 1.4, respectively. Results are shown in the Table 3.

To project the infiltration, a different approach is used. This value is not controlled by the population as it depends more on the environmental phenomenon. Because of this, it is projected based on an environmental factor instead. The *Environment, Land, Water and Planning* department of the *Victoria State Government* in their *Guidelines for the Adaptive Management of Wastewater Systems Under Climate Change in Victoria*, specified that the climate change factor can be taken as 1.2. The infiltration flow rate presented in Table 2 is therefore multiplied by this factor and presented in the Table 3.

Table 2: Current population and flow rates

Contribution type	Population Equivalent	Minimum flow rate (m ³ /day)	Average flow rate (m ³ /day)	Maximum flow rate (m ³ /day)
Domestic	340000	16524	55080	104652
Commercial	28461	2390.72	7969.08	15141.25
Infiltration	—	11657.00	11657.00	11657.00
Total	—	30572	74706	131450

Table 3: Projected population and flow rates

Contribution type	Population	Minimum flow rate (m ³ /day)	Average flow rate (m ³ /day)	Maximum flow rate (m ³ /day)
Domestic	508715	24724	82412	24724
Commercial	42584	3577	11924	22655
Infiltration	—	13988.40	13988.40	13988.40
Total	—	42289	108324	193226

1.6 Influent Pollutant Load Estimation

Following the guidelines established for wastewater treatment design [Water Environment Federation, 2017, Metcalf and Eddy, 2014], the analysis focused on two key pollutants: Biochemical Oxygen Demand (*BOD*) and Total Suspended Solids (*TSS*), which are considered the most relevant indicators for organic and solids loading.

As no specific influent quality data was available for the study area, representative pollutant concentrations were obtained from Melbourne Water's open dataset on the Eastern Treatment Plant (ETP) [Melbourne Water, 2024b]. This dataset, which includes daily influent quality records from 2014 onwards, is publicly accessible at <https://data-melbournewater.opendata.arcgis.com>, see Appendix 2 for the whole data presentation. From this data, the average inflow *BOD* is 370 mg/L, which when analyzed by pollutant load yields results within typical reported values [Metcalf and Eddy, 2014]. The inflow *TSS* varies from 0.5 to 1.0 times the *BOD* concentration, thus conservatively it is taken as the same value.

2 PRELIMINARY TREATMENT DESIGN

2.1 Facility types

Coarse screens designed to retain gross pollutants greater than 10 mm are proposed to effectively protect downstream treatment processes. These screens, commonly employed in wastewater treatment plants (WWTPs), efficiently remove large solids, rags, and debris [Davis, 2010]. However, it is noted that their contribution to the removal of biochemical oxygen demand (BOD) is considered negligible.

Each mechanical screen unit should be installed in its own dedicated channel, facilitating isolation for maintenance or emergency purposes. Additionally, vortex grit chambers are proposed to remove grit particles up to 100 μm . This inclusion significantly reduces grit accumulation in downstream units and prevents sediment buildup within the equalization basin, thus enhancing the overall reliability and effectiveness of subsequent treatment processes.

2.2 Screens design

When the flow goes through the screens, it might suffer a headloss due to the obstruction and turbulence created by the screens limited open area. This headloss can be neglected in the primary screening as the opening area remains big enough to not cause significant headloss. However, in the case of secondary screens, as the open area is smaller, the headloss needs to be accounted for. To determine its value and sizing of the secondary screening, the velocity upstream of the bars, the hydraulic head loss, and the cross-sectional flow area are calculated. The design assumes that flow depth is governed by the downstream hydraulic condition, and the channel width is then determined accordingly. The calculations are based on the peak projected flow rate.

The values used for this calculation are summarized in Table 4. The flow velocity through the screen v_{screen} should be between 0.4 and 0.9 m/s [Davis, 2010], the latter is selected and checked against the other constraints of the design. The bar spacing or opening size (b) is taken as 0.009 m as the screen should remove particles bigger than 10mm. The bar width w is selected as 0.007 m based on commercial screens availability. The discharge coefficient (C_d) is commonly taken as 0.84, although it might be adjusted based on the screening specifications. Finally, the minimum required downstream flow depth was determined to be 1.0 m based on previous analysis.

Table 4: Design input values for screen and channel

Parameter	Value	Description
v_{screen}	0.9 m/s	Flow velocity through the screen
b	0.009 m	Spacing between bars
w	0.007 m	Bar width
C_d	0.84	Discharge coefficient
g	9.81 m/s^2	Gravitational acceleration
d	1.0 m	Assumed downstream flow depth

From Equations (2) to (5) the upstream velocity (v_{upstream}), the headloss (h_{loss}), the upstream flow depth (d_{upstream}), the wetted cross-sectional area (A_{wet}) and required channel width (W_{channel}) were determined. The screens' width is recommended to be between 0.6 and 1.1m [Metcalf and Eddy, 2014]. The number of screens required (N_{screens}) is therefore calculated based on the channel width divided by the maximum recommended screen width (1.1 m) and rounding up to the nearest integer. Results are

shown in the Table 5. In the Appendix 3 is shown the elevations of the screens indicating the hydraulic control.

$$v_{\text{upstream}} = v_{\text{screen}} \cdot \frac{b}{b + w} \quad (2)$$

$$h_{\text{loss}} = \frac{1}{2gC_d^2} (v_{\text{screen}}^2 - v_{\text{upstream}}^2) \quad (3)$$

$$A_{\text{wet}} = \frac{Q_{\text{peak}}}{v_{\text{upstream}}} \quad (4)$$

$$W_{\text{channel}} = \frac{A_{\text{wet}}}{d + h_{\text{loss}}} \quad (5)$$

$$W_{\text{screen}} = \frac{W_{\text{channel}}}{N_{\text{screens}}} \quad (6)$$

Table 5: Screen and channel design results

Output	Value	Description
v_{upstream}	0.506 m/s	Velocity before the screen
h_{loss}	0.04 m	Head loss across the screen
d_{upstream}	1.04 m	Flow depth upstream
A_{wet}	4.42 m ²	Wetted cross-sectional area
W_{channel}	4.4 m	Required screen channel width
N_{screens}	4	Number of screens required
W_{screen}	1.1 m	Screen width

2.2.1 Blockage Scenario

To ensure robustness of the preliminary screening system, the design is evaluated under a conservative condition of 50% screen blockage. This is important as screen clogging can lead to increased velocities and head losses, and subsequent failure of the system.

The analysis is based on the Bernoulli equation applied between the upstream and downstream sections of the screen channel:

$$h_1 + \frac{v_1^2}{2g} = h_2 + \frac{v_2^2}{2g} + \Delta h \quad (7)$$

where: - h_1 and h_2 : upstream and downstream water depths, respectively [m] - v_1 and v_2 : corresponding flow velocities [m/s] - Δh : head loss due to the screen [m] - g : gravitational acceleration (9.81 m/s²).

The upstream water depth h_1 is determined by:

$$h_1 = \frac{h_2 \cdot v_2}{v_1} \quad (8)$$

The head loss across a partially blocked screen is calculated using a modified version of Eq.(3) to account for the blockage ratio which affects the discharge velocity coefficient (v_{coef}):

$$\Delta h = \frac{(v_{\text{coef}}^2 - 1^2) \cdot v_1^2}{2gC_d^2} \quad (9)$$

$$v_{\text{sc}} = \frac{Q/N_{\text{screens}}}{h_2 \cdot \left(\frac{W_{\text{screen}} - w}{w + b} \cdot w \right)} \quad (10)$$

$$v_2 = \frac{Q/N_{\text{screens}}}{h_2 \cdot W_{\text{screen}}} \quad (11)$$

$$h_1 = \frac{h_2 \cdot v_2}{v_1} \quad (12)$$

$$v_{\text{coef}} = \frac{w + b}{0.5 \cdot w} \quad (13)$$

Substituting Eqs. (8) and (9) into the Bernoulli equation (7), it is obtained:

$$\frac{h_2 v_2}{v_1} + \frac{v_1^2}{2g} = h_2 + \frac{v_2^2}{2g} + \frac{(v_{\text{coef}}^2 - 1^2) \cdot v_1^2}{2gC_d^2} \quad (14)$$

Solving Eq.(14) for v_1 leads to a third grade equation, which is solved for both the current average and projected peak flow rates, see Tables 2 and 3. The purpose is to ensure that even with 50% blockage, the upstream velocity remains within acceptable hydraulic limits (typically not exceeding 1 m/s) and the head loss does not cause excessive backwater.

The obtained velocities are summarized in Table 6 and 7 for current average and projected peak condition, respectively. It can be noticed that for both cases, the velocities remain within the acceptable values.

Table 6: Blockage scenario - current average flow

Parameter	Result	Units	Description
v_{sc}	1.0	m/s	Screen design velocity
v_2	0.56	m/s	Velocity downstream
v_1	0.40	m/s	Velocity upstream

Table 7: Blockage scenario - projected peak flow

Parameter	Result	Units	Description
v_{sc}	0.87	m/s	Screen design velocity
v_2	0.51	m/s	Velocity downstream
v_1	0.44	m/s	Velocity upstream

2.2.2 Operational considerations

During the screens' operation, a target velocity of 0.7 - 0.9 m/s at peak flows is ideal for operational balance, ensuring effective screenings' removal without excessive turbulence or debris bypass. The channel dimensions were sized to maintain appropriate flow velocities under varying conditions and considering future growth and possible peak events. Finally, the screens should be easily accessible and designed for rapid maintenance response. Scheduled inspections and routine cleaning are required, especially after high flow events.

2.3 Equalization basin

The planned flow management strategy for the wastewater treatment plant incorporates a phased approach utilizing an equalization basin to optimize downstream treatment processes over the facility's operational lifetime. Initially, operation will commence with the equalization basin configured to deliver a constant flow rate downstream ($Q_{avg}^{current}$). This is facilitated by a significant design storage volume $V_{\text{equalization}}$, approximated at 21000.0 m^3 , obtained through the mass balance technique, see Appendix 5 inclusive of a safety factor of 1.1. It is noted that, according to the clarifier design shown in Section 3.3, the required equalization basin volume $V_{\text{equalization}}^{\text{req}}$ is actually 5355.78 m^3 . However, the objective of this initial constant flow regime is to establish stable hydraulic and organic loading conditions, thereby simplifying the commissioning process and enabling precise optimization of subsequent treatment units.

As the service area population increases and influent flow rates exhibit greater volume and variability, the operational strategy for the equalization basin will transition to one focused on smoothing flow variations. Simultaneously, a critical peak flow constraint of $114454.28 \text{ m}^3/\text{day}$ will be imposed on the flow discharged from the basin to the primary clarifier. This specific flow rate is selected to ensure the primary clarifier operates within its optimal hydraulic loading range, thereby maintaining the required detention time for efficient settleable solids' removal. This adaptive strategy ensures that the plant can effectively manage increasing influent flows while preserving the performance integrity of key treatment processes throughout its operational lifespan.

This equalization basin requires aeration so as to prevent odors and solids deposition. The proposed equalization is an off-line basin as seen in the diagram shown in Figure 8, see Appendix 6.

2.4 Protection against failure

To ensure continuous operation and protect against failure scenarios, the preliminary treatment facilities incorporate redundancy and safety features, described in Davis, 2010 and summarized in Table 8

3 PRIMARY TREATMENT

3.1 Facility Types

Primary treatment in this design is achieved using circular primary clarifiers. While both circular and rectangular clarifiers are widely used, circular tanks were selected due to their hydraulic efficiency, ease of sludge removal, and suitability for continuous operation in medium-to-large wastewater treatment plants. Circular clarifiers promote uniform radial flow distribution, reducing dead zones and short-circuiting, which enhances sedimentation performance [Metcalf and Eddy, 2014]. Mechanically, the rotating sludge scraper arms in circular clarifiers allow for reliable and consistent solids' removal with minimal manual intervention [Davis, 2010]. Additionally, circular tanks are preferred in many applications due to advantages like reduced maintenance needs, drive bearings positioned outside the wastewater, and typically lower construction expenses compared to rectangular designs [Davis, 2010]. Therefore, the decision to use this type of design is well-supported by its advantages.

Table 8: Operational resilience and contingency measures [Davis, 2010]

Aspect	Description
Redundancy Measures	An additional mechanically cleaned coarse screen and vortex grit chamber unit will be provided as backup units. This ensures operational continuity during periods when primary units are undergoing maintenance or experiencing overloading conditions.
Bypass Channels and Overflow	Dedicated bypass channels will be installed, allowing flows to bypass the primary screening and grit removal facilities. An overflow outlet will be provided for the equalization basin to prevent hydraulic overload during emergency situations.
Maintenance Accessibility	Screens, grit chambers, and the equalization basin are designed with ease of access in mind, facilitating rapid maintenance and emergency interventions.
Emergency Procedures and Alarms	An alarm system will trigger when blockage levels reach 50% or if there is a sudden halt in mechanical operation. This proactive alert mechanism enables prompt operator intervention, minimizing the risk of overflow or downstream process disruption.

3.2 Primary Sedimentation

3.2.1 Pollutant removal

To determine the volume of the primary clarifiers, the removal efficiency (E_r) required for biochemical oxygen demand (BOD) was calculated using Eq.(15), based on the inflow BOD and the effluent one that was found to comply with the environmental requirements, see Sections 4 and 5. The detention time required to achieve this efficiency is calculated using Eq.16 (where a , b are parameters defined for SST and BOD, see Table 9.) in conjunction with Eq.17. This last one is to increase the detention time when the temperature is lower than $20^\circ C$ as the water viscosity gets affected, see Appendix 4. For this report, a temperature T of $10^\circ C$ was chosen to incorporate this phenomenon, although further analysis could lead to adjust this value. The obtained detention time t_{BOD} is checked against the typical design values, which are: $1.5 \text{ h} < t < 2.5 \text{ h}$, see Appendix 9.

$$E_r = \frac{BOD_{5,\text{influent}} - BOD_{5,\text{effluent}}}{BOD_{5,\text{influent}}} \times 100 \quad (15)$$

$$t = \left(\frac{E_r \cdot a}{1 - E_r \cdot b} \right) \cdot T_{\text{factor}} \quad (16)$$

$$T_{\text{factor}} = \begin{cases} 1.82 \cdot e^{-0.03 \cdot T}, & \text{if } T < 20^\circ C \\ 1, & \text{if } T \geq 20^\circ C \end{cases} \quad (17)$$

As there is no environmental requirement for the TSS content on the waterway discharge, the required efficiency can not be calculated. Therefore, for the TSS removal the same detention time

Table 9: Standard parameters for detention time [Metcalf and Eddy, 2014].

Constituent	a	b
BOD_5	0.018	0.020
TSS	0.0075	0.014

as the BOD is used, and the subsequent efficiency is calculated from Eq.(15) with the corresponding parameters a and b for TSS . Input values and results are shown in Table 10.

The clarifier calculated volume ($V_{clarifier}^{calc}$) is determined using Eq.(18) replacing with the required detention time t_{BOD} shown in Table 10. However, the maximum volume $V_{clarifier}^{max}$ is defined with the smaller flow rate and the maximum detention time (see Eq.(19)), thus ensuring that for lower inflows the efficiency is still achieved as t_{BOD} is lower than t_{max} . As seen in Eq.(19), the minimum flow rate is defined as the current average flow rate. This allows that, the here proposed clarifiers design, will comply with the environmental and design requirements under both current and projected conditions.

If $V_{clarifier}^{calc}$ is greater than $V_{clarifier}^{max}$, then the excessive volume is used for the equalization basin ($V_{equilization}^{req}$) and the final clarifier volume ($V_{clarifier}^{final}$) will be equal to $V_{clarifier}^{max}$. Due to the equalization tank, there will be a new *adjusted* peak flow (Q_{peak}^{adj}), obtained with Eq.(21). Based on this, subsequent calculations will be based on this value instead of the Q_{peak} , when required. Input and result values are shown in Table 10.

$$V_{clarifier}^{calc} = Q_{peak}^{projected} \cdot t \cdot 3600 \quad (18)$$

$$V_{clarifiers}^{max} = Q_{avg}^{currents} \cdot t_{max} \cdot 3600 \quad (19)$$

$$V_{equilization}^{req} = V_{equilization}^{req} - V_{clarifiers}^{max} \quad (20)$$

$$Q_{peak}^{adj} = \frac{V_{clarifiers}^{max}}{t} \quad (21)$$

3.3 Design description

With the $V_{clarifier}^{max}$ obtained in the previous section, the sedimentation basin is dimensioned accordingly. Its height selected based on the typical design criteria, see Appendix 9. The 5 m side water depth height upper limit is selected, found after iterating the design that this height yields a clarifier area that complies with the SLR criteria. A freeboard distance of 0.5 m is added, thus $H_{clarifier} = 5.5m$. The area $A_{clarifier}$ is calculated using Eq.(22), the volume $V_{clarifier}^{final}$ used is equal to $V_{clarifier}^{max}$. The Surface Loading Rate (SLR) is calculated with Eq.(23), using both the Q_{avg} and Q_{peak}^{adj} . The clarifier diameter is then calculated using Eq.(24). The number of clarifiers n is selected equal as 3 to comply with the *Weir Loading Rate*, see next section. The clarifier volume $V_{clarifier}^{final}$ will be supplied in 3 equal volume tanks. Results are shown in Table 11, which are in compliance with the typical design criteria, see Appendix 9.

$$A_{clarifier} = \frac{V_{clarifier}^{final}}{n \cdot H} \quad (22)$$

$$SLR_{clarifier} = \frac{Q}{n \cdot A_{clarifier}} \cdot \frac{1}{3600 \cdot 24} \quad (23)$$

Table 10: Input values for clarifier sizing

Parameter	Value	Units	Description
$Q_{avg}^{current}$	3112.75	m ³ /h	Average projected flow rate, see Table 3
$Q_{peak}^{projected}$	8051.08	m ³ /h	Peak projected flow rate, see Table 3
$BOD_{5,influent}$	370.08	mg/L	Inflow BOD_5 , see Section 1.6
$BOD_{5,effluent}$	263.93	mg/L	Found BOD_5 that complies with environmental impact, see Section 4
T	10	C	Assumed the water lowest Temperature to adjust detention time.
E_{BOD}	28.68	%	BOD_5 Efficiency removal, obtained with Eq.(15)
t_{BOD}	1.632	h	Obtained with Eq.(16)
t_{TSS}	1.632	h	Same as t_{BOD}
E_{TSS}	50.0	%	Solved from Eq.(15), replacing t_{TSS}
$TSS_{effluent}$	185	mg/L	TSS effluent from primary treatment to secondary treatment
$V_{clarifiers}^{calc}$	13137.67	m ³	Calculated with Eq.(18)
$V_{clarifiers}^{max}$	7781.88	m ³	Calculated with Eq.(19)
$V_{equilization}^{req}$	5355.78	m ³	Calculated with Eq.(20)
Q_{peak}^{adj}	4768.93	m ³ /h	Adjusted peak flow rate calculated with Eq.(21)

$$D_{clarifier} = \sqrt{\frac{4V_{clarifier}^{final}}{\pi \cdot n \cdot H_{clarifier}}} \quad (24)$$

3.3.1 Weir Loading Rate (WLR) Assessment

The weir length for circular clarifiers with two-sided launders is given by Eq.(25), where d_{offset} is the distance from clarifier edge to centerline of the launder and $n_{launder-sides}$ is the number of launder sides. The Weir Length Ratio (WLR) is given by Eq.(26). The critical condition is then evaluated with Eq.(27), where n is the number of clarifier tanks. As explained in the previous section, this value (n) was set as 3 so $WLR_{critical}$, i.e. when one clarifier is out of service, does not exceed the design limit (500 m³/m/day).

$$L_w = n_{launder-sides} \cdot \pi \cdot (D_{clarifier} - 2 \cdot d_{offset}) \quad (25)$$

$$WLR = \frac{Q_{peak}}{L_w \cdot 86400} \quad (26)$$

$$WLR_{critical} = \frac{WLR}{n - 1} \quad (27)$$

$$(28)$$

The process abovementioned was iterated until achieving results that comply with the typical design criteria (Appendix 9), results are shown in the Table 11.

Table 11: Final clarifier design parameters

Parameter	Result	Units	Description
$H_{clarifier}$	5.5	m	5m height of water plus 0.5m freeboard
$A_{clarifier}$	518.79	m^2	Area per clarifier.
$SLR_{clarifier}^{avg}$	48.0	$m^3/m^2/day$	Surface Loading Rate under average flow rate.
$SLR_{clarifier}^{peak}$	73.54	$m^3/m^2/day$	Surface Loading Rate under peak flow rate.
$D_{clarifier}$	25.7	m	Diameter of clarifier tank.
n	3	–	Number of clarifier tanks.
$n_{launder-sides}$	2	–	Number of launder sides.
L_w	148.92	m	Weir length.
WLR	768.57	$m^3/m/day$	Weir Length Ratio.
$WLR_{critical}$	384.29	$m^3/m/day$	Weir Length Ratio under critical condition.
$V_{equalization}^{req}$	5355.78	m^3	Required equalization tank volume.

3.3.2 V-Notch and Launder Design

To ensure stable discharge from the primary clarifiers and prevent hydraulic overloading, a V-notch weir system was designed in accordance with standard hydraulic principles. The number of notches is determined based on the launder length per tank and a center-to-center spacing of 0.3 m between notches:

$$N_{notch} = \left\lceil \frac{L_{launder}}{s} \right\rceil \quad (29)$$

where $s = 0.3$ m is the typical spacing [Metcalf and Eddy, 2014]. The total number of V-notches in the system is then:

$$N_{notch, total} = N_{notch} \times N_{clarifiers} \quad (30)$$

The flow per notch is calculated based on the design peak hourly flow:

$$Q_{notch} = \frac{Q_{peak}^{adj}}{N_{notch, total}} \quad (31)$$

The maximum height of water over each V-notch is determined using the general formula for triangular weirs:

$$h = \left(\frac{Q_{notch}}{\frac{8}{15} C_d \sqrt{2g} \tan(\theta/2)} \right)^{2/5} \quad (32)$$

where $C_d = 0.58$, $g = 9.81 \text{ m/s}^2$, and $\theta = 90^\circ$. A 15% safety margin is applied to determine the actual design depth above the notch:

$$y = 1.15 \cdot h \quad (33)$$

The top width of the V-notch opening is twice the water depth:

$$w_{\text{notch}} = 2y \quad (34)$$

3.3.3 Launder Flow Conditions

To assess flow behavior in the launder channel, the discharge per unit length of the launder is:

$$q = WLR_{\text{critical}} \cdot \frac{1}{3600 \cdot 24} \cdot n_{\text{launder-sides}} \quad (35)$$

The maximum water depth in the launder is calculated from the critical section at the discharge point:

$$Y_c = \left(\frac{q^2 \cdot L_{\text{launder}}^2}{4gw_{\text{notch}}^2} \right)^{1/3} \quad (36)$$

The perimeter of a clarifier with a 2-sided launder can be determined by dividing the total weir length (L_w) by the number of launder sides:

$$P_{\text{clarifier}} = \frac{L_w}{n_{\text{launder-sides}}} \quad (37)$$

The farthest distance that water travels into the launder, denoted as x , is half the perimeter divided by the number of discharge points:

$$x = \frac{P_{\text{clarifier}}}{2 \cdot n_{\text{disch}}} \quad (38)$$

The final maximum water depth is obtained using the equation for uniform flow depth in a launder with one outlet point ($n_{\text{disch}} = 1$) at distance x :

$$H = \sqrt{Y_c^2 + \frac{2q^2x^2}{gw^2Y_c}} \quad (39)$$

A 50% safety factor is applied to this maximum depth for design purposes:

$$H_{\text{final}} = 1.5 \cdot H \quad (40)$$

In the Table 12 the results of V-Notch and Launder design are presented.

Table 12: V-notch and launder design summary

Parameter	Result	Units	Description
$N_{\text{notch/clarifier}}$	497.0	–	Number of notches per clarifier.
$N_{\text{notch, total}}$	1491.0	–	Total number of notches across all clarifiers.
Q_{notch}	0.00089	m ³ /s	Flow per individual V-notch.
h	0.05306	m	Maximum height of water over the notch.
y	0.06102	m	Water depth over the notch including 15% safety factor.
w_{notch}	0.122	m	Top width of the V-notch.
q	0.0089	m ³ /m/s	Discharge per unit length of launder.
Y_c	0.355	m	Critical height at launder discharge point.
H	0.615	m	Maximum calculated water depth in launder.
H_{final}	0.922	m	Final design depth in launder with 50% safety factor.

3.3.4 Inlet design

An inlet is proposed to act as energy dissipator (EDI), used within the feed well to distribute flow (influent) and enhance mixing, further promoting flocculation, see Appendix 7 - Figure 9. Its detention time is selected as 20 minutes. Their diameter was defined as a 10 percent of the tank diameter (max. 13%), promoting a detention time between 8 and 10 seconds [Metcalf and Eddy, 2014]. Additionally, the author mentions that if the inlet assembly is quite large, it can reduce the effective flocculation volume and increase downward velocities, potentially hindering performance. Furthermore, the height of the feed well was defined as 1.25m, which is 25% of the water depth (5m), see Appendix 7 - Figure 10.

3.4 Protection and future considerations

The proposed design includes several features to ensure reliability and performance of the circular primary clarifiers under normal and extreme operating conditions:

- **Redundancy:** The system comprises three circular tanks, allowing independent operation. This configuration prevents complete shutdown during maintenance or failure of any single unit [Davis, 2010].
- **Hydraulic Load Management:** The tanks are designed to handle peak flows up to 2.2 times the average flow rate, thus accommodating diurnal fluctuations and surges from return streams without compromising downstream processes [Davis, 2010].
- **Overflow Rates:** The surface loading rate (SLR) is kept within recommended values to ensure efficient sedimentation and avoid solids washout. This includes the capacity to operate effectively under peak conditions at least twice the average flow [Davis, 2010, Metcalf and Eddy, 2014].
- **Adequate Detention Time:** The clarifiers are designed to maintain detention times between 1.5 and 2.5 hours, which is sufficient for effective settling while minimizing the risk of anaerobic conditions that could result in sludge lifting [Davis, 2010].

- **Velocity Control:** Flow velocities within the tanks are limited to reduce turbulence and prevent resuspension of settled solids [Davis, 2010].
- **Effective Flow Distribution:** Inlet structures and baffles are incorporated ahead of effluent weirs to ensure uniform flow distribution, minimize short-circuiting, and prevent scum from escaping with the effluent [Metcalf and Eddy, 2014].
- **Robust Sludge Removal:** The tank bottom is sloped at 1:12 (vertical to horizontal), forming an inverted cone. Sludge scrapers effectively transport settled solids to a central hopper for collection and removal [Metcalf and Eddy, 2014].
- **Launder protection:** Maintenance cost and algal growth are both controlled by supplying a cover surface over the weirs [Metcalf and Eddy, 2014].

4 SECONDARY TREATMENT

4.1 Facility type

The Complete-Mix Activated Sludge (CMAS) process is selected for the secondary treatment stage, following guidelines from the Water Environment Federation [Water Environment Federation, 2017]. CMAS provides robust and consistent performance under fluctuating loads and conditions. Despite the process's susceptibility to filamentous microbial growth, careful selection of operational parameters addresses this potential issue.

Circular secondary clarifiers are proposed because their radial symmetry flow minimizes short-circuiting. The inlet structure needs to have a good energy dissipation and symmetrical flow, therefore a center-feed column with energy-dissipating inlet and flocculation baffle is recommended [Davis, 2010].

Secondary treatment produces excess biomass that must be removed from the secondary clarifier and conveyed for further processing or disposal. A centrifugal non-clog pump is recommended: it delivers dilute sludge and fine solids handling without blockage, operates at high hydraulic efficiency, and requires minimal maintenance. Compared to positive-displacement alternatives, it also offers a lower cost and strong lifecycle value [Davis, 2010].

4.2 Pollutant removal

The CMAS process presents efficiencies that can vary from 85% up to 95%, see Appendix 10. Through iterations along with the primary treatment processes, the optimal efficiency was selected as 93.2 % BOD_5 removal. As shown in Section 5, in order to comply with the minimum DO in the waterway, the maximum BOD present in the effluent must be equal to 18 mg/L. Based on that and the removal efficiency, the inflow BOD concentration can be calculated as:

$$BOD_{in} = \frac{BOD_{effluent}}{1 - \eta} \quad (41)$$

where η is the BOD removal efficiency. Computed, the BOD_{in} is equal to 263.9 mg/L, and it represents the target for the removal in the primary treatment, see Section 3. The typical design parameters for activated sludge processes are shown in Appendix 12.

As shown in Table 10, the TSS received from the primary treatment is equal to 185 mg/L. A

well-operated secondary treatment process can yield from 4 to 10 mg/L *TSS* effluent concentration [Metcalf and Eddy, 2014]. Conservatively, the upper limit value is chosen, and the *TSS* removal efficiency is therefore obtained as 95%.

4.3 Design description

4.3.1 Sludge Characteristics and Return Flow

As the wastewater temperature highly impacts the activated sludge performance, a value of 10°C is used to represent critical weather conditions in Melbourne. A Mixed Liquor Suspended Solids (*MLSS*) concentration of 4000 mg/L; and the Suspended Volume Index (*SVI*) is adopted as 91 mg/L, which promotes good sludge settling characteristics. It also fosters adequate flocculation and settling as it maintains approximately 50% filamentous microorganism content [Chudoba et al., 1973]. Both selected values of *MLSS* and *SVI* are appropriate for conventional activated sludge processes at low temperatures, according to Metcalf & Eddy (2014, Table 8-19) The return sludge concentration X_R is computed, and the return sludge flow Q_R is determined using mass balance:

$$X_R = \frac{10^6}{SVI} \quad (42)$$

$$Q_R = Q_{\text{peak}}^{\text{adj}} \cdot \frac{V_{\text{settled}}}{1000 - V_{\text{settled}}} \quad (43)$$

$$V_{\text{settled}} = SVI \cdot MLSS / 1000 \quad (44)$$

$$RSR = \frac{Q_R}{Q_{\text{peak}}^{\text{adj}}} \quad (45)$$

The resulting return sludge ratio (*RSR*) is verified within the recommended range (0.25–1.0) for CMAS processes [Metcalf and Eddy, 2014].

4.3.2 Aeration Tank Sizing

Selecting the upper limit of the Food-to-Microorganism (*F/M*) ratio (0.4 day^{-1} , Metcalf & Eddy, 2014) minimizes the required aeration tank volume. The aeration volume (V_a) and hydraulic residence time (*HRT*) are calculated using:

$$V_a = \frac{BOD_{5,\text{in}} \cdot Q_{\text{peak}}^{\text{adj}}}{F/M \cdot X_R} \quad (46)$$

$$HRT = \frac{V_a}{Q} \cdot 24 \quad (47)$$

The calculated *HRT* is within the recommended range (3–6 hours) for CMAS processes, confirming adequate contact time and biological stabilization.

4.3.3 Organic Loading Rate

The organic loading rate, an essential performance parameter, is computed as follows:

$$OLR = \frac{BOD_{5,in} \cdot Q_{peak}^{adjusted}}{V_a} \quad (48)$$

4.3.4 Sludge Age and Stability

Typically, kinetic coefficients for activated sludge processes are reported at 20 °C. To adapt these to our chosen design temperature (10 °C), the Arrhenius equation is employed to adjust the yield coefficient (Y) and decay coefficient (K_d):

$$U = F/M \cdot \eta \quad (49)$$

$$\theta_c = \frac{1}{Y \cdot U - K_d} \quad (50)$$

Using these adjusted coefficients, the required sludge age is calculated. A safety factor of 1.3 is applied to accommodate peak organic load variations:

$$\theta_{c,adj} = 1.3 \cdot \theta_c \quad (51)$$

The waste flow (Q_w) is calculated with the $MLSS$, V_a , X_R and θ_c as shown below. Similarly, the adjusted waste flow ($Q_{w,adj}$) rate is calculated, but using $\theta_{c,adj}$ instead.

$$Q_w = \frac{MLSS \cdot V_a}{X_R \cdot \theta_c} \quad (52)$$

$$Q_{w,adj} = \frac{MLSS \cdot V_a}{X_R \cdot \theta_{c,adj}} \quad (53)$$

Finally, the base effluent flow (Q_e) and adjusted one ($Q_{e,adj}$) are obtained as:

$$Q_e = Q_{peak}^{adjusted} - Q_w \quad (54)$$

$$Q_{e,adj} = Q_{peak}^{adjusted} - Q_{w,adj} \quad (55)$$

4.3.5 Aeration Tank Configuration

Aeration basins are sized to avoid large volumes that promote dead zones, potentially exacerbating filamentous microbial growth [Chudoba et al., 1973]. Thus, individual basin volume is limited to 3,500 m^3 , and the length-to-width ratio is capped at 3:1, consistent with Davis (2010). The number of aeration tanks required is therefore calculated as:

$$N_{tanks}^{aeration} = \frac{V_a}{3500} \quad (56)$$

rounding the result to the next integer, ensuring therefore a volume per tank below the abovementioned limit. The volume per tank is then recalculated as:

$$V_{\text{tank}}^{\text{aeration}} = \frac{V_a}{N_{\text{tanks}}^{\text{aeration}}} \quad (57)$$

The aeration tank dimensions are then selected to yield the obtained $V_{\text{tank}}^{\text{aeration}}$. The tank depth was selected as 4.2 m, as values within 3 to 6 m are recommended [Metcalf and Eddy, 2014]; the tank width was estimated as four times the depth, i.e. 16.8 m: and finally the length is calculated to reach the desired volume: $L_{\text{tank}}^{\text{aeration}} = V_{\text{tank}}^{\text{aeration}} / (\text{Width} \cdot \text{Depth}) = 45 \text{ m}$.

A **Solids Loading Ratio** (SLR_{peak}) during peak flow conditions is selected as $6 \text{ kg/m}^2/\text{day}$, as this value is recommended to within 4 to $6 \text{ kg/m}^2/\text{day}$. The upper limit was selected as after the equalization basin, the adjusted peak flow is close to the average flow.

The area of the secondary clarifier is given by the equation below.

$$A_{\text{clarifier}} = \frac{(Q_{\text{peak}}^{\text{adj}} + RSR \cdot Q_{\text{peak}}^{\text{adj}}) \cdot MLSS}{SLR_{\text{peak}} \cdot 24 \cdot 1000} \quad (58)$$

Finally, the SLR_{avg} is calculated as shown below

$$SLR_{\text{avg}} = \frac{(Q_{\text{peak}}^{\text{adj}} + RSR \cdot Q_{\text{peak}}^{\text{adj}}) \cdot MLSS}{A_{\text{clarifier}} \cdot 24 \cdot 1000} \quad (59)$$

4.3.6 Aeration requirements

Aeration equipment selection requires evaluating both peak hourly and daily average oxygen demands. Peak oxygen demand determines aerator power, while daily average demand informs energy consumption. Oxygen demands and biomass production are calculated using the observed yield coefficient (Y_{obs}) and the biomass generation ratio (P_x) for both average and adjusted peak conditions:

$$Y_{\text{obs,avg}} = \frac{Y}{1 + K_d \cdot \theta_c} \quad (60)$$

$$P_{x,\text{avg}} = Y_{\text{obs,avg}} \cdot Q_{\text{avg}}^{\text{projected}} \cdot \frac{BOD_{\text{in}} - BOD_{\text{eff}}}{1000} \quad (61)$$

$$Y_{\text{obs,peak}} = \frac{Y}{1 + K_d \cdot \theta_{c,\text{adj}}} \quad (62)$$

$$P_{x,\text{peak}} = Y_{\text{obs,avg}} \cdot Q_{\text{peak}}^{\text{adj}} \cdot \frac{BOD_{\text{in}} - BOD_{\text{eff}}}{1000} \quad (63)$$

To determine blower capacity during peak demand, the oxygen demand for the activated sludge process is first estimated by accounting for both the oxidation of BOD and the synthesis of biomass. The fraction of BOD oxidized is calculated using a first-order decay model. The total oxygen requirement is then adjusted by subtracting the oxygen used for biomass synthesis. Finally, the volume of air required is calculated by incorporating a safety factor of 2 and assuming a standard oxygen transfer efficiency of 10% (based on a transfer capacity of $0.279 \text{ kg } O_2 \text{ per } m^3 \text{ of air}$). This process ensures reliable aeration system sizing under peak loading conditions. The complete formulation is given in Equations (64)–(68).

$$f = 1 - e^{-5k} \quad (64)$$

$$O_{2,\text{peak}} = \frac{Q_{\text{peak}}^{\text{adj}} \cdot (BOD_{\text{in}} - BOD_{\text{eff}})}{f \cdot 1000} - 1.42 \cdot P_{x,\text{peak}} \quad (65)$$

$$O_{2,\text{avg}} = \frac{Q_{\text{avg}} \cdot (BOD_{\text{in}} - BOD_{\text{eff}})}{f \cdot 1000} - 1.42 \cdot P_{x,\text{avg}} \quad (66)$$

$$V_{\text{air,peak}}^{\text{hourly}} = \frac{O_{2,\text{peak}}}{0.279 \cdot 0.1} \cdot \frac{2}{24} \quad (67)$$

$$V_{\text{air,avg}}^{\text{daily}} = \frac{O_{2,\text{avg}}}{0.279 \cdot 0.1} \quad (68)$$

4.3.7 Design Summary

Table 13 and Table 14 summarize the design inputs and resulting key parameters for the secondary treatment system.

Table 13: Design input values for secondary treatment

Parameter	Value	Units	Description
$Q_{\text{peak}}^{\text{adj}}$	114454.3	m ³ /d	Adjusted projected peak flow, see Section 3
Q_{avg}	108323.8	m ³ /d	Average projected flow rate, see Table 3
$BOD_{5,\text{eff}}$	18	mg/L	Target BOD ₅ for waterway effluent, see Section 5
η_{BOD}	93.2	%	Selected BOD ₅ removal efficiency
$MLSS$	4000	mg/L	Selected Mixed Liquor Suspended Solids (MLSS)
SVI	91	mL/g	Selected Suspended Volume Index
Y	0.45	SS/mg	Yield coefficient of BOD ₅
K_d	0.04	day ⁻¹	Decay coefficient
k	0.2	day ⁻¹	Conversion rate constant
F/M	0.4	–	Selected Food to Microorganism ratio

4.4 Protection against failure

The process design follows a duty/stand-by (N+1) philosophy for all blowers and return-sludge pumps, providing high resilience. If one secondary clarifier is unavailable for maintenance, the remaining unit will sustain the full peak hydraulic and solids load. Likewise, the six complete-mix aeration tanks allow any single basin to be isolated for maintenance while the plant continues to meet permit limits under normal operating conditions.

5 ENVIRONMENTAL IMPACT ASSESSMENT

5.1 Impact on receiving waterway

To evaluate the impact of the effluent discharge on the receiving waterway, the *Streeter-Phelps model* was applied. This model predicts the variation in dissolved oxygen (DO) downstream of the discharge point

Table 14: Key results for secondary treatment design

Parameter	Value	Units	Description
$BOD_{5,in}$	263.9	mg/L	BOD ₅ at secondary influent
X_R	10989.0	mg/L	Return sludge concentration
$V_{settled}$	364.0	mL/L	Volume of settled sludge
Q_R	65505.3	m ³ /d	Return sludge flow
V_a	18880.0	m ³	Aeration volume, see Eq. (46)
HRT	4.0	h	Hydraulic Residence Time, see Eq. (47)
OLR	1600.0	g/m ³ /d	Organic loading rate, see Eq (48)
U	0.4	–	Specific Utilization Ratio, see Eq. (49)
θ_c	7.9	d	Sludge age, see Eq. (50)
$\theta_{c,adj}$	10.3	d	Sludge age adjusted for peak loading, see Eq. (51)
Q_w	868.8	m ³ /d	Waste flow, see Eq. (52)
$Q_{w,adj}$	668.5	m ³ /d	Waste flow adjusted, see Eq. (53)
Q_e	113585.5	m ³ /d	Effluent flow, see Eq. (54)
$Q_{e,adj}$	113785.8	m ³ /d	Effluent flow adjusted, see Eq. (55)
$N_{aeration\ tanks}$	6	–	Number of aeration tanks, see Eq. (56)
$V_{aeration\ tank}$	3175.2	m ³	Aeration tank to construct volume, see Section 4.3.5 for tank's dimensions
$A_{clarifier}$	4991.5	m ²	Total clarifiers area, see Eq. (58)
SLR_{avg}	5.8	kg/m ² /day	Solids loading ratio average flow, see Eq. (59)
$Y_{obs,avg}$	0.34	–	Observed yield for average flow rate, see Eq. (60)
$P_{X,avg}$	9057.6	kg/d	Biomass production for average flow, see Eq. (61)
$Y_{obs,peak}$	0.319	–	Observed yield for peak flow rate, see Eq. (62)
$P_{X,peak}$	8975.7	kg/d	Biomass production for peak flow, see Eq. (63)
$O_{2,peak}$	31783.6	kg/d	Oxygen demand for peak flow, see Eq. (65)
$O_{2,avg}$	29398.5	kg/d	Oxygen demand for average flow, see Eq. (66)
Air_{peak}	94933.1	m ³ /h	Required air for peak conditions, see Eq. (67)
Air_{avg}	1053708.6	m ³ /d	Required air for average conditions, see Eq. (68)

based on oxygen consumption due to biochemical oxygen demand (BOD) and oxygen replenishment through atmospheric re-aeration.

According to previous surveys and analysis, the receiving waterway has a base flow that complies with: $Q_{waterway} = 300,000 \cdot (10)^{0.2} = 475468 \text{ m}^3/\text{day}$. The water quality upstream of the discharge point is characterized by a background BOD of 4 mg/L (L_r), a dissolved oxygen saturation of 8.5 mg/L ($DO_{saturation}$), and an initial oxygen deficit of 0.8 mg/L (DO_0). Regulatory guidelines for this region require that DO levels remain above 70% saturation [Yarra Valley Water, 2023], yielding a minimum acceptable DO of 5.95 mg/L (Eq (69)). Additionally, the re-aeration rate constant (k_r), oxygenation rate constant (k_d), BOD reaction rate (k_1) and stream velocity (v) are known due to previous assessments. In the Table 15 the input values for the *Streeter-Phelps model* are shown.

The effluent discharge was evaluated based on the project adjusted peak flow rate (114454.28 m³/day), calculated in Section 3 and a effluent BOD₅ of 18 mg/L (L_w). This value was estimated through iterating the calculations explained in this section until achieving a $DO_{calculated}$ that complies with the $DO_{min,threshold}$. Thus, this value was set as the target for the upstream processes, see Sections 3 and 4.

Table 15: Input values for Streeter-Phelps DO sag model

Parameter	Value	Description
$DO_{\text{saturation}}$	8.5 mg/L	DO at saturation
DO_0	0.8 mg/L	Initial oxygen deficit
$DO_{\text{min, threshold}}$	5.95 mg/L	Minimum DO required
L_r	4 mg/L	BOD upstream
L_w	18 mg/L	BOD ₅ of effluent
k_r	0.71 day ⁻¹	Re-aeration rate constant
k_d	0.51 day ⁻¹	De-oxygenation rate constant
k_1	0.23 day ⁻¹	BOD reaction rate
v	0.07 m/s	Stream velocity
Q_w	475468 m ³ /day	Waterway flow rate
Q_e	114454.28 m ³ /day	Adjusted flow rate

From this, the ultimate BOD (L_u) is estimated using Eq. (70) with the time t equals to 5 days. Using Eq. (71) the mixed BOD in the receiving stream is determined, and with Eq. (71) mixed conditions downstream are calculated. The model then determines the time (t_c) at which the minimum DO level occurs using Eq. (72). Finally, the maximum deficit $D(t_c)$ is calculated using Eq. (74). The distance downstream at which the maximum deficit/minimum saturation is reached can also be calculated using Eq. (73) Results are shown in Table 17.

$$DO_{\text{min, threshold}} = 0.7 \cdot DO_{\text{saturation}} \quad (69)$$

$$L_u = \frac{L_w}{1 - e^{-k_1 t}} \quad (70)$$

$$L_0 = \frac{Q_w \cdot L_r + Q_e \cdot L_u}{Q_w + Q_e} \quad (71)$$

$$t_c = \frac{1}{k_r - k_d} \ln \left[\frac{k_r}{k_d} \left(1 - D_0 \cdot \frac{k_r - k_d}{k_d L_0} \right) \right] \quad (72)$$

$$x_c = t_c \cdot 86400 \cdot v \quad (73)$$

$$D(t_c) = \frac{k_d L_0}{k_r - k_d} \left(e^{-k_d t_c} - e^{-k_r t_c} \right) + D_0 e^{-k_r t_c} \quad (74)$$

$$DO_{\text{calculated}} = DO_{\text{saturation}} - D(t_c) \quad (75)$$

Table 16: Environmental impact model results

Output	Value	Description
L_u	26.34 mg/L	Ultimate BOD of effluent
L_0	7.33 mg/L	Mixed BOD in receiving stream
t_c	1.44 days	Time to reach minimum DO
x_c	8682.24 m	Distance to minimum DO level
$D(t_c)$	2.53 mg/L	DO deficit at time t_c
$DO_{\text{calculated}}$	5.97 mg/L	DO in waterway when maximum deficit $D(t_c)$

As shown in Table 17, the dissolved oxygen deficit at its critical point is 2.53 mg/L. Subtracting from the saturation $DO_{\text{saturation}}$ the critical deficit $D(t_c)$ the $DO_{\text{calculated}}$ is obtained, equal to 5.97 mg/L which

is above the regulatory $DO_{\min, \text{threshold}}$ of 5.95 mg/L. Therefore, the discharge complies with the receiving waterway's environmental standard. In the Figure 11, Appendix 8, the deficit variation across time can be observed, obtained base on Eq. (74) varying t . The distance downstream at which the waterway reaches the initial DO is 32718.83 m.

5.2 Biosolids management

The solids balance indicates 20 t d^{-1} of primary sludge ($\approx 285 \text{ m}^3 d^{-1}$ at 7 % TS) and 28 t d^{-1} of secondary sludge ($1,375 \text{ m}^3 d^{-1}$ at 2 % TS), with primary rising to 25 t d^{-1} at peak. Gravity or DAF thickening to 4-6 % TS trims volumes by 60 % [Metcalf and Eddy, 2014]. The mixed sludge is then anaerobically digested, producing Class B biosolids and biogas for plant energy [Water Environment Federation, 2017]. Centrifuge dewatering to 18-22 % TS cuts haulage to 250 $\text{m}^3 d^{-1}$ [United States Environmental Protection Agency, 1993]. The resulting cake can be land-applied under EPA Victoria nutrient and metal caps [United States Environmental Protection Agency, 1993], composted to Class A, or thermally dried/incinerated with energy recovery and ash reuse in cement or bricks. This treatment train minimizes transport, recovers resources and energy, complies with regulations, and keeps disposal options adaptable to changing markets and policies.

5.3 Carbon emissions

Aeration is the main operational carbon hot-spot in municipal plants, accounting for roughly 50-60 % of total power demand [Metcalf and Eddy, 2014]. Our design tempers that load by installing a 23000 m^3 equalization basin that flattens flow peaks, thus reducing total aeration requirements. Pumps for RAS, WAS and sludge are the next largest electrical consumer. Hence, a gravity-stepped hydraulic profile is suggested and high-efficiency non-clog centrifugal trim total dynamic head, cutting pumping energy by roughly a quarter.

6 SUMMARY

This wastewater treatment plant is designed to serve a growing region in northwest Melbourne, with an average daily flow of approximately 108,000 m^3 and a peak capacity of 193,000 m^3 . The treatment process consists of preliminary screening using mechanical coarse screens and vortex grit chambers to remove large debris and grit, followed by primary treatment with three circular clarifiers that achieve substantial removal of suspended solids and organic matter. Secondary treatment is provided by a Complete-Mix Activated Sludge (CMAS) process, utilizing six aeration tanks and secondary clarifiers to remove the remaining organic and particulate pollutants. The design incorporates an equalization basin to regulate flow and enhance treatment performance, along with robust redundancy across all critical units to ensure operational resilience.

The environmental impact assessment confirms that the effluent meets regulatory requirements, with a final BOD concentration of 18 mg/L. Using the Streeter-Phelps model, the treated discharge is shown to maintain downstream dissolved oxygen levels above the threshold of 5.95 mg/L, reaching a minimum of 5.97 mg/L and returning to saturation within 32.7 km of the outfall. Biosolids generated from the process, including primary and secondary sludge, are thickened, anaerobically digested to recover biogas, and dewatered for beneficial reuse such as land application or composting. The plant design also considers carbon emissions, with energy-efficient equipment and biogas recovery systems incorporated to minimize the environmental footprint while maintaining compliance with long-term sustainability goals.

In the Figure 1, a conceptual diagram of the layout of the wastewater treatment plant designed in the present document is presented.

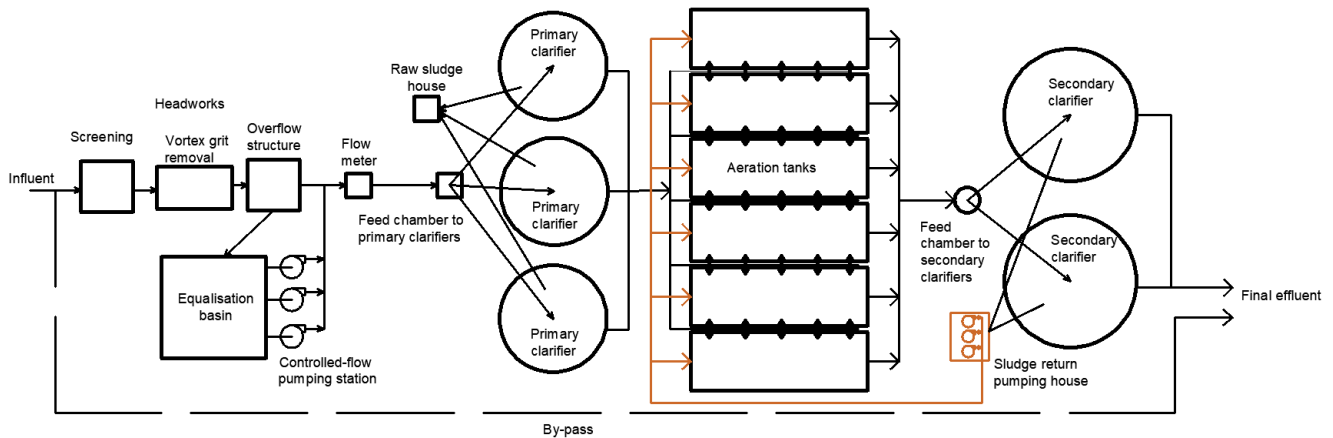


Figure 1: Wastewater treatment plan conceptual diagram, by Authors.

Table 17: Environmental impact model results

Process	BOD mg/L	$BOD_{efficiency}\%$	TSS mg/L	$TSS_{efficiency}\%$
Catchment	370	—	370	—
Primary treatment	264	29	185	50
Secondary treatment	18	93.2	10	95

REFERENCES

- [Australian Bureau of Statistics, 2021] Australian Bureau of Statistics (2021). Suburbs and localities.
- [Chudoba et al., 1973] Chudoba, J., Ottova, V., and Madera, V. (1973). Control of activated sludge filamentous bulking—i. effect of the hydraulic regime or degree of mixing in an aeration tank.
- [Davis, 2010] Davis, M. L. (2010). *Water and Wastewater Engineering: Design Principles and Practice*. McGraw-Hill Companies, Inc., New York.
- [Melbourne Water, 2024a] Melbourne Water (2024a). Customer outcomes performance report 2023–24. <https://www.melbournewater.com.au>. Accessed: 2025-05-15.
- [Melbourne Water, 2024b] Melbourne Water (2024b). Eastern treatment plant daily influent quality data. <https://data-melbournewater.opendata.arcgis.com>. Accessed: 2025-05-07.
- [Metcalf and Eddy, 2014] Metcalf and Eddy (2014). *Wastewater Engineering, Treatment and resource recovery*. McGraw, 5 edition.
- [United States Environmental Protection Agency, 1993] United States Environmental Protection Agency (1993). Standards for the use or disposal of sewage sludge. Code of Federal Regulations, Title 40, Part 503. Federal Register, Vol. 58, No. 32, pp. 9248-9415, 19 February 1993.
- [Water Environment Federation, 2017] Water Environment Federation (2017). *Design of Water Resource Recovery Facilities*. McGraw-Hill Education, New York, NY, 6 edition.
- [Yarra Valley Water, 2023] Yarra Valley Water (2023). Amf I8 sewerage planning and design principles.

APPENDIX 1 PEAKING FACTORS

According to previous studies, the peaking factors of the wastewater catchment for the understudy suburb are shown in the table below.

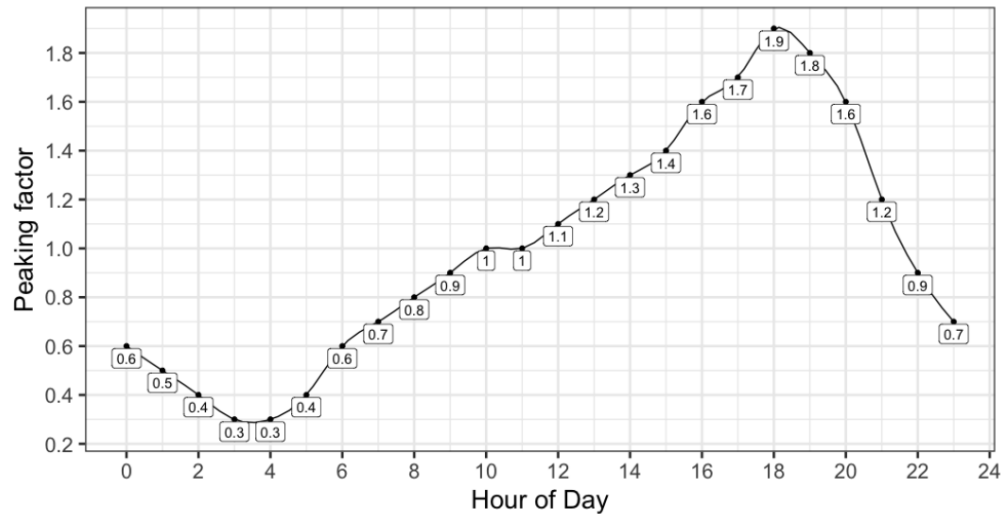


Figure 2: Daily fluctuation of peaking factors at each hour of the day for flow rates on an average day in a similar wastewater catchment.

APPENDIX 2 AREA AND POPULATION PER SUBURB

Table of Melbourne suburbs with their area and population [Australian Bureau of Statistics, 2021].

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Southbank	11962.280000	22631	Melbourne (City)	1.890000
Carlton	10597.480000	16055	Melbourne (City)	1.510000
Fitzroy	7355.630000	10431	Yarra (City)	1.420000
Melbourne	7269.020000	16055	Melbourne (City)	2.210000
South Yarra	7087.660000	25028	Melbourne (City)	3.530000
Balaclava	7062.830000	5392	Port Phillip (City)	0.760000
Prahran	6975.820000	12203	Stonnington (City)	1.750000
Windsor	6729.210000	7273	Port Phillip (City)	1.080000
Collingwood	6719.020000	9179	Yarra (City)	1.370000
St Kilda	6365.640000	19490	Port Phillip (City)	3.060000
North Melbourne	6325.150000	14953	Melbourne (City)	2.360000
Richmond	6252.540000	28587	Yarra (City)	4.570000
Elwood	5989.600000	15153	Port Phillip (City)	2.530000
St Kilda West	5977.320000	2951	Port Phillip (City)	0.490000
Travancore	5933.010000	2116	Moonee Valley (City)	0.360000
St Kilda East	5784.110000	12571	Glen Eira (City)	2.170000
Glen Huntly	5637.580000	4905	Glen Eira (City)	0.870000
Seddon	5586.700000	5143	Maribyrnong (City)	0.920000
Ripponlea	5510.490000	1532	Port Phillip (City)	0.280000
Kingsville	5442.760000	3920	Maribyrnong (City)	0.720000
Brunswick East	5097.030000	13279	Moreland (City)	2.610000
Kensington	5073.670000	10745	Melbourne (City)	2.120000
Fitzroy North	5007.710000	12781	Moreland (City)	2.550000
South Melbourne	4986.300000	11548	Port Phillip (City)	2.320000
Brunswick	4910.310000	24896	Moreland (City)	5.070000
Princes Hill	4909.930000	2005	Yarra (City)	0.410000
Middle Park	4885.610000	4000	Port Phillip (City)	0.820000
Carnegie	4716.030000	17909	Glen Eira (City)	3.800000
Abbotsford	4700.750000	9088	Yarra (City)	1.930000
Brunswick West	4357.960000	14746	Moreland (City)	3.380000
Armada	4201.390000	9368	Stonnington (City)	2.230000
Hawthorn East	4110.510000	14834	Boroondara (City)	3.610000
Northcote	4087.370000	25276	Darebin (City)	6.180000
Essendon North	4080.600000	3071	Moonee Valley (City)	0.750000
Ormond	4064.220000	8328	Glen Eira (City)	2.050000
Hawthorn	4000.510000	22322	Boroondara (City)	5.580000
Elsternwick	3975.800000	10887	Glen Eira (City)	2.740000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
McKinnon	3897.170000	6878	Glen Eira (City)	1.760000
Gardenvale	3884.170000	1019	Glen Eira (City)	0.260000
Ascot Vale	3845.150000	15197	Moonee Valley (City)	3.950000
Caulfield	3806.120000	5748	Glen Eira (City)	1.510000
Coburg	3785.050000	26574	Moreland (City)	7.020000
Murrumbeena	3771.280000	9996	Glen Eira (City)	2.650000
Hughesdale	3759.200000	7563	Monash (City)	2.010000
Clifton Hill	3646.350000	6606	Yarra (City)	1.810000
Caulfield North	3633.750000	16903	Glen Eira (City)	4.650000
Noble Park	3624.230000	32257	Greater Dandenong (City)	8.900000
Caulfield South	3621.750000	12328	Glen Eira (City)	3.400000
Thornbury	3617.380000	19005	Darebin (City)	5.250000
Docklands	3482.850000	15495	Melbourne (City)	4.450000
Malvern	3453.170000	9929	Stonnington (City)	2.880000
Bentleigh	3426.600000	17921	Glen Eira (City)	5.230000
Pascoe Vale	3423.210000	18171	Moreland (City)	5.310000
Carlton North	3367.180000	6177	Melbourne (City)	1.830000
Essendon	3344.050000	21240	Moonee Valley (City)	6.350000
Pascoe Vale South	3309.990000	10534	Moreland (City)	3.180000
Hampton East	3295.150000	5069	Bayside (City)	1.540000
Parkdale	3292.040000	12308	Kingston (City)	3.740000
Moonee Ponds	3272.100000	16224	Moonee Valley (City)	4.960000
Taylors Hill	3254.310000	15419	Melton (City)	4.740000
Kings Park	3242.880000	8203	Brimbank (City)	2.530000
Footscray	3236.630000	17131	Maribyrnong (City)	5.290000
Box Hill	3235.380000	14353	Whitehorse (City)	4.440000
Meadow Heights	3209.080000	14890	Hume (City)	4.640000
Roxburgh Park	3171.080000	24129	Hume (City)	7.610000
Seabrook	3162.960000	4952	Hobsons Bay (City)	1.570000
Oakleigh East	3131.200000	6804	Monash (City)	2.170000
Hampton	3120.720000	13518	Bayside (City)	4.330000
Niddrie	3083.290000	5901	Moonee Valley (City)	1.910000
Glen Iris	3069.860000	26131	Boroondara (City)	8.510000
Surrey Hills	3069.720000	13655	Boroondara (City)	4.450000
Bentleigh East	3057.980000	30159	Glen Eira (City)	9.860000
West Footscray	3056.590000	11729	Maribyrnong (City)	3.840000
Oak Park	3053.640000	6714	Moreland (City)	2.200000
Sydenham	3044.380000	10578	Brimbank (City)	3.470000
Burnside Heights	3043.610000	6377	Melton (City)	2.100000
Balwyn	3026.140000	13495	Boroondara (City)	4.460000
Toorak	3021.060000	12817	Stonnington (City)	4.240000
Blackburn South	3000.560000	10939	Whitehorse (City)	3.650000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Box Hill North	2993.190000	12337	Whitehorse (City)	4.120000
Cremorne	2967.650000	2158	Yarra (City)	0.730000
Kingsbury	2964.710000	3460	Darebin (City)	1.170000
Chelsea	2961.140000	8347	Kingston (City)	2.820000
Albanvale	2947.400000	5641	Brimbank (City)	1.910000
Lalor	2924.790000	23219	Whittlesea (City)	7.940000
Heidelberg Heights	2916.630000	6758	Banyule (City)	2.320000
Camberwell	2909.610000	21965	Boroondara (City)	7.550000
Caroline Springs	2904.020000	24488	Melton (City)	8.430000
St Albans	2899.360000	38042	Brimbank (City)	13.120000
Preston	2864.580000	33790	Darebin (City)	11.800000
Mentone	2863.300000	13197	Kingston (City)	4.610000
Maidstone	2858.230000	9389	Maribyrnong (City)	3.280000
Highett	2857.850000	12016	Bayside (City)	4.200000
Mont Albert	2843.710000	4948	Boroondara (City)	1.740000
Brighton East	2823.010000	16757	Bayside (City)	5.940000
Edithvale	2811.620000	6276	Kingston (City)	2.230000
Malvern East	2811.060000	22296	Stonnington (City)	7.930000
Dallas	2808.250000	6762	Hume (City)	2.410000
Springvale South	2782.910000	12766	Greater Dandenong (City)	4.590000
Brighton	2778.140000	23252	Bayside (City)	8.370000
Chadstone	2769.550000	9552	Monash (City)	3.450000
Sandringham	2769.330000	10926	Bayside (City)	3.950000
Blackburn North	2744.920000	7627	Whitehorse (City)	2.780000
Ashwood	2705.700000	7154	Monash (City)	2.640000
Flemington	2687.670000	15197	Melbourne (City)	5.650000
Fawkner	2685.090000	14274	Hume (City)	5.320000
East Melbourne	2674.570000	4896	Melbourne (City)	1.830000
South Kingsville	2663.470000	2156	Hobsons Bay (City)	0.810000
Reservoir (Vic.)	2661.150000	51096	Darebin (City)	19.200000
Ashburton	2651.730000	7952	Boroondara (City)	3.000000
Gowanbrae	2645.990000	2971	Moreland (City)	1.120000
Yarraville	2644.460000	15636	Maribyrnong (City)	5.910000
Dandenong	2636.980000	30127	Greater Dandenong (City)	11.420000
Watsonia North	2630.340000	3799	Banyule (City)	1.440000
Forest Hill	2610.810000	10780	Whitehorse (City)	4.130000
Beaumaris	2588.020000	13947	Bayside (City)	5.390000
Newport	2580.100000	13658	Hobsons Bay (City)	5.290000
Canterbury	2573.800000	7800	Boroondara (City)	3.030000
Carrum	2559.490000	4239	Kingston (City)	1.660000
Doncaster East	2553.110000	30926	Manningham (City)	12.110000
Mont Albert North	2542.980000	5609	Whitehorse (City)	2.210000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Rosanna	2518.670000	8616	Banyule (City)	3.420000
Aberfeldie	2510.950000	3925	Moonee Valley (City)	1.560000
Williamstown	2504.300000	14407	Hobsons Bay (City)	5.750000
Clayton	2475.130000	18988	Monash (City)	7.670000
Burwood	2469.010000	15147	Monash (City)	6.130000
Strathmore	2438.170000	8980	Moonee Valley (City)	3.680000
Mitcham	2436.700000	16795	Whitehorse (City)	6.890000
Box Hill South	2416.620000	8491	Whitehorse (City)	3.510000
Glenroy	2411.640000	23792	Moreland (City)	9.870000
Burwood East	2409.810000	10675	Whitehorse (City)	4.430000
Glen Waverley	2394.710000	42642	Monash (City)	17.810000
Delahey	2378.490000	8077	Brimbank (City)	3.400000
Dandenong North	2371.500000	22550	Greater Dandenong (City)	9.510000
Doncaster	2366.780000	11219	Manningham (City)	4.740000
Blackburn	2363.110000	14478	Whitehorse (City)	6.130000
Briar Hill	2354.000000	3220	Banyule (City)	1.370000
Kew	2346.690000	24499	Boroondara (City)	10.440000
Nunawading	2338.260000	12413	Manningham (City)	5.310000
Montmorency	2338.200000	9250	Banyule (City)	3.960000
Huntingdale	2333.330000	1949	Monash (City)	0.840000
Deepdene	2325.710000	2101	Boroondara (City)	0.900000
Doveton	2297.010000	9603	Casey (City)	4.180000
Bonbeach	2293.070000	6855	Kingston (City)	2.990000
Aspendale	2291.190000	7285	Kingston (City)	3.180000
Narre Warren South	2290.990000	30909	Casey (City)	13.490000
Templestowe Lower	2285.400000	14098	Manningham (City)	6.170000
Mill Park	2273.840000	28712	Whittlesea (City)	12.630000
Oakleigh	2270.060000	8442	Monash (City)	3.720000
Watsonia	2265.970000	5352	Banyule (City)	2.360000
Vermont	2265.080000	10993	Maroondah (City)	4.850000
Balwyn North	2244.140000	21302	Boroondara (City)	9.490000
Maribyrnong	2223.920000	12573	Maribyrnong (City)	5.650000
Mount Waverley	2213.430000	35340	Monash (City)	15.970000
Ivanhoe	2196.930000	13374	Banyule (City)	6.090000
Ringwood East	2194.780000	10764	Maroondah (City)	4.900000
Cairnlea	2191.780000	10038	Brimbank (City)	4.580000
Albert Park	2189.150000	6044	Port Phillip (City)	2.760000
Heathmont	2171.220000	9933	Maroondah (City)	4.570000
Hoppers Crossing	2159.050000	37216	Wyndham (City)	17.240000
Gladstone Park	2157.870000	8213	Hume (City)	3.810000
Avondale Heights	2150.670000	12388	Moonee Valley (City)	5.760000
Macleod	2137.170000	9892	Banyule (City)	4.630000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Croydon Hills	2136.070000	4839	Maroondah (City)	2.270000
Clarinda	2131.950000	7441	Kingston (City)	3.490000
Braybrook	2126.010000	9682	Maribyrnong (City)	4.550000
Heidelberg	2121.680000	7360	Banyule (City)	3.470000
Deer Park	2113.570000	18145	Brimbank (City)	8.590000
Cheltenham	2107.700000	13947	Bayside (City)	6.620000
Lynbrook	2093.120000	9121	Casey (City)	4.360000
Hillside	2078.730000	17331	Brimbank (City)	8.340000
Taylors Lakes	2075.840000	15174	Brimbank (City)	7.310000
Sunshine	2064.240000	9445	Brimbank (City)	4.580000
Airport West	2058.230000	8173	Moonee Valley (City)	3.970000
Keilor Lodge	2037.210000	1668	Brimbank (City)	0.820000
Seaholme	2035.900000	2067	Hobsons Bay (City)	1.020000
Greensborough	2030.520000	21070	Banyule (City)	10.380000
Aspendale Gardens	2015.430000	6427	Kingston (City)	3.190000
Sunshine West	1995.490000	18552	Brimbank (City)	9.300000
Keilor Downs	1992.620000	9857	Brimbank (City)	4.950000
Noble Park North	1991.470000	7436	Greater Dandenong (City)	3.730000
Cranbourne North	1964.060000	24683	Casey (City)	12.570000
Croydon North	1961.570000	8092	Maroondah (City)	4.130000
Springvale	1958.690000	22174	Greater Dandenong (City)	11.320000
Boronia	1957.230000	23607	Knox (City)	12.060000
Jacana	1934.550000	2187	Hume (City)	1.130000
Black Rock	1933.750000	6389	Bayside (City)	3.300000
Bellfield	1925.890000	1996	Banyule (City)	1.040000
Notting Hill	1918.240000	2895	Monash (City)	1.510000
Hampton Park	1913.510000	26082	Casey (City)	13.630000
Croydon	1911.060000	28608	Maroondah (City)	14.970000
Albion	1905.360000	4334	Brimbank (City)	2.270000
Wheelers Hill	1892.230000	20652	Monash (City)	10.910000
Vermont South	1888.420000	11954	Whitehorse (City)	6.330000
Ferntree Gully	1887.580000	27398	Knox (City)	14.510000
Ringwood North	1880.280000	9964	Manningham (City)	5.300000
Burnside	1861.680000	5800	Melton (City)	3.120000
Fairfield	1853.590000	6535	Darebin (City)	3.530000
Mordialloc	1852.960000	8886	Kingston (City)	4.800000
Narre Warren	1849.840000	27689	Casey (City)	14.970000
Hadfield	1843.580000	6269	Moreland (City)	3.400000
Parkville	1829.830000	7074	Melbourne (City)	3.870000
Frankston	1825.020000	37331	Frankston (City)	20.460000
Altona Meadows	1817.840000	18479	Hobsons Bay (City)	10.170000
Ivanhoe East	1817.530000	3762	Banyule (City)	2.070000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Mulgrave	1806.880000	19889	Monash (City)	11.010000
Alphington	1803.340000	5702	Darebin (City)	3.160000
St Helena	1783.400000	2890	Banyule (City)	1.620000
Patterson Lakes	1771.840000	7793	Kingston (City)	4.400000
Croydon South	1763.890000	4759	Maroondah (City)	2.700000
Mooroolbark	1757.080000	23059	Yarra Ranges (Shire)	13.120000
Eaglemont	1754.870000	3960	Banyule (City)	2.260000
Eumemmerring	1739.290000	2285	Casey (City)	1.310000
Williams Landing	1733.890000	9448	Wyndham (City)	5.450000
Ringwood	1726.210000	19144	Maroondah (City)	11.090000
Heidelberg West	1725.800000	5252	Banyule (City)	3.040000
Eltham North	1693.630000	6830	Banyule (City)	4.030000
Port Melbourne	1690.180000	17633	Melbourne (City)	10.430000
Wantirna	1672.880000	14237	Knox (City)	8.510000
Chelsea Heights	1669.270000	5393	Kingston (City)	3.230000
Bulleen	1669.170000	11219	Manningham (City)	6.720000
Bundoora	1662.880000	28068	Banyule (City)	16.880000
Kew East	1632.230000	6620	Boroondara (City)	4.060000
Keilor East	1630.420000	15078	Brimbank (City)	9.250000
Warranwood	1603.940000	4820	Maroondah (City)	3.010000
Clayton South	1592.590000	13381	Kingston (City)	8.400000
Coburg North	1588.840000	8327	Moreland (City)	5.240000
Sandhurst	1569.310000	5211	Frankston (City)	3.320000
Endeavour Hills	1568.470000	24455	Casey (City)	15.590000
Rowville	1532.710000	7645	Knox (City)	4.990000
Viewbank	1522.880000	7030	Banyule (City)	4.620000
Broadmeadows	1503.580000	12524	Hume (City)	8.330000
Bayswater	1483.470000	12262	Knox (City)	8.270000
Sunshine North	1458.130000	12047	Brimbank (City)	8.260000
Ardeer	1456.120000	3170	Brimbank (City)	2.180000
Strathmore Heights	1452.970000	1047	Moonee Valley (City)	0.720000
Waterways	1451.920000	2422	Kingston (City)	1.670000
Berwick	1438.260000	50298	Casey (City)	34.970000
Craigieburn	1420.390000	65178	Hume (City)	45.890000
Yallambie	1403.680000	4161	Banyule (City)	2.960000
Essendon West	1402.530000	1559	Moonee Valley (City)	1.110000
Oakleigh South	1392.210000	9851	Kingston (City)	7.080000
Thomastown	1391.960000	20234	Whittlesea (City)	14.540000
Wantirna South	1376.500000	20754	Knox (City)	15.080000
Knoxfield	1365.420000	7645	Knox (City)	5.600000
Bayswater North	1337.340000	9014	Maroondah (City)	6.740000
Seaford	1327.980000	17215	Frankston (City)	12.960000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km ²)
Cranbourne West	1317.470000	19969	Casey (City)	15.160000
Hallam	1315.080000	11355	Casey (City)	8.630000
Laverton	1307.180000	4760	Hobsons Bay (City)	3.640000
Moorabbin	1286.280000	6287	Kingston (City)	4.890000
Dingley Village	1264.710000	10495	Kingston (City)	8.300000
Caulfield East	1258.140000	1293	Glen Eira (City)	1.030000
Frankston South	1249.930000	18801	Frankston (City)	15.040000
Kealba	1240.870000	3226	Brimbank (City)	2.600000
Cranbourne	1216.930000	21281	Casey (City)	17.490000
Keysborough	1187.480000	30018	Greater Dandenong (City)	25.280000
Kilsyth	1167.310000	11699	Maroondah (City)	10.020000
Westmeadows	1166.570000	6502	Hume (City)	5.570000
Cranbourne East	1164.270000	11177	Casey (City)	9.600000
Tecoma	1147.110000	2064	Yarra Ranges (Shire)	1.800000
Mornington	1137.300000	25759	Mornington Peninsula (Shire)	22.650000
Frankston North	1125.830000	5711	Frankston (City)	5.070000
South Morang	1121.470000	24989	Whittlesea (City)	22.280000
Point Cook	1109.730000	66781	Wyndham (City)	60.180000
Eltham	1104.120000	18847	Banyule (City)	17.070000
Upwey	1069.970000	6818	Yarra Ranges (Shire)	6.370000
Donvale	1056.110000	12644	Manningham (City)	11.970000
Templestowe	1031.020000	16966	Manningham (City)	16.460000
Carrum Downs	1019.840000	21976	Frankston (City)	21.550000
Upper Ferntree Gully	1019.090000	3417	Knox (City)	3.350000
Coolaroo	1017.540000	3193	Hume (City)	3.140000
Kurunjang	971.630000	10711	Melton (City)	11.020000
Belgrave	963.700000	3894	Yarra Ranges (Shire)	4.040000
Werribee	940.770000	50027	Wyndham (City)	53.180000
Rosebud	939.680000	14381	Mornington Peninsula (Shire)	15.300000
Epping	918.380000	33489	Whittlesea (City)	36.470000
Tarneit	905.020000	56370	Wyndham (City)	62.290000
Rosebud West	902.270000	5246	Mornington Peninsula (Shire)	5.810000
Kilsyth South	891.140000	2862	Maroondah (City)	3.210000
Williamstown North	887.470000	1622	Hobsons Bay (City)	1.830000
Tullamarine	881.840000	6733	Brimbank (City)	7.640000
Altona North	871.490000	12962	Hobsons Bay (City)	14.870000
Junction Village	856.780000	1051	Casey (City)	1.230000
Brookfield	839.190000	10782	Melton (City)	12.850000
West Melbourne	837.890000	8025	Melbourne (City)	9.580000
Spotswood	823.270000	2820	Hobsons Bay (City)	3.430000
Tootgarook	822.530000	3178	Mornington Peninsula (Shire)	3.860000
Keilor Park	811.640000	2684	Brimbank (City)	3.310000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Safety Beach	810.660000	6328	Mornington Peninsula (Shire)	7.810000
Langwarrin	783.080000	23588	Frankston (City)	30.120000
Mount Eliza	770.770000	18734	Mornington Peninsula (Shire)	24.310000
The Basin	767.470000	4497	Knox (City)	5.860000
Lyndhurst	722.110000	8926	Casey (City)	12.360000
Scoresby	692.980000	6066	Knox (City)	8.750000
Mernda	690.840000	23369	Whittlesea (City)	33.830000
Attwood	675.830000	3309	Hume (City)	4.900000
Melton West	671.540000	12463	Melton (City)	18.560000
Doreen	660.690000	27122	Nillumbik (Shire)	41.050000
McCrae	658.040000	3311	Mornington Peninsula (Shire)	5.030000
Diamond Creek	657.830000	12503	Nillumbik (Shire)	19.010000
Altona	643.010000	11490	Hobsons Bay (City)	17.870000
Botanic Ridge	635.890000	6739	Casey (City)	10.600000
Derrimut	634.610000	8651	Brimbank (City)	13.630000
Beaconsfield	620.920000	7267	Cardinia (Shire)	11.700000
Mount Martha	613.990000	19846	Mornington Peninsula (Shire)	32.320000
Montrose	612.960000	6900	Yarra Ranges (Shire)	11.260000
Bacchus Marsh	583.980000	7808	Moorabool (Shire)	13.370000
Belgrave Heights	577.250000	1398	Yarra Ranges (Shire)	2.420000
Mount Evelyn	571.080000	9799	Yarra Ranges (Shire)	17.160000
Lower Plenty	570.360000	3962	Banyule (City)	6.950000
Lilydale	566.630000	17348	Yarra Ranges (Shire)	30.620000
Wyndham Vale	552.790000	12675	Wyndham (City)	22.930000
Keilor	546.140000	5906	Brimbank (City)	10.810000
Pakenham	543.480000	54118	Cardinia (Shire)	99.580000
Greenvale	506.040000	21274	Hume (City)	42.040000
Skye	486.480000	8088	Frankston (City)	16.630000
Crib Point	484.990000	3343	Mornington Peninsula (Shire)	6.890000
Rye	472.170000	9438	Mornington Peninsula (Shire)	19.990000
Chirnside Park	444.640000	11779	Yarra Ranges (Shire)	26.490000
Truganina	442.160000	36305	Melton (City)	82.110000
Campbellfield	412.840000	4977	Hume (City)	12.060000
Heatherton	408.460000	2826	Kingston (City)	6.920000
Park Orchards	403.380000	3835	Manningham (City)	9.510000
Melton	396.060000	7953	Melton (City)	20.080000
Melton South	377.720000	3601	Melton (City)	9.530000
Hastings	370.920000	10369	Mornington Peninsula (Shire)	27.950000
Blairgowrie	367.960000	2786	Mornington Peninsula (Shire)	7.570000
Narre Warren North	348.470000	8033	Casey (City)	23.050000
Brooklyn	339.000000	1979	Brimbank (City)	5.840000
North Warrandyte	337.210000	3027	Nillumbik (Shire)	8.980000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Ferny Creek	336.210000	1524	Yarra Ranges (Shire)	4.530000
Warrandyte	317.250000	5541	Manningham (City)	17.470000
Darley	310.710000	9190	Moorabool (Shire)	29.580000
Somerville	293.290000	11767	Mornington Peninsula (Shire)	40.120000
Clyde North	277.570000	31681	Casey (City)	114.140000
Research	275.650000	2695	Nillumbik (Shire)	9.780000
Sunbury	272.900000	38851	Hume (City)	142.360000
The Patch	267.860000	1046	Yarra Ranges (Shire)	3.910000
Selby	251.910000	1626	Yarra Ranges (Shire)	6.450000
Officer	250.410000	18503	Cardinia (Shire)	73.890000
Hurstbridge	232.450000	3554	Nillumbik (Shire)	15.290000
Sorrento	226.970000	2013	Mornington Peninsula (Shire)	8.870000
Wattle Glen	218.160000	1911	Nillumbik (Shire)	8.760000
Baxter	218.050000	2166	Mornington Peninsula (Shire)	9.930000
Plenty	215.400000	2575	Nillumbik (Shire)	11.950000
Bittern	215.210000	4276	Mornington Peninsula (Shire)	19.870000
Lysterfield	210.960000	6681	Knox (City)	31.670000
Blind Bight	207.220000	1290	Casey (City)	6.230000
Dromana	191.550000	6626	Mornington Peninsula (Shire)	34.590000
Wandin North	183.530000	3132	Yarra Ranges (Shire)	17.070000
Monbulk	179.880000	3651	Yarra Ranges (Shire)	20.300000
Millgrove	176.530000	1666	Yarra Ranges (Shire)	9.440000
Badger Creek	174.150000	1610	Yarra Ranges (Shire)	9.240000
Wollert	168.110000	24407	Whittlesea (City)	145.180000
Wonga Park	163.500000	3843	Manningham (City)	23.500000
Wallan	159.090000	15004	Mitchell (Shire)	94.310000
Mount Dandenong	157.640000	1271	Yarra Ranges (Shire)	8.060000
Cockatoo	150.830000	4408	Cardinia (Shire)	29.220000
Kalorama	146.350000	1277	Yarra Ranges (Shire)	8.730000
Yarra Junction	145.510000	2875	Yarra Ranges (Shire)	19.760000
Langwarrin South	142.220000	1346	Frankston (City)	9.460000
Tyabb	141.340000	3449	Mornington Peninsula (Shire)	24.400000
Maddingley	135.080000	5491	Moorabool (Shire)	40.650000
Belgrave South	129.550000	1670	Yarra Ranges (Shire)	12.890000
Kallista	119.220000	1418	Yarra Ranges (Shire)	11.890000
Emerald	114.590000	5890	Cardinia (Shire)	51.400000
Pearcedale	113.460000	3867	Casey (City)	34.080000
Somers	111.800000	1857	Mornington Peninsula (Shire)	16.610000
Seville	111.240000	2559	Yarra Ranges (Shire)	23.000000
Woori Yallock	107.680000	2964	Yarra Ranges (Shire)	27.530000
Yarrambat	103.490000	1602	Nillumbik (Shire)	15.480000
Bunyip	101.830000	3131	Cardinia (Shire)	30.750000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Beaconsfield Upper	100.400000	2997	Cardinia (Shire)	29.850000
Olinda	97.900000	1773	Yarra Ranges (Shire)	18.110000
Gisborne	93.100000	10142	Macedon Ranges (Shire)	108.940000
New Gisborne	91.020000	2509	Macedon Ranges (Shire)	27.570000
Ravenhall	84.760000	2295	Melton (City)	27.080000
Cranbourne South	83.710000	3241	Casey (City)	38.720000
Balnarring	82.660000	2371	Mornington Peninsula (Shire)	28.680000
Yarra Glen	81.560000	3012	Yarra Ranges (Shire)	36.930000
Garfield	67.600000	2114	Cardinia (Shire)	31.270000
Harkaway	67.530000	1011	Casey (City)	14.970000
Hmas Cerberus	67.520000	1124	Mornington Peninsula (Shire)	16.650000
Launching Place	66.280000	2495	Yarra Ranges (Shire)	37.640000
Devon Meadows	65.540000	1551	Casey (City)	23.660000
Koo Wee Rup	60.680000	4047	Cardinia (Shire)	66.690000
Werribee South	58.490000	2392	Wyndham (City)	40.900000
Romsey	58.430000	5797	Macedon Ranges (Shire)	99.210000
Mickleham	57.410000	17452	Hume (City)	303.990000
Tooradin	56.550000	1722	Cardinia (Shire)	30.450000
Riddells Creek	54.370000	4390	Macedon Ranges (Shire)	80.740000
Clyde	53.650000	11177	Casey (City)	208.330000
Whittlesea	52.810000	6117	Whittlesea (City)	115.830000
Healesville	52.640000	7589	Yarra Ranges (Shire)	144.170000
Macedon	52.340000	2073	Macedon Ranges (Shire)	39.610000
Panton Hill	50.360000	1063	Nillumbik (Shire)	21.110000
Eden Park	49.230000	1194	Whittlesea (City)	24.250000
Eynesbury	42.290000	2838	Melton (City)	67.110000
Silvan	41.290000	1323	Yarra Ranges (Shire)	32.040000
Diggers Rest	40.780000	5669	Hume (City)	139.010000
Wandong	39.560000	1477	Mitchell (Shire)	37.340000
Mount Macedon	39.250000	1450	Macedon Ranges (Shire)	36.940000
Red Hill	39.170000	1009	Mornington Peninsula (Shire)	25.760000
Coldstream	38.910000	2199	Yarra Ranges (Shire)	56.520000
Pakenham Upper	36.610000	1196	Cardinia (Shire)	32.670000
Rockbank	36.570000	7982	Melton (City)	218.270000
Kangaroo Ground	34.850000	1208	Nillumbik (Shire)	34.660000
St Andrews	30.810000	1186	Nillumbik (Shire)	38.490000
Moorooduc	25.610000	1004	Mornington Peninsula (Shire)	39.200000
Wesburn	24.690000	1052	Yarra Ranges (Shire)	42.610000
Beveridge	20.860000	4642	Whittlesea (City)	222.530000
Lancefield	19.830000	2743	Macedon Ranges (Shire)	138.330000
Nar Nar Goon	19.470000	1023	Cardinia (Shire)	52.540000
Kinglake West	17.860000	1305	Murrindindi (Shire)	73.070000

Continued on next page

Table 18: Melbourne suburbs with area and population

Suburb	Population Density	Population	LGA	Area (km2)
Flinders	16.420000	1130	Mornington Peninsula (Shire)	68.820000
Warburton	15.180000	2020	Yarra Ranges (Shire)	133.070000
Kalkallo	14.950000	5548	Hume (City)	371.100000
Kinglake	13.340000	1662	Murrindindi (Shire)	124.590000
Gembrook	11.610000	2559	Cardinia (Shire)	220.410000

From the above data, the following box plot can be obtained.

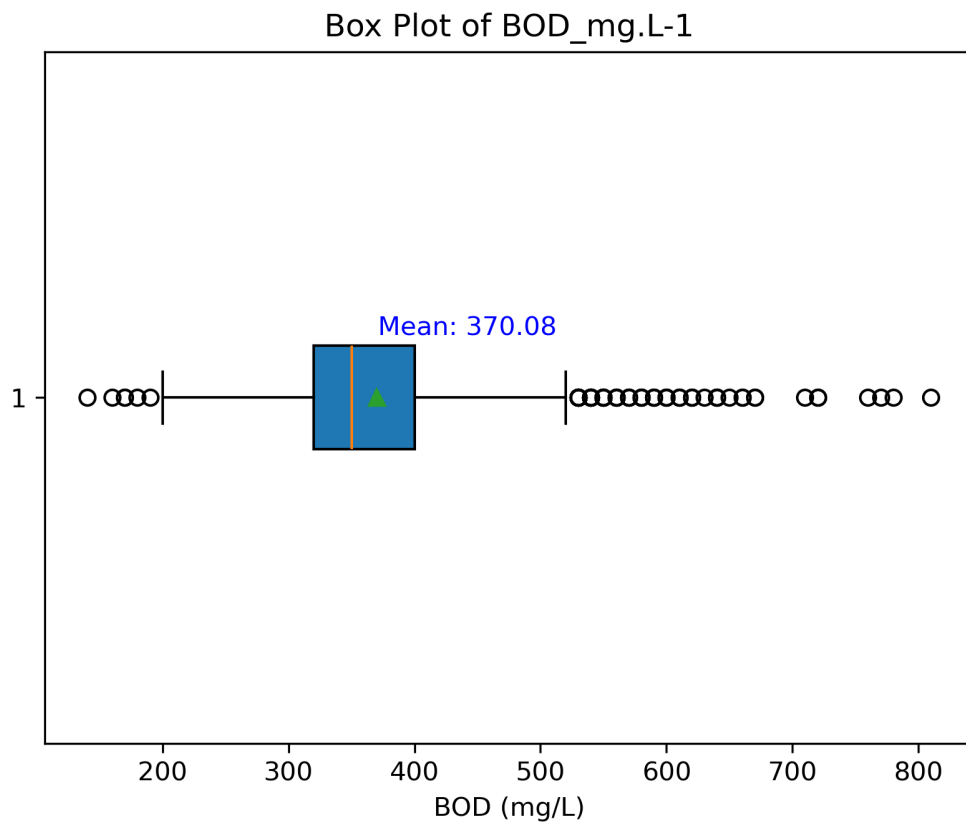


Figure 3: Boxplots of Biochemical Oxygen Demand (BOD) from Melbourne Water ETP dataset

APPENDIX 3 PRELIMINARY TREATMENT, SCREENS SECTION

Below it is shown a sketch of elevation of the screens and the hydraulic control provided in the preliminary treatment. The above section represents the results based on the projected peak flow rate while the below one the results with the current average flow rate.

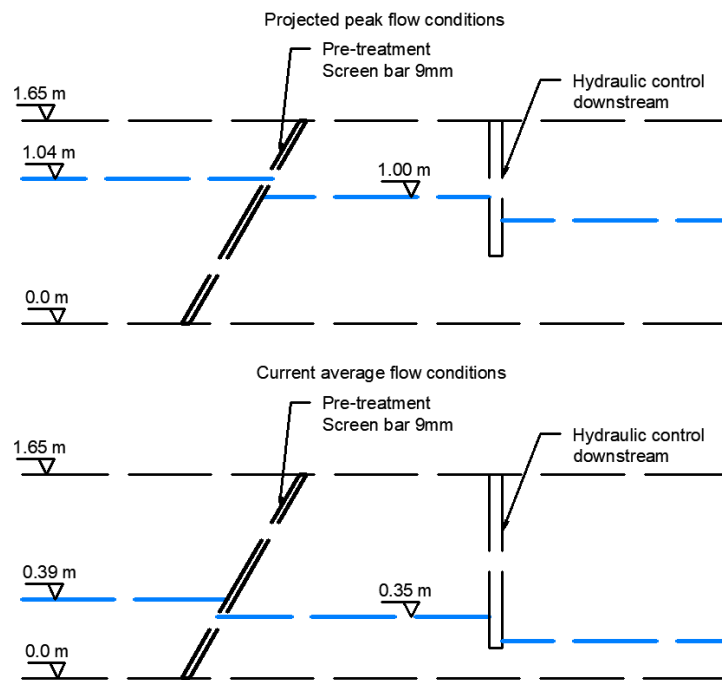


Figure 4: Preliminary treatment, screens section.

APPENDIX 4 TEMPERATURE MULTIPLIER FOR DETENTION TIME

Curve of the Temperature factor (T_{factor}) to increase detention time in sedimentation when wastewater is at a lower temperature than 20°C [Metcalf and Eddy, 2014].

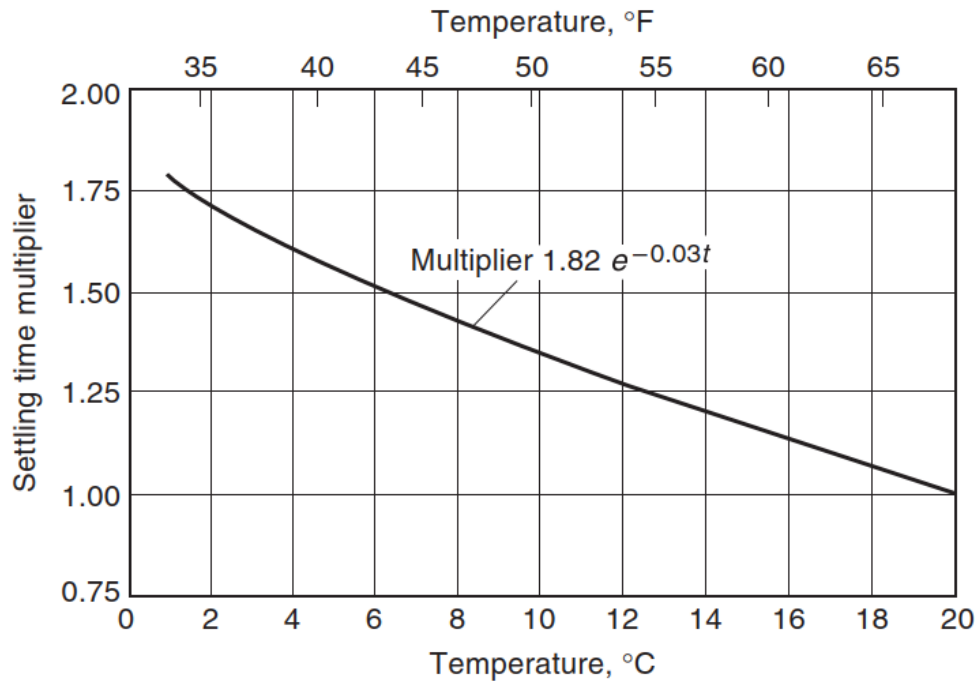


Figure 5: Temperature multiplier for detention time [Metcalf and Eddy, 2014].

APPENDIX 5 MASS BALANCE TECHNIQUE FOR EQUILIZATION VOLUME

The required volume to ensure a constant flow downstream of the equilization basin during current conditions was obtained using the mass balance technique, ensuring that the inflow surplus during daily function of the plant is balanced with the deficit. In the Figure 6 and in the Figure 7 the projected condition is represented showing the *adjusted peak flow*.

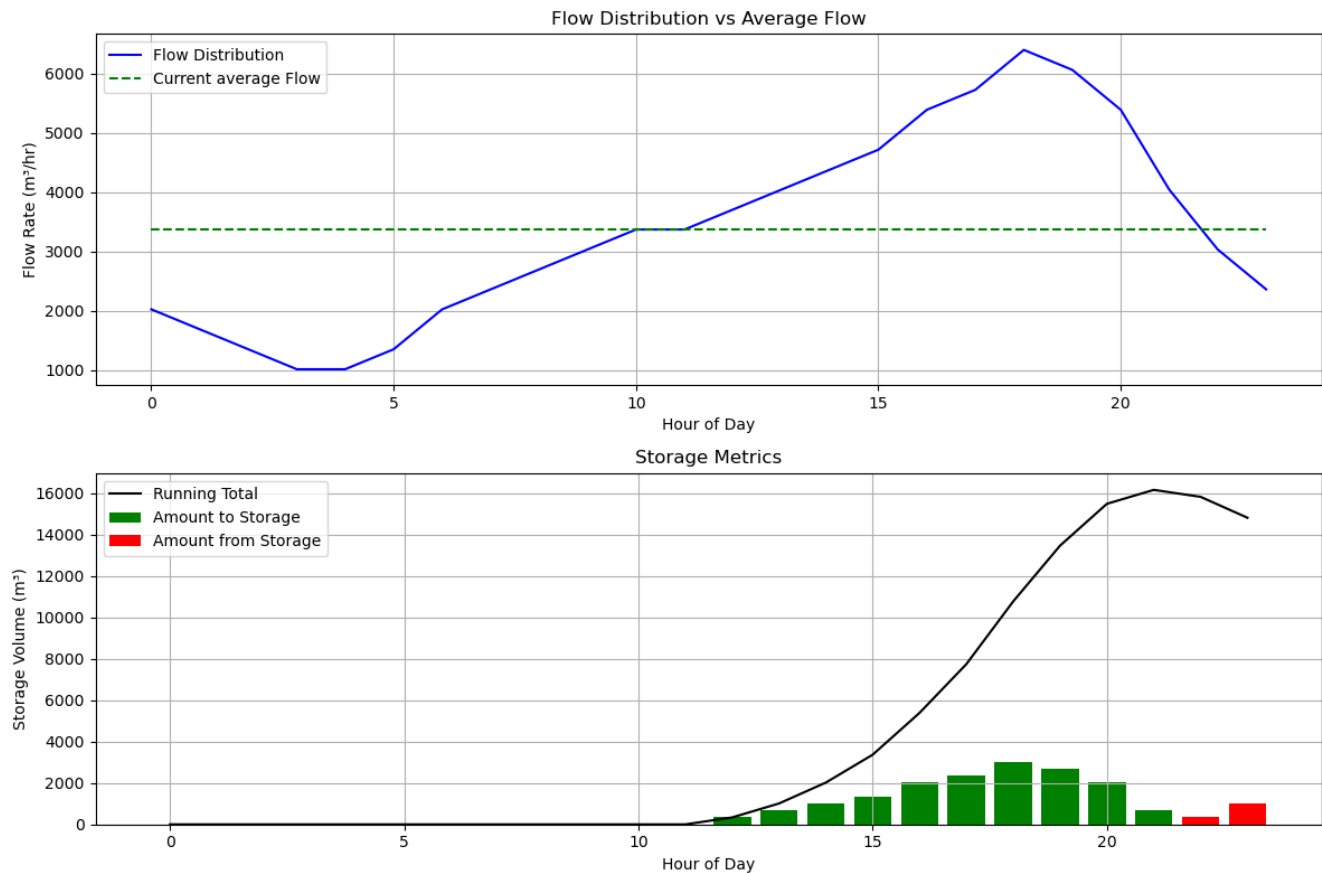


Figure 6: Mass balance diagram for current conditions.

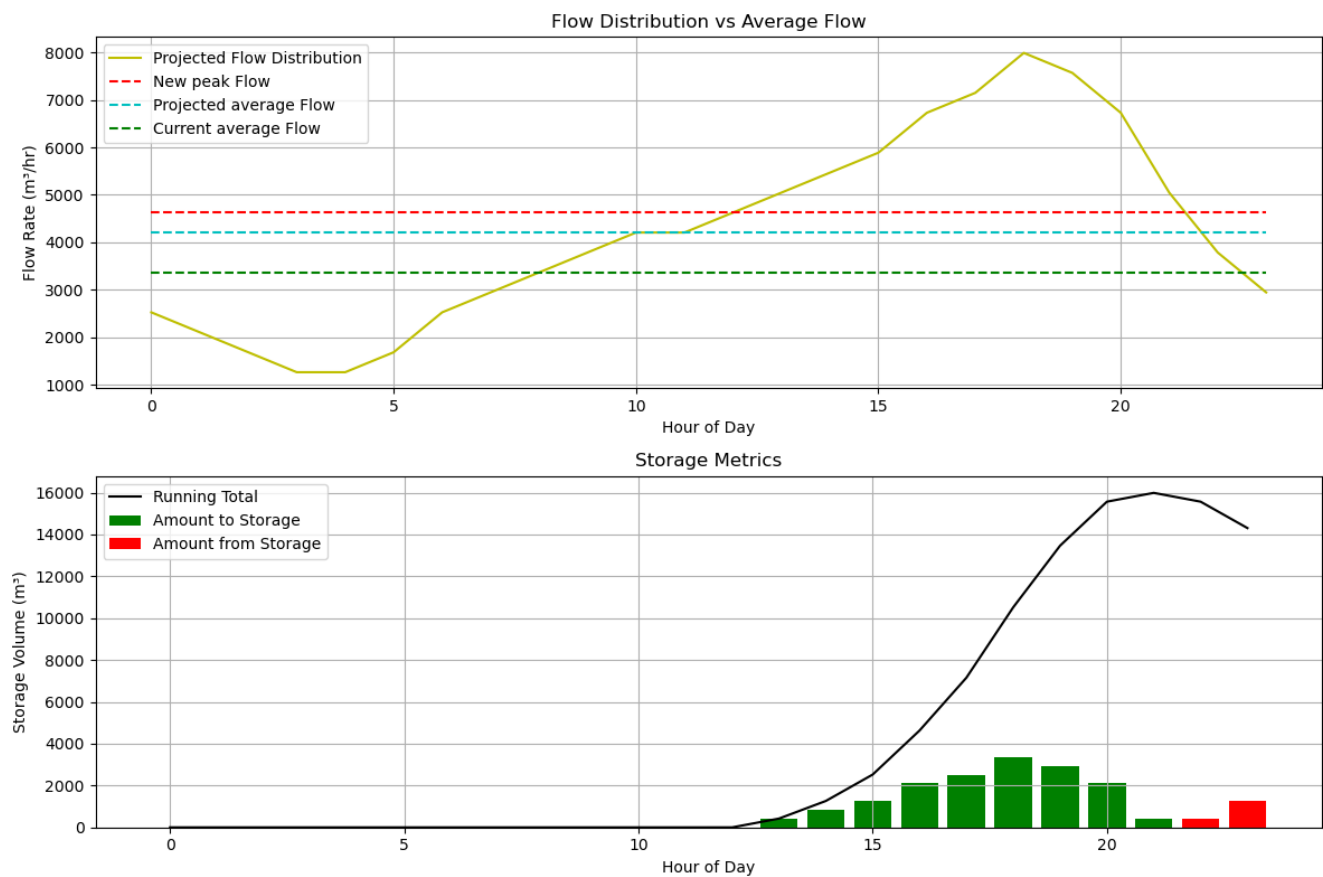


Figure 7: Mass balance diagram for projected conditions.

APPENDIX 6 EQUALIZATION OFF-LINE DIAGRAM

The selected off-line configuration for the equalization basin is shown in the figure below.

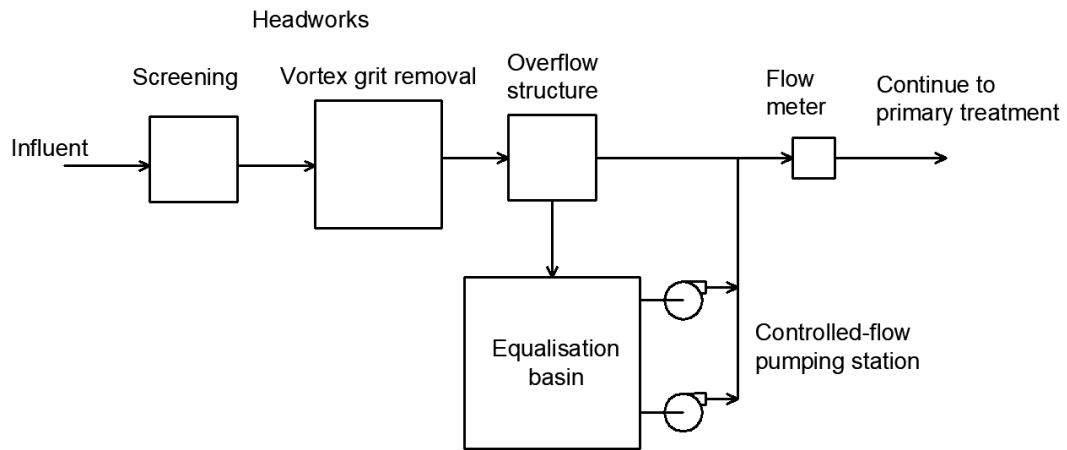


Figure 8: Off-line equalization basin –adapted from Fig 3-20, Davis,2010.

APPENDIX 7 INLET SECTIONS VIEWS

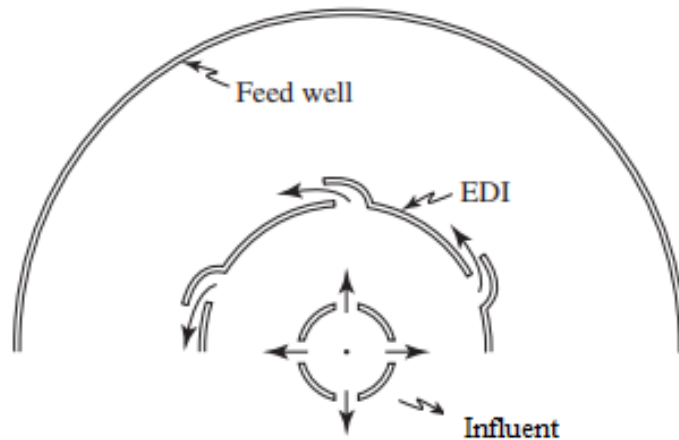


Figure 9: Feed well with energy dissipating inlet (EDI), plan view [Metcalf and Eddy, 2014].

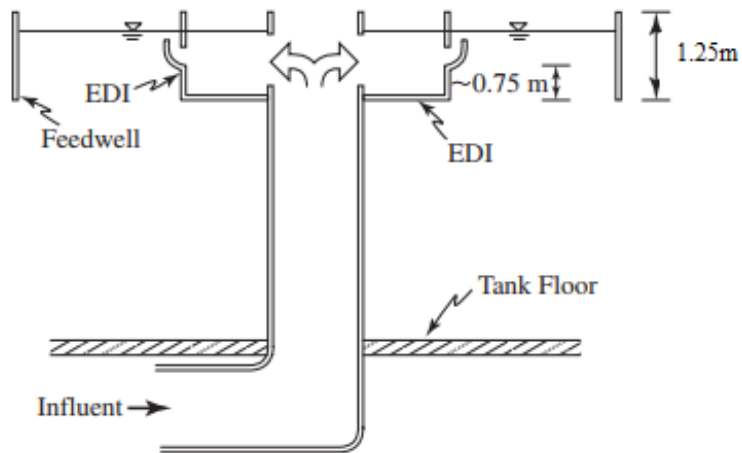


Figure 10: Inlet profile view [Metcalf and Eddy, 2014].

APPENDIX 8 STREETER-PHELPS MODEL

In the image below, the Streeter-Phelps model for the waterway discharge is presented. The vertical axes represents the deficit, thus zero value indicates the waterway returning to a $DO_{saturation}$ level. In the figure, it can be observed the maximum deficit which is achieved before $t = 2\text{days}$. It can also be observed that after approximately 5 days the DO returns to the initial waterway upstream initial value (DO_0).

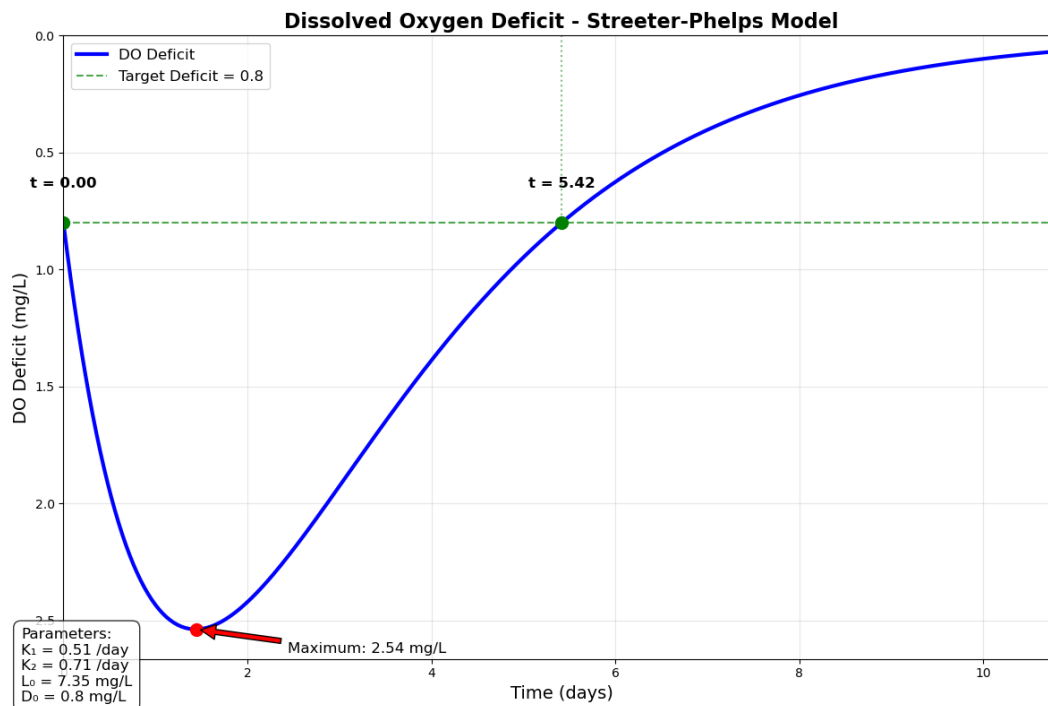


Figure 11: Dissolved oxygen deficit. Vertical axes being the deficit, with zero representation saturation.

APPENDIX 9 DESIGN CRITERIA FOR PRIMARY SEDIMENTATION BASINS

Table 19: Typical design criteria for primary sedimentation basins [Davis, 2010]

Parameter	Range of values	Typical/comment
<i>General</i>		
Overflow rate (average flow)	30 to 50 m ³ /d·m ²	40 m ³ /d·m ²
Overflow rate (peak flow)	60 to 120 m ³ /d·m ²	100 m ³ /d·m ²
Detention time (average flow)	1.5 to 2.5 h	2.0 h
Flow velocity	0.020 to 0.025 m/s	–
Weir loading rate	125 to 500 m ³ /d·m	250 m ³ /d·m
Sludge hoppers	1.7 vertical to 1 horizontal	Minimum; bottom width < 0.6 m
Geotechnical	–	Consider potential for flotation when tank is empty
<i>Circular tanks</i>		
<i>Dimensions</i>		
Diameter	3 to 100 m	12 to 45 m
Standard	9 to 45 m	In 1.5 m increments
Side water depth	3 to 5 m	4.3 m
Floor slope	1 vertical to 12 horizontal	–
Splitter box inlet velocity	< 0.3 m/s	At peak flow
<i>Inlet configuration</i>		
Detention time	20 minutes	Feedwell
Submergence	30 to 75% of depth	Size to prevent scour
EDI detention time	8 to 10 s	–

APPENDIX 10 OPERATIONAL CHARACTERISTICS OF ACTIVATED SLUDGE PROCESS

Table 20: Operational Characteristics of Activated Sludge Process [Metcalf and Eddy, 2014]

Process Modification	Flow Model	Aeration System	BOD Removal Efficiency (%)	Remarks
Conventional	Plug flow	Diffused-air, mechanical aerators	85–95	Use for low-strength domestic wastes; process is susceptible to shock loads
Complete-mix	Continuous-flow stirred-tank reactor	Diffused-air, mechanical aerators	85–95	Use for general application; process is resistant to shock loads but is susceptible to filamentous growths
Step feed	Plug flow	Diffused-air	85–95	Use for general application for a wide range of wastes
Modified aeration	Plug flow	Diffused-air	60–75	Use for intermediate degree of treatment where cell tissue in the effluent is not objectionable
Contact stabilization	Plug flow	Diffused-air, mechanical	80–90	Use for expansion of existing systems and package plants

APPENDIX 11 PYTHON SCRIPTS


```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

def insert_text_in_tex(file_path: str, insertion_keyword: str, text) -> bool:
    """
    Inserts `text` (string or list of strings) into a .tex file,
    replacing everything between the markers:

    % BEGIN <insertion_keyword>
    (old lines here)
    % END <insertion_keyword>

    Returns True on success, False if markers aren't found.
    """
    # Build the exact marker lines
    start_tag = f"% BEGIN {insertion_keyword}"
    end_tag = f"% END {insertion_keyword}"

    # Normalize `text` to a list of lines (each ending in `\n`)
    if isinstance(text, str):
        lines_to_insert = [text.rstrip("\n") + "\n"]
    elif isinstance(text, list):
        lines_to_insert = [str(t).rstrip("\n") + "\n" for t in text]
    else:
        raise TypeError("`text` must be a str or a list of str")

    # Read the file
    with open(file_path, "r", encoding="utf-8") as f:
        lines = f.readlines()

    # Find the markers
    start_idx = next((i for i, L in enumerate(lines) if L.strip() == start_tag), None)
    end_idx = next((i for i, L in enumerate(lines) if L.strip() == end_tag), None)

    if start_idx is None or end_idx is None or end_idx <= start_idx:
        return False

    # Replace everything *between* the markers by slice assignment
    # (this preserves the lines at start_idx and end_idx)
    lines[start_idx+1 : end_idx] = lines_to_insert

    # Write it back
    with open(file_path, "w", encoding="utf-8") as f:
        f.writelines(lines)

    return True

#Load pollution analysis
#From Table 3- 15. use the following average constituent contributions:
bod_load_g_per_capita = 60 # g/person/day, range 50-120 g/person/day
tss_load_g_per_capita = 60 # g/person/day, range 60-150 g/person/day

Q_domesticpercapita=162 #L/PE/day taken from standards.\n",
bod_concentration= round(bod_load_g_per_capita/Q_domesticpercapita*1000,2)# mg/L\n",
print('BOD average concentration: '+str(bod_concentration)+ 'g/m3')
tss_concentration= round(tss_load_g_per_capita/Q_domesticpercapita*1000,2)# mg/L\n",
print ('TSS average concentration : '+str(tss_concentration)+ 'g/m3')

BOD average concentration: 370.37g/m3
TSS average concentration : 370.37g/m3

```

✓ Estimation of design flows and pollutant loads.

```

import math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

```

```
#Estimation of design flows and pollutant loads.
```

```
group_number=10
PE_domestic=340000 #Equivalent population.
Q_domesticpercapita=162 #L/PE/day taken from standards.
PE_commercial=math.ceil(group_number**0.5*9000) #Equivalent population.
Q_commercialpercapita=280 #L/PE/day given in the briefing.
Q_infiltration=11657 #m3/day
```

```
#Current average flow rates
Q_currentavgdomestic=Q_domesticpercapita*PE_domestic/1000
Q_currentavgcommercial=Q_commercialpercapita*PE_commercial/1000
Q_currentavginfiltration=Q_infiltration
```

```
#Current minimum flow rates
peak_minimum_factor=0.3 #Taken from the graph given in the briefing
Q_minimumdomestic=Q_currentavgdomestic*peak_minimum_factor
Q_minimumcommercial=Q_currentavgcommercial*peak_minimum_factor
Q_minimuminfiltration=Q_currentavginfiltration
```

```
#Current maximum flow rates
peak_maximum_factor=1.9 #Taken from the graph given in the briefing
Q_maximumdomestic=Q_currentavgdomestic*peak_maximum_factor
Q_maximumcommercial=Q_currentavgcommercial*peak_maximum_factor
Q_maximuminfiltration=Q_currentavginfiltration
```

```
#Equivalent population projection
growth_percentage=1+group_number**0.015
Life_time=20 #Life time decided to our WWTP. Provisional
```

```
PE_projected_domestic=math.ceil(PE_domestic*(1+growth_percentage/100)**Life_time)
PE_projected_comercial=math.ceil(PE_commercial*(1+growth_percentage/100)**Life_time)
```

```
climate_chage_factor=1.2 #Taken from a projection studies.
```

```
#Projected average flow rates (m3/day)
Q_projected_domestic=Q_domesticpercapita*PE_projected_domestic/1000
Q_projected_comercial=Q_commercialpercapita*PE_projected_comercial/1000
Q_projected_infiltration=Q_currentavginfiltration*climate_chage_factor
```

```
#Projected minimum flow rates (m3/day)
```

```
Q_projected_minimum_domestic=Q_projected_domestic*peak_minimum_factor
Q_projected_minimum_commercial=Q_projected_comercial*peak_minimum_factor
Q_projected_minimuminfiltration=Q_minimuminfiltration*climate_chage_factor
```

```
#Projected maximum flow rates (m3/day)
Q_projected_maximum_domestic=Q_projected_domestic*peak_maximum_factor
Q_projected_maximum_commercial=Q_projected_comercial*peak_maximum_factor
Q_projected_maximuminfiltration=Q_maximuminfiltration*climate_chage_factor
```

```
#Total Projected flow rates (m3/day)
Q_total_projected_maximum=Q_projected_maximum_domestic+Q_projected_maximum_commercial+Q_projected_maximuminfiltration
Q_total_projected_minimum=Q_projected_minimum_domestic+Q_projected_minimum_commercial+Q_projected_minimuminfiltration
Q_total_projected_average=Q_projected_domestic+Q_projected_comercial+Q_projected_infiltration
```

```
#Total current flow rates (m3/day)
Q_total_current_maximum=Q_maximumdomestic+Q_maximumcommercial+Q_maximuminfiltration
Q_total_current_minimum=Q_minimumdomestic+Q_minimumcommercial+Q_minimuminfiltration
Q_total_current_average=Q_currentavgdomestic+Q_currentavgcommercial+Q_currentavginfiltration
```

```
print (PE_projected_domestic)
print (PE_projected_comercial)
```

```
data_current = {
    'Contribution type': ['Domestic', 'Commercial', 'Infiltration', 'Total'],
    'Population': [PE_domestic, PE_commercial, '--', '--'],
    'Minimum flow rate (m3/day)': [Q_minimumdomestic, Q_minimumcommercial, Q_minimuminfiltration, Q_total_current_minimum],
    'Average flow rate (m3/day)': [Q_currentavgdomestic, Q_currentavgcommercial, Q_currentavginfiltration, Q_total_current_average],
    'Maximum flow rate (m3/day)': [Q_maximumdomestic, Q_maximumcommercial, Q_maximuminfiltration, Q_total_current_maximum],
}
```

```
df1 = pd.DataFrame(data_current)
display(df1)
```

```
data_projected = {
    'Contribution type': ['Domestic ', 'Commercial', 'Infiltration', 'Total'],
```

```

'Population':[PE_projected_domestic,PE_projected_comercial,'--','--'],
'Minimum flow rate (m3/day)': [Q_projected_minimum_domestic, Q_projected_minimum_commercial, Q_projected_minimuminfiltration, Q_total_pi
'Average flow rate (m3/day)': [Q_projected_domestic,Q_projected_comercial, Q_projected_infiltration,Q_total_projected_average],
'Maximum flow rate (m3/day)': [Q_projected_minimum_domestic, Q_projected_minimum_commercial, Q_projected_minimuminfiltration, Q_total_pi
}
df2 = pd.DataFrame(data_projected)
display(df2)

```

↔ 508715
42584

	Contribution type	Population	Minimum flow rate (m3/day)	Average flow rate (m3/day)	Maximum flow rate (m3/day)	
0	Domestic	340000	16524	55080	104652	
1	Commercial	28461	2391	7969	15141	
2	Infiltration	--	11657	11657	11657	
3	Total	--	30572	74706	131450	
	Contribution type	Population	Minimum flow rate (m3/day)	Average flow rate (m3/day)	Maximum flow rate (m3/day)	
0	Domestic	508715	24724	82412	24724	
1	Commercial	42584	3577	11924	3577	
2	Infiltration	--	13988	13988	13988	
3	Total	--	42289	108324	193226	

Next steps:

Generate code with df1

View recommended plots

New interactive sheet

Generate code with df2

View recommended plots

New inter

✓ Design of screens - Preliminary treatment

5.4.3 <https://www.youtube.com/watch?v=2WwA7Mdrm8k>

#Preliminary treatment design:

#Velocity should be between 0.4 and 0.9 m/s

#Screen Width is recommended between 0.6 and 1.1 m

velocity_screen=0.90 #m/s, this value the first approximation

cd=0.84 #dimensionless, this is a value of discharge coefficient for the screen

gravity=9.81 #m/s^2

width_between_bars=0.009 #m, given that it need to trap gross pollutants >10 mm

bar_width=0.007 #m, depends on commercial availability. 8mm is within the commercial range.

velocity_upstream_screen=velocity_screen*width_between_bars/(width_between_bars+bar_width) #m/s

print ('The velocity upstream the screen is: '+str(velocity_upstream_screen))

head_loss_screen=round(1/(2*gravity*cd**2)*(velocity_screen**2-velocity_upstream_screen**2),2)

print ('the head loss screen is: '+str(head_loss_screen))

#Design of the screen channel

flow_depth_downstream=1 #m during desing peak flow.

flow_depth_upstream=flow_depth_downstream+head_loss_screen

print ('the flow depth upstream is: '+str(flow_depth_upstream))

wet_area=round((Q_total_projected_maximum/86400)/velocity_upstream_screen,2) #Flow converted into m^3/s, area in m^2

print ('The weat area is: '+str(wet_area)+' m2')

width_channel_screen = (wet_area / flow_depth_upstream)

Round up to the nearest multiple of 0.1

width_channel_screen_rounded = round(math.ceil(width_channel_screen / 0.2) * 0.2,2) #m

print ('The suitable width channel is: '+str(width_channel_screen_rounded)+' m')

if (width_channel_screen_rounded>1.1):

screen_number=math.ceil(width_channel_screen_rounded/1.1)

if not 0.3<screen_number<1.1:

screen_number=math.ceil(screen_number)

else:

```

print('it is too narrow')

print ('The number of screen should be: '+str(screen_number))

velocity_upstream_screen_projected_maximum=Q_total_projected_maximum/(86400*flow_depth_upstream*width_channel_screen_rounded) #m/s
print ('The velocity upstream the screen is: '+str(velocity_upstream_screen_projected_maximum)+ ' m/s')

#It need to comply with the minimum velocity with the minimum current flow. In this case, we want to find the
#downstream depth flow controlled by a hydraulic structure since this is not provided in the briefing.

velocity_upstream_screen_current_average=0.5 #m/s desired velocity

flow_depth_downstream_current_average=round((Q_total_current_average/86400)/(velocity_upstream_screen_current_average*width_channel_screen_1

print ('The depth flow controlled by a hydraulic structure should be less than '+str(flow_depth_downstream_current_average)+' m in order to

print( )
print( )

import sympy as sp

# Velocity with a 50 percent blockage and the average current flow

# 1) define symbols and constants
width_screen=width_channel_screen_rounded/screen_number
print ('The width of the screen is: '+str(width_screen))
v1_average = sp.symbols('v1', real=True)
h2_average = flow_depth_downstream_current_average

# 2) express h1(v1) and Δh(v1) from your derivation
vsc_average= (Q_total_current_average/86400/screen_number)/(h2_average*((width_screen-bar_width)/(bar_width+width_between_bars)*width_betwe
print ('The velocity screen chamber average flow is: '+str(vsc_average)+'m/s')
v2_average = (Q_total_current_average/screen_number/86400)/(h2_average*width_screen)
print ('The velocity downstream is: '+str(v2_average)+'m/s')
h1_average= h2_average*v2_average/v1_average #
vsc_coefficient_average=(width_between_bars+bar_width)/(width_between_bars*0.5)
delta = ((vsc_coefficient_average**2-1**2)* v1_average**2)/(2*gravity*cd**2) # Δh =

# 3) build the Bernoulli equation: h1 + v1^2/(2g) = h2 + v2^2/(2g) + Δh
eq_average = h1_average + v1_average**2/(2*gravity) - (h2_average + v2_average**2/(2*gravity) + delta)

# 4) solve for v1_average
solutions_average = sp.solve(eq_average, v1_average)
print('The velocity with a 50 percent blockage and the average current flow is: '+str(solutions_average)+' m/s')

# Velocity with a 50 percent blockage and the peak projected flow

v1_peak = sp.symbols('v1', real=True)
h2_peak = flow_depth_downstream
# 2) express h1(v1) and Δh(v1) from your derivation
vsc_peak= (Q_total_projected_maximum/86400/screen_number)/(flow_depth_upstream*((width_screen-bar_width)/(bar_width+width_between_bars)*wid
print ('The velocity screen chamber peak flow is: '+str(vsc_peak)+'m/s')
v2_peak = (Q_total_projected_maximum/screen_number/86400)/(flow_depth_downstream*width_screen)
print ('The velocity downstream is: '+str(v2_peak)+'m/s')
h1_peak = h2_peak*v2_peak/v1_peak
vsc_coefficient_peak=(width_between_bars+bar_width)/(width_between_bars*0.5)
delta = ((vsc_coefficient_peak**2-1**2)* v1_peak**2)/(2*gravity*cd**2) # Δh =

# 3) build the Bernoulli equation: h1 + v1^2/(2g) = h2 + v2^2/(2g) + Δh
eq_peak = h1_peak + v1_peak**2/(2*gravity) - (h2_peak + v2_peak**2/(2*gravity) + delta)

# 4) solve for v1_average
solutions_peak = sp.solve(eq_peak, v1_peak)
print('The velocity with a 50 percent blockage and the peak projected flow is: '+str(solutions_peak)+' m/s')

```

 The velocity upstream the screen is: 0.50625
 the head loss screen is: 0.04
 the flow depth upstream is: 1.04
 The weat area is: 4.42 m2
 The suitable width channel is: 4.4 m
 The number of screen should be: 4
 The velocity upstream the screen is: 0.48872530644991574 m/s

The depth flow controlled by a hydraulic structure should be less than 0.35 m in order to comply with the minimum velocity with the ave

The width of the screen is: 1.1
 The velocity screen chamber average flow is: 1.00454982349869m/s
 The velocity downstream is: 0.5614634439634439m/s
 The velocity with a 50 percent blockage and the average current flow is: [0.399364243398226] m/s
 The velocity screen chamber peak flow is: 0.8744094127801683m/s
 The velocity downstream is: 0.5082743187079123m/s
 The velocity with a 50 percent blockage and the peak projected flow is: [0.436722737979358] m/s

✓ Equalization basin

9.7.6. Equalization Sustainability and Resource Recovery Equalization operations have little resource recovery potential. However, they could provide significant sustainable features to a WRRF, as peak demand loadings and their associated larger energy requirements can be avoided when equalization is practiced.

Botero, Lucas, Joel C. Rife, Kendra D. Sveum, and Alex Szerwinski. 2018. "Equalization." Chap. 9.7 in Design of Water Resource Recovery Facilities. 6th ed., edited by The Water Environment Federation (WEF). McGraw-Hill Education: New York, Chicago, San Francisco, Athens, London, Madrid, Mexico City, Milan, New Delhi, Singapore, Sydney, Toronto.

<https://www.accessengineeringlibrary.com/content/book/9781260031188/toc-chapter/chapter9/section/section101>

CURRENT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

Q_new_peak = Q_total_current_average # m³/day, this becomes the new peak flow rate taken from the calculation of the primary treatment.

# Create data for the table
data = {
    'Time of Day': [f"{i} to {i+1}" for i in range(0, 24)],
    'Q average current flow (m³/hr)': [Q_total_current_average/24] * 24, # Constant average flow
    'Q new peak (m³/hr)': [Q_new_peak/24] * 24, # Constant average flow
}

# Flow distribution that follows a pattern (low at night, high during day)
# This values are taken from Daily fluctuation of peaking factors graph given in the briefing.
flow_coefficients = [
    0.6, 0.5, 0.4, 0.3, 0.3, 0.4, 0.6, 0.7,
    0.8, 0.9, 1, 1, 1.1, 1.2, 1.3, 1.4,
    1.6, 1.7, 1.9, 1.8, 1.6, 1.2, 0.9, 0.7
]

# Calculate actual flow values by multiplying coefficients by the average
flow_values_current = [coef * Q_total_current_average/24 for coef in flow_coefficients]
data['Flow distribution current Q (m³/hr)'] = flow_values_current[:24]

# Calculate the difference
data['Difference (Flow current-Q new peak) (m³/hr)'] = [q - data['Q new peak (m³/hr)'][0] for q in data['Flow distribution current Q (m³/hr)']]
data['Difference (Flow current-Qavg) (m³/hr)'] = [q - data['Q average current flow (m³/hr)'][0] for q in data['Flow distribution current Q (m³/hr)']]

# Initialize storage calculations
data['Amount to Storage (m³)'] = [0] * 24
data['Amount from Storage (m³)'] = [0] * 24
data['Running Total in Storage (m³)'] = [0] * 24

# Calculate storage values
running_total = 0
for i in range(24):
    diff1 = data['Difference (Flow current-Q new peak) (m³/hr)'][i]
    diff2 = data['Difference (Flow current-Qavg) (m³/hr)'][i]

    # If flow > avg, we add to storage
    if diff2 > 0 and diff1 > 0:
        data['Amount to Storage (m³)'][i] = diff1
        data['Amount from Storage (m³)'][i] = 0
        running_total += diff1
    # If flow < avg, we take from storage
    else:
        if running_total == 0:
```

```

    data['Amount to Storage (m³)'][i] = 0
    data['Amount from Storage (m³)'][i] = 0
    running_total += 0
else:
    data['Amount to Storage (m³)'][i] = 0
    data['Amount from Storage (m³)'][i] = abs(diff2)
    running_total -= abs(diff2)

data['Running Total in Storage (m³)'][i] = running_total

# Create DataFrame
df = pd.DataFrame(data)

# Format the DataFrame for better display
# Format time of day column
df['Time of Day'] = [f"{i:2d} to {i+1:2d}" for i in range(0, 24)]

# Apply formatting to numeric columns to match the table
pd.options.display.float_format = '{:.0f}'.format

# Create styled table
styled_df = df.style.set_table_styles([
    {'selector': 'th', 'props': [('color', 'black'),
                                  ('font-weight', 'bold'),
                                  ('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': 'td', 'props': [('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': '', 'props': [('border-collapse', 'collapse'),
                                ('width', '100%'),
                                ('font-size', '14px')]}
])

# Group headers
header_style = {
    'selector': 'th.col_heading',
    'props': [('background-color', '#0a6e3c'),
               ('color', 'white'),
               ('text-align', 'center'),
               ('font-weight', 'bold'),
               ('border', '1px solid black'),
               ('padding', '5px')]
}

# Display the styled table
display(styled_df)

# If you want to save to Excel
df.to_excel("water_flow_table.xlsx", index=False)

# Create a visualization of the data
plt.figure(figsize=(12, 8))

# Plot flow distribution vs average flow
plt.subplot(2, 1, 1)
plt.plot(range(24), df['Flow distribution current Q (m³/hr)'], 'b-', label='Flow Distribution')
plt.plot(range(24), df['Q average current flow (m³/hr)'], 'g--', label='Current average Flow')
plt.xlabel('Hour of Day')
plt.ylabel('Flow Rate (m³/hr)')
plt.title('Flow Distribution vs Average Flow')
plt.grid(True)
plt.legend()

# Plot storage metrics
plt.subplot(2, 1, 2)
plt.bar(range(24), df['Amount to Storage (m³)'], color='green', label='Amount to Storage')
plt.bar(range(24), df['Amount from Storage (m³)'], color='red', label='Amount from Storage')
plt.plot(range(24), df['Running Total in Storage (m³)'], 'k-', label='Running Total')
plt.xlabel('Hour of Day')
plt.ylabel('Storage Volume (m³)')
plt.title('Storage Metrics')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

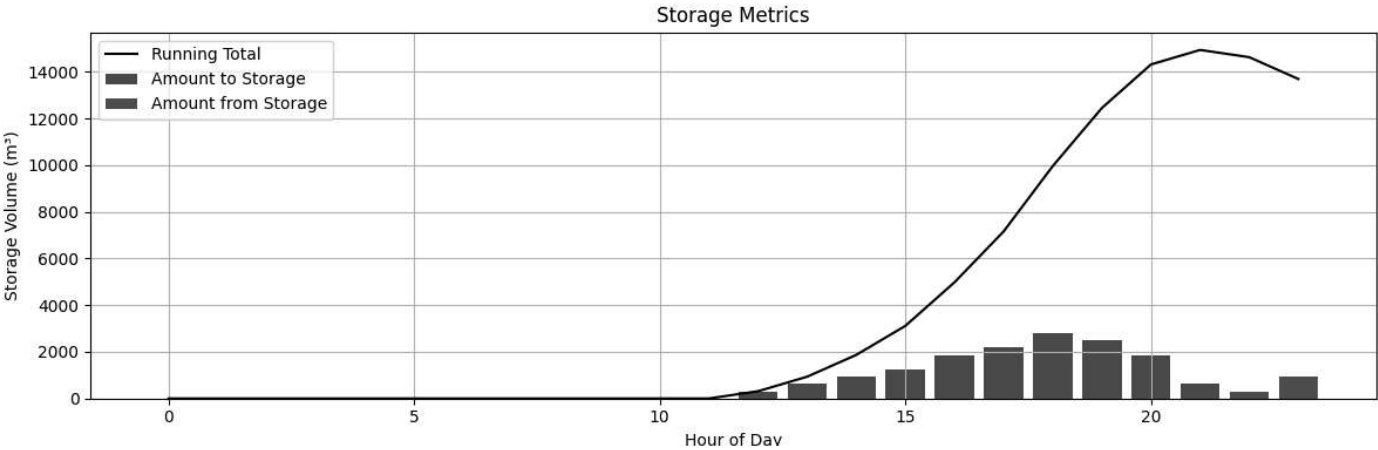
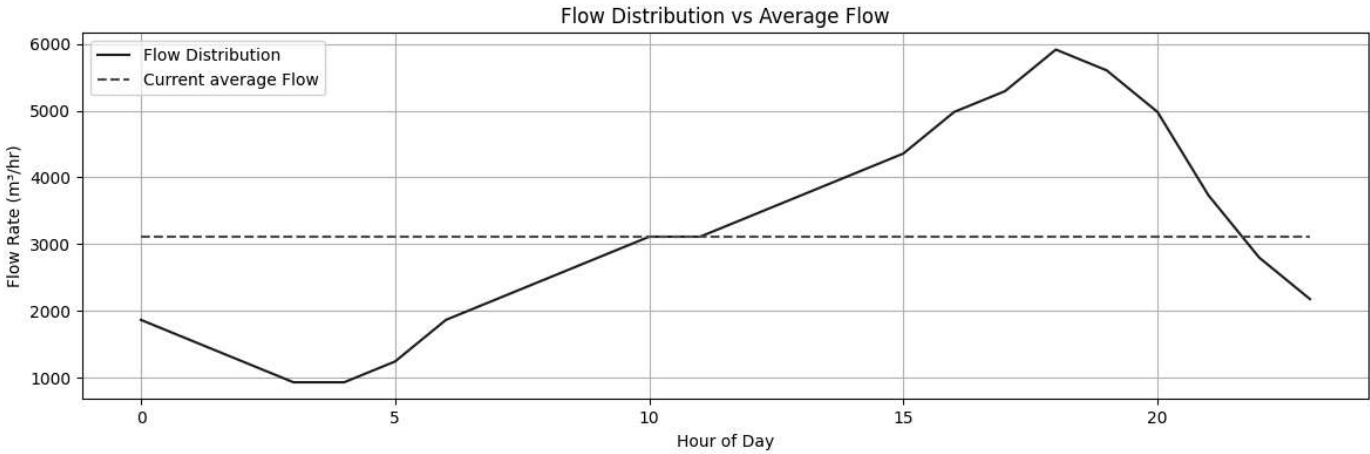
```

```
#Include adquate safety factors
```

```
safety_factor=1.2 #We need to justify this.  
required_volume_equalizer=safety_factor*max(df['Running Total in Storage (m³)'])  
print (max(df['Running Total in Storage (m³)']))  
rounded_required_volume_equalizer=round(required_volume_equalizer,-3) #m3  
print ('The required volume of the equalizer is: '+str(required_volume_equalizer))
```



	Time of Day	Q average current flow (m³/hr)	Q new peak (m³/hr)	Flow distribution current Q (m³/hr)	Difference (Flow current-Q new peak) (m³/hr)	Difference (Flow current- Qavg) (m³/hr)	Amount to Storage (m³)	Amount from Storage (m³)	Running Total in Storage (m³)
0	0 to 1	3112.753333	3112.753333	1867.652000	-1245.101333	-1245.101333	0.000000	0.000000	0.000000
1	1 to 2	3112.753333	3112.753333	1556.376667	-1556.376667	-1556.376667	0.000000	0.000000	0.000000
2	2 to 3	3112.753333	3112.753333	1245.101333	-1867.652000	-1867.652000	0.000000	0.000000	0.000000
3	3 to 4	3112.753333	3112.753333	933.826000	-2178.927333	-2178.927333	0.000000	0.000000	0.000000
4	4 to 5	3112.753333	3112.753333	933.826000	-2178.927333	-2178.927333	0.000000	0.000000	0.000000
5	5 to 6	3112.753333	3112.753333	1245.101333	-1867.652000	-1867.652000	0.000000	0.000000	0.000000
6	6 to 7	3112.753333	3112.753333	1867.652000	-1245.101333	-1245.101333	0.000000	0.000000	0.000000
7	7 to 8	3112.753333	3112.753333	2178.927333	-933.826000	-933.826000	0.000000	0.000000	0.000000
8	8 to 9	3112.753333	3112.753333	2490.202667	-622.550667	-622.550667	0.000000	0.000000	0.000000
9	9 to 10	3112.753333	3112.753333	2801.478000	-311.275333	-311.275333	0.000000	0.000000	0.000000
10	10 to 11	3112.753333	3112.753333	3112.753333	0.000000	0.000000	0.000000	0.000000	0.000000
11	11 to 12	3112.753333	3112.753333	3112.753333	0.000000	0.000000	0.000000	0.000000	0.000000
12	12 to 13	3112.753333	3112.753333	3424.028667	311.275333	311.275333	311.275333	0.000000	311.275333
13	13 to 14	3112.753333	3112.753333	3735.304000	622.550667	622.550667	622.550667	0.000000	933.826000
14	14 to 15	3112.753333	3112.753333	4046.579333	933.826000	933.826000	933.826000	0.000000	1867.652000
15	15 to 16	3112.753333	3112.753333	4357.854667	1245.101333	1245.101333	1245.101333	0.000000	3112.753333
16	16 to 17	3112.753333	3112.753333	4980.405333	1867.652000	1867.652000	1867.652000	0.000000	4980.405333
17	17 to 18	3112.753333	3112.753333	5291.680667	2178.927333	2178.927333	2178.927333	0.000000	7159.332667
18	18 to 19	3112.753333	3112.753333	5914.231333	2801.478000	2801.478000	2801.478000	0.000000	9960.810667
19	19 to 20	3112.753333	3112.753333	5602.956000	2490.202667	2490.202667	2490.202667	0.000000	12451.013333
20	20 to 21	3112.753333	3112.753333	4980.405333	1867.652000	1867.652000	1867.652000	0.000000	14318.665333
21	21 to 22	3112.753333	3112.753333	3735.304000	622.550667	622.550667	622.550667	0.000000	14941.216000
22	22 to 23	3112.753333	3112.753333	2801.478000	-311.275333	-311.275333	0.000000	311.275333	14629.940667
23	23 to 24	3112.753333	3112.753333	2178.927333	-933.826000	-933.826000	0.000000	933.826000	13696.114667



14941.215999999999

The required volume of the equalizer is: 17929.459199999998

PROJECTED

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

Q_new_peak = 4768.93*24 # m³/day, this becomes the new peak flow rate taken from the calculation of the primary treatment.

# Create data for the table
data = {
    'Time of Day': [f"{i}" for i in range(0, 24)],
    'Q average projected flow (m³/hr)': [Q_total_projected_average/24] * 24, # Constant average flow
    'Q new peak (m³/hr)': [Q_new_peak/24] * 24, # Constant average flow
}

# Flow distribution that follows a pattern (low at night, high during day)
# This values are taken from Daily fluctuation of peaking factors graph given in the briefing.
flow_coefficients = [
    0.6, 0.5, 0.4, 0.3, 0.3, 0.4, 0.6, 0.7,
    0.8, 0.9, 1, 1, 1.1, 1.2, 1.3, 1.4,
    1.6, 1.7, 1.9, 1.8, 1.6, 1.2, 0.9, 0.7
]

# Calculate actual flow values by multiplying coefficients by the average
flow_values_projected = [coef * Q_total_projected_average/24 for coef in flow_coefficients]
data['Flow distribution projected Q (m³/hr)'] = flow_values_projected[:24]

# Calculate the difference
data['Difference (Flow projected-Q new peak) (m³/hr)'] = [q - data['Q new peak (m³/hr)'][0] for q in data['Flow distribution projected Q (m³/hr)']]
data['Difference (Flow projected-Qavg) (m³/hr)'] = [q - data['Q average projected flow (m³/hr)'][0] for q in data['Flow distribution projected Q (m³/hr)']]

# Initialize storage calculations
data['Amount to Storage (m³)'] = [0] * 24
data['Amount from Storage (m³)'] = [0] * 24
data['Running Total in Storage (m³)'] = [0] * 24

# Calculate storage values
running_total = 0
for i in range(24):
    diff1 = data['Difference (Flow projected-Q new peak) (m³/hr)'][i]
    diff2 = data['Difference (Flow projected-Qavg) (m³/hr)'][i]

    # If flow > avg, we add to storage
    if diff2 > 0 and diff1 > 0:
        data['Amount to Storage (m³)'][i] = diff1
        data['Amount from Storage (m³)'][i] = 0
        running_total += diff1
    # If flow < avg, we take from storage
    else:
        if running_total == 0:
            data['Amount to Storage (m³)'][i] = 0
            data['Amount from Storage (m³)'][i] = 0
            running_total += 0
        else:
            data['Amount to Storage (m³)'][i] = 0
            data['Amount from Storage (m³)'][i] = abs(diff2)
            running_total -= abs(diff2)

    data['Running Total in Storage (m³)'][i] = running_total

# Create DataFrame
df2 = pd.DataFrame(data)

# Format the DataFrame for better display
# Format time of day column
df2['Time of Day'] = [f"{i:2d}" for i in range(0, 24)]

# Apply formatting to numeric columns to match the table
pd.options.display.float_format = '{:.0f}'.format

```

```

# Create styled table
styled_df2 = df2.style.set_table_styles([
    {'selector': 'th', 'props': [('color', 'black'),
                                  ('font-weight', 'bold'),
                                  ('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': 'td', 'props': [('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': '', 'props': [('border-collapse', 'collapse'),
                              ('width', '100%'),
                              ('font-size', '14px')]}
])

# Group headers
header_style = {
    'selector': 'th.col_heading',
    'props': [('background-color', '#0a6e3c'),
              ('color', 'white'),
              ('text-align', 'center'),
              ('font-weight', 'bold'),
              ('border', '1px solid black'),
              ('padding', '5px')]
}

# Display the styled table
display(styled_df2)

# If you want to save to Excel
df2.to_excel("water_flow_table.xlsx", index=False)

# Create a visualization of the data
plt.figure(figsize=(12, 8))

# Plot flow distribution vs average flow
plt.subplot(2, 1, 1)
plt.plot(range(24), df2['Flow distribution projected Q (m³/hr)'], 'y-', label='Projected Flow Distribution')
plt.plot(range(24), df2['Q new peak (m³/hr)'], 'r--', label='New peak Flow')
plt.plot(range(24), df2['Q average projected flow (m³/hr)'], 'c--', label='Projected average Flow')
plt.plot(range(24), df2['Q average current flow (m³/hr)'], 'g--', label='Current average Flow')
plt.xlabel('Hour of Day')
plt.ylabel('Flow Rate (m³/hr)')
plt.title('Flow Distribution vs Average Flow')
plt.grid(True)
plt.legend()

# Plot storage metrics
plt.subplot(2, 1, 2)
plt.bar(range(24), df2['Amount to Storage (m³)'], color='green', label='Amount to Storage')
plt.bar(range(24), df2['Amount from Storage (m³)'], color='red', label='Amount from Storage')
plt.plot(range(24), df2['Running Total in Storage (m³)'], 'k-', label='Running Total')
plt.xlabel('Hour of Day')
plt.ylabel('Storage Volume (m³)')
plt.title('Storage Metrics')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

#Include adequate safety factors

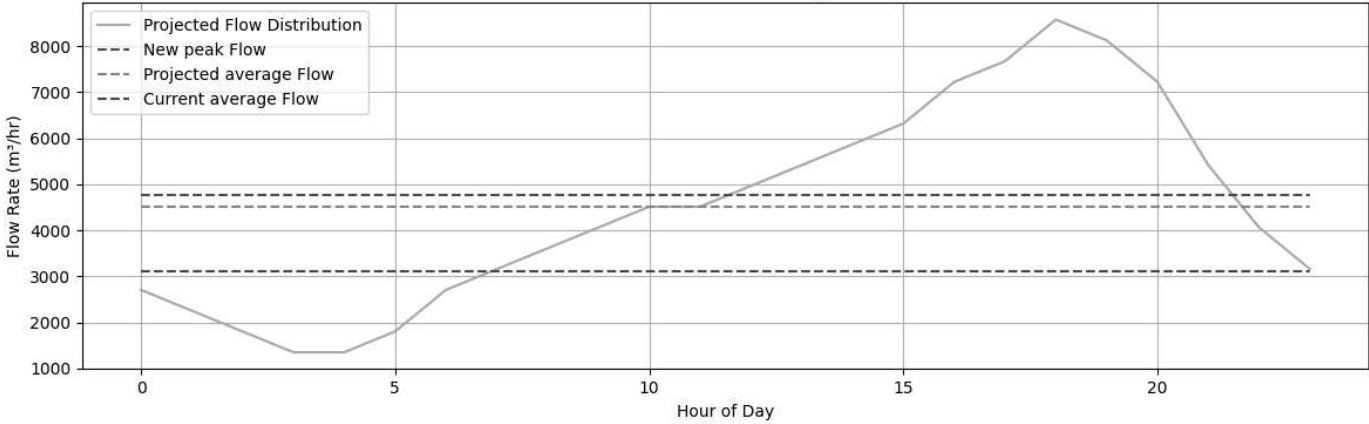
safety_factor=1.1 #We need to justify this.
required_volume_equalizer=safety_factor*max(df2['Running Total in Storage (m³)'])
print(max(df2['Running Total in Storage (m³)']))
rounded_required_volume_equalizer=round(required_volume_equalizer,-3) #m3
print('The required volume of the equalizer is: '+str(required_volume_equalizer))

```

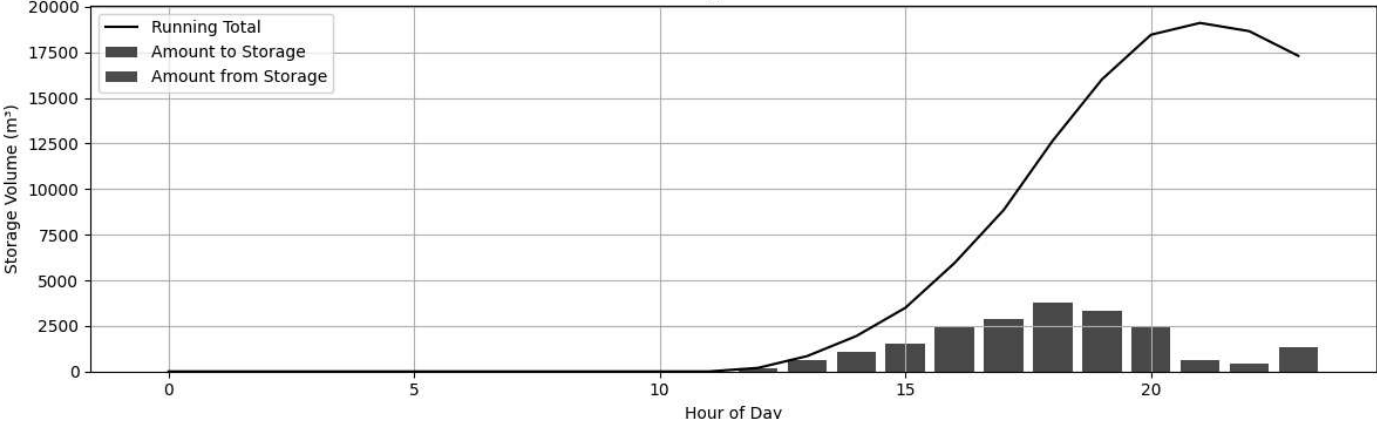


	Time of Day	Q average projected flow (m³/hr)	Q new peak (m³/hr)	Flow distribution projected Q (m³/hr)	Difference (Flow projected-Q new peak) (m³/hr)	Difference (Flow projected- Qavg) (m³/hr)	Amount to Storage (m³)	Amount from Storage (m³)	Running Total in Storage (m³)
0	0 to 1	4513.489583	4768.930000	2708.093750	-2060.836250	-1805.395833	0.000000	0.000000	0.000000
1	1 to 2	4513.489583	4768.930000	2256.744792	-2512.185208	-2256.744792	0.000000	0.000000	0.000000
2	2 to 3	4513.489583	4768.930000	1805.395833	-2963.534167	-2708.093750	0.000000	0.000000	0.000000
3	3 to 4	4513.489583	4768.930000	1354.046875	-3414.883125	-3159.442708	0.000000	0.000000	0.000000
4	4 to 5	4513.489583	4768.930000	1354.046875	-3414.883125	-3159.442708	0.000000	0.000000	0.000000
5	5 to 6	4513.489583	4768.930000	1805.395833	-2963.534167	-2708.093750	0.000000	0.000000	0.000000
6	6 to 7	4513.489583	4768.930000	2708.093750	-2060.836250	-1805.395833	0.000000	0.000000	0.000000
7	7 to 8	4513.489583	4768.930000	3159.442708	-1609.487292	-1354.046875	0.000000	0.000000	0.000000
8	8 to 9	4513.489583	4768.930000	3610.791667	-1158.138333	-902.697917	0.000000	0.000000	0.000000
9	9 to 10	4513.489583	4768.930000	4062.140625	-706.789375	-451.348958	0.000000	0.000000	0.000000
10	10 to 11	4513.489583	4768.930000	4513.489583	-255.440417	0.000000	0.000000	0.000000	0.000000
11	11 to 12	4513.489583	4768.930000	4513.489583	-255.440417	0.000000	0.000000	0.000000	0.000000
12	12 to 13	4513.489583	4768.930000	4964.838542	195.908542	451.348958	195.908542	0.000000	195.908542
13	13 to 14	4513.489583	4768.930000	5416.187500	647.257500	902.697917	647.257500	0.000000	843.166042
14	14 to 15	4513.489583	4768.930000	5867.536458	1098.606458	1354.046875	1098.606458	0.000000	1941.772500
15	15 to 16	4513.489583	4768.930000	6318.885417	1549.955417	1805.395833	1549.955417	0.000000	3491.727917
16	16 to 17	4513.489583	4768.930000	7221.583333	2452.653333	2708.093750	2452.653333	0.000000	5944.381250
17	17 to 18	4513.489583	4768.930000	7672.932292	2904.002292	3159.442708	2904.002292	0.000000	8848.383542
18	18 to 19	4513.489583	4768.930000	8575.630208	3806.700208	4062.140625	3806.700208	0.000000	12655.083750
19	19 to 20	4513.489583	4768.930000	8124.281250	3355.351250	3610.791667	3355.351250	0.000000	16010.435000
20	20 to 21	4513.489583	4768.930000	7221.583333	2452.653333	2708.093750	2452.653333	0.000000	18463.088333
21	21 to 22	4513.489583	4768.930000	5416.187500	647.257500	902.697917	647.257500	0.000000	19110.345833
22	22 to 23	4513.489583	4768.930000	4062.140625	-706.789375	-451.348958	0.000000	451.348958	18658.996875
23	23 to 24	4513.489583	4768.930000	3159.442708	-1609.487292	-1354.046875	0.000000	1354.046875	17304.950000

Flow Distribution vs Average Flow



Storage Metrics



```
19110.345833333333
```

```
The required volume of the equalizer is: 21021.380416666667
```

✓ Secondary Treatment

First Iteration - Using Conventional process

```
#First, we will calculate the influent BOD5 concentration from the primary clarifier knowing that the BOD5 removal efficiency is
#93% and the effluent BOD5 concentration is 18 mg/L. This is the maximum value that we can have in the effluent and the maximum
# efficiency that we can have in the secondary clarifier.

BOD_removal_efficiency=0.9318 #dimensionless, this is the range of maximum value that we can have in the secondary clarifier for the conventional process
print ('The BOD removal efficiency is: '+str(BOD_removal_efficiency))
BOD_required_to_effluent=18 #mg/L

#After that we calculate the BOD5 concentration in the primary clarifier outflow using the following equation:
BOD_primary_clarifier=round(BOD_required_to_effluent/(1-BOD_removal_efficiency),2) #mg/L
print ('BOD primary clarifier is: ' +str(BOD_primary_clarifier)+ ' mg/L')

Q1=Q_new_peak #m3/d, This is the flow from the equalization tank upstream.

#MLSS value is taken since we have a 10 degree water temperature and it is the maximum value that we can have
#in the secondary clarifier according to Table 8-19 Metcalf & Eddy.

mixed_liquor_suspended_solids=3000 #mg/L(X), this should be between 1000-3000 mg/L for a conventional activated sludge process.

#Typical values of the yield coefficient and decay coefficient. Given that we have a 10 degree water temperature
#the values are modified using the Arrhenius equation.
yield_coefficient=0.45 #SS/mg BOD5 (Y)
decay_coefficient=0.04 #/day (Kd)
conversion_rate_constant=0.2 #/day (k)

#We want the SVI less than 120 mL/g, this is recommended to have a good settling sludge. Therefore, using 91 mL/g value
# we can calculate the return sludge concentration, reported by Chudoba et al (1973) in order to get a good mix of microbial community.

suspended_volumen_index=91 #mL/g
print ('SVI is: '+str(suspended_volumen_index)+' mL/g')

#Therefore, the return sludge concentration is:

return_sludge_suspended_solids=10**6/suspended_volumen_index #mg/L(XR),
print ('The return sludge concentration is: '+str(return_sludge_suspended_solids)+' mg/L')

#Using this SVI value, and having selected the MLSS value, we can calculate the volumen settled sludge.
volumen_settled_sludge=suspended_volumen_index*mixed_liquor_suspended_solids/1000 #v
print ('The volumen settled is: '+str(volumen_settled_sludge)+' mL/L')

#Knowing the volumen settled sludge, we can calculate the return sludge flow.
return_sludge_flow=(Q1*volumen_settled_sludge)/(1000-volumen_settled_sludge) #m3/d
print ('The return sludge is: '+str(return_sludge_flow)+' m3/d')

#Using the return sludge flow, we can calculate the return sludge ratio
return_sludge_ratio=return_sludge_flow/Q1 # this should be between 0.25-0.75 for a conventional activated sludge process.
print ('The return sludge ratio is: '+str(return_sludge_ratio))

food_to_microorganism_ratio=0.4 #This value depends on the process, for this case it is a conventional process.
#We select the minimum Try to minimize the aeration volumen. it could be 0.2-0.4 for a conventional activated sludge process according to 1

#The aeration volume is calculated using the BOD5 concentration from the primary clarifier, the flow rate, the food to microorganism ratio,
aeration_volumen=(BOD_primary_clarifier*Q1)/(food_to_microorganism_ratio*mixed_liquor_suspended_solids)
print ('aeration volumen is: ' +str(aeration_volumen)+ ' m3')

#The aeration time is calculated using the aeration volume and the flow rate.
aeration_time=(aeration_volumen/Q1)*24 #hours. This value should be between 4-8 hours for a conventional activated sludge process.
print ('aeration time is: ' +str(aeration_time)+ ' hours')

#This value should be between 300-700 g/m3/day for a conventional activated sludge process. Therefore, we need to select another process.
BOD_load=(BOD_primary_clarifier*Q1)/aeration_volumen #g/m3/day
print ('BOD load is: ' +str(BOD_load)+ ' g/m3/day')

#Even though I tried to vary the F/M ratio either the aeration time or BOD load are not in the range of the conventional activated sludge process
```

```

→ The BOD removal efficiency is: 0.9318
BOD primary clarifier is: 263.93 mg/L
SVI is: 91 mL/g
The return sludge concentration is: 10989.010989010989 mg/L
The volumen settled is: 273.0 mL/L
The return sludge is: 42979.407647867956 m3/d
The return sludge ratio is: 0.375515818431912
aereation volumen is: 25173.273898000003 m3
aereation time is: 5.2786 hours
BOD load is: 1200.0 g/m3/day

```

Second Iteration - Using CMAS

#First, we will calculate the influent BOD5 concentration from the primary clarifier kwnowing that the BOD5 removal efficiency is #93% and the effluent BOD5 concentration is 18 mg/L. This is the maximum value that we can have in the effluent and the maximim # efficiency that we can have in the secondary clarifier.

```

BOD_removal_efficiency=0.9318 #dimensionless, this is the range of maximum value that we can have in the secondary clarifier for the CMAS prc
print ('The BOD removal efficiency is: '+str(BOD_removal_efficiency))
text=[str(round(BOD_removal_efficiency*100,1))]
keyword='py_BOD_removal_eff_sec'

```

```

BOD_required_to_effluent=18 #mg/L
text=[str(round(BOD_required_to_effluent,1))]
keyword='BOD_eff_sec'

```

```

BOD_primary_clarifier=round(BOD_required_to_effluent/(1-BOD_removal_efficiency),2) #mg/L
text=[str(round(BOD_primary_clarifier,1))]
keyword='py_BOD_in_sec'

```

```

print ('BOD primary clarifier is: ' +str(BOD_primary_clarifier)+ ' mg/L')

```

```

Q1=Q_new_peak #m3/d, This is the flow from the equalization tank upstream.
text=[str(round(Q1,1))]
keyword='py_Q_new_peak_secondary'

```

```

text=[str(round(Q_total_projected_average,1))]
keyword='py_Q_avg_sec'

```

#MLSS value is taken since we have a 10 degree water temperature and it is the maximum value that we can have #in the secondary clarifier according to Table 8-19 Metcalf & Eddy.

```

mixed_liquor_suspended_solids=4000 #mg/L(X), this shoould be between 2500-4000 mg/L for a CMAS activated sludge process.
text=[str(round(mixed_liquor_suspended_solids,1))]
keyword='py_MLSS_sec'
print ('The MLSS is: '+str(mixed_liquor_suspended_solids)+' mg/L')

```

```

#Typical values of the yield coefficient and decay coefficient. Given that we have a 10 degree water temperature
#the values are modified using the Arrhenius equation.
yield_coefficient=0.45 #SS/mg BOD5 (Y)
text=[str(round(yield_coefficient,2))]
keyword='py_Y_sec'

```

```

decay_coefficient=0.04 #/day (Kd)
text=[str(round(decay_coefficient,2))]
keyword='py_Kd_sec'

```

```

conversion_rate_constant=0.2 #/day (k), This a constant for BOD.
text=[str(round(conversion_rate_constant,1))]
keyword='py_k_sec'

```

#We want the SVI less than 120 mL/g, this is recommended to have a good settling sludge. Therefore, using 91 mL/g value # we can calculate the return sludge concentration, reported by Chudoba et al (1973) in order to get a good mix of microbial community

```

suspended_volumen_index=91 #mL/g
text=[str(round(suspended_volumen_index,1))]
keyword='py_SVI_sec'

print ('SVI is: '+str(suspended_volumen_index)+' mL/g')

#Therefore, the return sludge concentration is:

return_sludge_suspended_solids=round(10**6/suspended_volumen_index,2) #mg/L(XR),
text=[str(round(return_sludge_suspended_solids,1))]
keyword='py_X_R_sec'

print ('The return sludge concentration is: '+str(return_sludge_suspended_solids)+' mg/L')

#Using this SVI value, and havind selected the MLSS value, we can calculate the volumen settled sludge.
volumen_settled_sludge=suspended_volumen_index*mixed_liquor_suspended_solids/1000 #v
text=[str(round(volumen_settled_sludge,1))]
keyword='py_VolumeSettledSludge'

print ('The volumen settled is: '+str(volumen_settled_sludge)+' mL/L')

#Knowing the volumen settled sludge, we can calculate the return sludge flow.
return_sluge_flow=round((Q1*volumen_settled_sludge)/(1000-volumen_settled_sludge),2) #m3/d
text=[str(round(return_sluge_flow,1))]
keyword='py_ReturnFlow_sec'

print ('The return sludge flow is: '+str(return_sluge_flow)+' m3/d')

#Using the return sludge flow, we can calculate the return sludge ratio
return_sludge_ratio=round(return_sluge_flow/Q1,2) # this should be between 0.25-1 for a CMAS activated sludge process.
print ('The return sludge ratio is: '+str(return_sludge_ratio))

food_to_microorganism_ratio=0.4 #This value depends on the process, for this case it is a CMAS process.
#We select the minimum Try to minimize the volumen. it could be 0.2-0.6 for a CMAS activated sludge process according to the table 8-19 Metca
#This value was initial supposed to be 0.6 but it was changed to 0.4 in order to get a aeration time.

text=[str(round(food_to_microorganism_ratio,1))]
keyword='py_F_M_sec'

#The aeration volume is calculated using the BOD5 concentration from the primary clarifier, the flow rate, the food to microorganism ratio, a
aeration_volumen=round((BOD_primary_clarifier*Q1)/(food_to_microorganism_ratio*mixed_liquor_suspended_solids),2)
text=[str(round(aeration_volumen,1))]
keyword='py_Aeration_Volume_sec'

print ('aeration volumen is: ' +str(aeration_volumen)+ ' m3')

#The aeration time is calculated using the aeration volume and the flow rate.
aeration_time=round((aeration_volumen/Q1)*24,2) #hours. This value should be between 3-6 hours for a CMAS activated sludge process.
text=[str(round(aeration_time,1))]
keyword='py_HRT_sec'

print ('aeration time is: ' +str(aeration_time) + ' hours')

#The organic loading rate, an essential performance parameter, is computed as follows:
#This value should be between 300-1600 g/m3/day for a CMAS activated sludge process.
BOD_load=round((BOD_primary_clarifier*Q1)/aeration_volumen,2) #g/m3/day
text=[str(round(BOD_load,1))]
keyword='py_Organic_load_rate_sec'

print ('BOD load is: ' +str(BOD_load)+ ' g/m3/day')

#Typically, kinetic coefficients for activated sludge processes are reported at 20 °C. To adapt these to our chosen design temperature (10 °C
specific_utilisation_ratio=round((food_to_microorganism_ratio*BOD_removal_efficiency),2) #dimensionless
text=[str(round(specific_utilisation_ratio,1))]
keyword='py_UtilizationRatio'

print ('specific utilisation ratio is: ' +str(specific_utilisation_ratio))

#Using these adjusted coefficients, the required sludge age is calculated. A safety factor of 1.3 is applied to accommodate peak organic load
#The sludge age should be between 5-15 days for a CMAS activated sludge process.
#Using these adjusted coefficients, the required sludge age is calculated. A safety factor of 1.3 is applied to accommodate peak organic load
sludge_age=round(1/(yield_coefficient*specific_utilisation_ratio-decay_coefficient),2) #days. This value should be between 5-15 days for a CM
text=[str(round(sludge_age,1))]
keyword='py_sludgeAge_sec'

```

```

print( 'sludge age is: ' +str(sludge_age)+ ' days ')

safety_factor_peak_org_loading=1.3 #This try to account the variation of BOD regardless the flow.

sludge_age_increased=round(sludge_age*safety_factor_peak_org_loading,2) #days. This value should be between 5-15 days for a CMAS activated sl
text=[str(round(sludge_age_increased,1))]
keyword='py_sludgeAgeAdj_sec'

print('sludge age increased is: ' +str(sludge_age_increased))

#Waste flow is calculated using the mixed liquor suspended solids concentration, the aeration volume, the return sludge suspended solids conc

flow_waste=(mixed_liquor_suspended_solids*aeration_volumen)/(return_sludge_suspended_solids*sludge_age)
text=[str(round(flow_waste,1))]
keyword='py_flowWaste'

print ('flow waste with a sludge age is: ' +str(flow_waste)+ ' m3/d')

flow_waste_increased=round((mixed_liquor_suspended_solids*aeration_volumen)/(return_sludge_suspended_solids*sludge_age_increased),2) #m3/d
text=[str(round(flow_waste_increased,1))]
keyword='py_flowWasteAdj'

print ('flow waste increased with a sludge age increased is: ' +str(flow_waste_increased)+ ' m3/d')

flow_effluent=round(Q1-flow_waste,2) #m3/d
text=[str(round(flow_effluent,1))]
keyword='py_effluentFlow'

print ('flow effluent with a sludge age is: ' +str(flow_effluent)+ ' m3/d')

flow_effluent_increased=round(Q1-flow_waste_increased,2) #m3/d
text=[str(round(flow_effluent_increased,1))]
keyword='py_effluentFlowAdj'

print ('flow effluent with a sludge age increased is: ' +str(flow_effluent_increased)+ ' m3/d')

print ()

maximum_volumen_cmas_tank=6*24*90 #m3, this is the maximum value that we can have in the tank.
print('maximum volumen cmass tank is: ' +str(maximum_volumen_cmas_tank)+ ' m3')

number_aeration_tanks=6
text=[str(round(number_aeration_tanks,1))]
keyword='py_NAerationTanks'

if aeration_volumen<maximum_volumen_cmas_tank:
    aeration_volumen_tank=maximum_volumen_cmas_tank
    print('aeration volumen tank is: ' +str(aeration_volumen)+ ' m3')
else:
    #I divide this value by 6 because the tank should be less than 3500 m3. This is to address the filamentous bulking problem.
    aeration_volumen_tank=round(aeration_volumen/number_aeration_tanks,2) #m3, this is the maximum value that we can have in the tank.
    print('each aeration volumen tank is: ' +str(aeration_volumen_tank)+ ' m3')

text=[str(round(aeration_volumen_tank,1))]
keyword='py_VolumePerAerationTank'

cmass_tank_depth=4.2 #m, it should between 3-6 m.
text=[str(round(cmass_tank_depth,1))]
keyword='py_aerationTankDepth'

print('cmass tank depth is: ' +str(cmass_tank_depth)+ ' m')

cmass_tank_width=4*cmass_tank_depth #m,
text=[str(round(cmass_tank_width,1))]
keyword='py_aerationTankWidth'

print('cmass tank width is: ' +str(cmass_tank_width)+ ' m')

cmass_tank_length=round(math.ceil(aeration_volumen_tank/(cmass_tank_depth*cmass_tank_width)/1)*1,2) #m
text=[str(round(cmass_tank_length,1))]
keyword='py_aerationTankLength'

print('cmass tank length is: ' +str(cmass_tank_length)+ ' m')

cmass tank calculated volume=cmass tank depth*cmass tank width*cmass tank length

```

```

cmas_tank_calculated_volume cmas_tank_oper cmas_tank_max cmas_tank_slopes
text=[str(round(cmas_tank_calculated_volume,1))]
keyword='py_VolumeAerationTankFinal'

print( )

solids_loading_ratio_peak_flow=6 #kg/m2/h, this value is recommended between
text=[str(round(solids_loading_ratio_peak_flow,1))]
keyword='py_SLR_selected'

# 4-6 kg/m2/d for a average flow in the secondary clarifier, given that we have a similar peak and average flow due to equalisation basin.

area_secondary_clarifier=((Q1+return_sludge_ratio*Q1)*mixed_liquor_suspended_solids)/(solids_loading_ratio_peak_flow*24*1000) #m2, this is th
text=[str(round(area_secondary_clarifier,1))]
keyword='py_areaClarifier_sec'

print('area secondary clarifier is: ' +str(area_secondary_clarifier)+ ' m2')

solids_loading_ratio_average_flow=((Q_total_projected_average+return_sludge_ratio*Q1)*mixed_liquor_suspended_solids)/(area_secondary_clarifie
text=[str(round(solids_loading_ratio_average_flow,1))]
keyword='py_SLR_sec'

print('solids loading ratio average projected flow is: ' +str(solids_loading_ratio_average_flow)+ ' kg/m2/h')

print( )

#Aeration requirements
yield_observed_average=round(yield_coefficient/(1+decay_coefficient*sludge_age),2) #SS/mg BOD5 (Yobs)
text=[str(round(yield_observed_average,3))]
keyword='py_Yobs_sec'

rate_generation_biomass_Q_average=round(yield_observed_average*Q_total_projected_average*(BOD_primary_clarifier-BOD_required_to_effluent)/100
text=[str(round(rate_generation_biomass_Q_average,1))]
keyword='py_P_avg_sec'

print('Generation of biomass rate for Q average is: ' +str(rate_generation_biomass_Q_average)+ ' m3/d')

yield_observed_peak=round(yield_coefficient/(1+decay_coefficient*sludge_age_increased) #SS/mg BOD5 (Yobs)
text=[str(round(yield_observed_peak,3))]
keyword='py_Y_peaksec'

rate_generation_biomass_Q_peak=round(yield_observed_peak*Q_new_peak*(BOD_primary_clarifier-BOD_required_to_effluent)/1000,2) #Px
text=[str(round(rate_generation_biomass_Q_peak,1))]
keyword='py_P_peak_sec'

print('Generation of biomass rate for Q peak is: ' +str(rate_generation_biomass_Q_peak)+ ' m3/d')

f=1-math.exp(-5*conversion_rate_constant) #dimensionless
oxygen_demand_Q_peak=round((Q_new_peak*(BOD_primary_clarifier-BOD_required_to_effluent))/f/1000-1.42*rate_generation_biomass_Q_peak,2) #kg O2
text=[str(round(oxygen_demand_Q_peak,1))]
keyword='py_Oxygen_peak'

print('Oxygen demand for Q peak is: ' +str(oxygen_demand_Q_peak)+ ' kg O2/d')

oxygen_demand_Q_average=round((Q_total_projected_average*(BOD_primary_clarifier-BOD_required_to_effluent))/f/1000-1.42*rate_generation_biomass
text=[str(round(oxygen_demand_Q_average,1))]
keyword='py_OxygenAvg'

print('Oxygen demand for Q average is: ' +str(oxygen_demand_Q_average)+ ' kg O2/d')

air_required_peak=round(oxygen_demand_Q_peak/0.279/0.1*2/24,2) #kg O2/h, this is the value of the air saturation in the water.
text=[str(round(air_required_peak,1))]
keyword='py_air_peak'

print('Air required for Q peak is: ' +str(air_required_peak)+ ' m3 air/h')

air_required_typical=round(oxygen_demand_Q_average/0.279/0.1,2) #kg O2/h, this is the value of the air saturation in the water.
text=[str(round(air_required_typical,1))]
keyword='py_air_avg'

print('Air required for Q average is: ' +str(air_required_typical)+ ' m3 air/day')

```

 The BOD removal efficiency is: 0.9318
BOD primary clarifier is: 263.93 mg/L


```

The MLSS is: 4000 mg/L
SVI is: 91 mL/g
The return sludge concentration is: 10989.01 mg/L
The volumen settled is: 364.0 mL/L
The return sludge flow is: 65505.3 m3/d
The return sludge ratio is: 0.57
aereation volumen is: 18879.96 m3
aereation time is: 3.96 hours
BOD load is: 1600.0 g/m3/day
specific utilisation ratio is: 0.37
sludge age is: 7.91 days
sludge age increased is: 10.28
flow waste with a sludge age is: 868.8123967771864 m3/d
flow waste increased with a sludge age increased is: 668.51 m3/d
flow effluent with a sludge age is: 113585.51 m3/d
flow effluent with a sludge age increased is: 113785.81 m3/d

maximum volumen cmas tank is: 12960 m3
each aereation volumen tank is: 3146.66 m3
cmas tank depth is: 4.2 m
cmas tank width is: 16.8 m
cmas tank length is: 45 m

area secondary clarifier is: 4991.480066666667 m2
solids loading ratio average projected flow is: 5.795298858651155 kg/m2/h

Generation of biomass rate for Q average is: 9057.62 m3/d
Generation of biomass rate for Q peak is: 8975.69 m3/d
Oxygen demand for Q peak is: 31783.61 kg O2/d
Oxygen demand for Q average is: 29398.47 kg O2/d
Air required for Q peak is: 94933.12 m3 air/h
Air required for Q average is: 1053708.6 m3 air/day

```

```
#Initial information Environmental impact assessment.
```

```
#Flow receiving waterway according to the group number given in the briefing.
```

```
flow_receiving_waterway=round((group_number**0.2*300000),3) #m3/day, this is a value taken from the briefing.
print('Flow receiving waterway: ' +str(flow_receiving_waterway)+ ' m3/day')
```

```
print('Flow discharge waterway: ' +str(flow_effluent_increased)+ ' m3/day')
```

```
#BOD5 receiving waterway directly upstream from discharge point (Lr).
```

```
bod_upstream_receiving_waterway= 4 #mg/L, this is a value taken from the briefing.
```

```
saturation_dissolved_oxygen=8.5 #mg/L, this is a value taken from the briefing.
```

```
#Initial dissolve oxygen deficit in the receiving waterway (D0).
```

```
initial_disolved_oxygen_deficit=0.8 #mg/L, this is a value taken from the briefing.
```

```
#Reaeration rate constant base e (kr).
```

```
reaeration_rate_constant=0.71 #1/day, this is a value taken from the briefing.
```

```
#desoxygenation rate constant base e (kd).
```

```
desoxygenation_rate_constant=0.51 #1/day, this is a value taken from the briefing.
```

```
#stream velocity
```

```
stream_velocity=0.07 #m/s, this is a value taken from the briefing.
```

```
#disolved oxygen needs to be greater than 70% of saturation. According to ENVIRONMENT REFERENCE STANDARD for Victoria.
```

```
#This is for a surface waterway located in the north west of Melbourne.
```

```
minimum_dissolved_oxygen=round(0.7*saturation_dissolved_oxygen,2) #mg/L.
```

```
print('Minimum dissolved oxygen: ' +str(minimum_dissolved_oxygen)+' mg/L')
```

```
maximum_dissolved_oxygend_deficit=saturation_dissolved_oxygen-minimum_dissolved_oxygen #mg/L,
```

```
print('Maximum dissolved oxygen deficit: ' +str(maximum_dissolved_oxygend_deficit)+' mg/L')
```

```
max_bod_5_effluent=18 #mg/L, this is a value taken from EPA guidelines (Lw).
```

```
first_reaction_order_constant=0.23 #1/day, this is a value taken book reference Mcalf eddy pag 118-119.
```

```
bod_ultimate_effluent=round((max_bod_5_effluent/(1-math.exp(-first_reaction_order_constant*5))),2) #mg/L, this is a value taken from the br:
```

```
print('BOD ultimate effluent: ' +str(bod_ultimate_effluent)+' mg/L')
```

```
#This is the equation for the BOD ultimate mixing zone (Lo)
```

```
bod_ultimate_mixing_zone=round((flow_receiving_waterway*bod_upstream_receiving_waterway+flow_effluent_increased*bod_ultimate_effluent)/(flow_receiving_waterway+flow_effluent_increased),2) #mg/L,
```

```
print('BOD ultimate mixing zone: ' +str(bod_ultimate_mixing_zone)+' mg/L')
```

```
#time at which DO will reach minimum level (tc).
```

```

time_minimum_dissolved_oxygen=1/(reaeration_rate_constant-desoxygenation_rate_constant)*math.log(reaeration_rate_constant/desoxygenation_rate_constant)

print('Time at which DO will reach minimum level: ' +str(time_minimum_dissolved_oxygen)+' days')

#In order to calculate the length of the minimum DO level, we use (tc) and the stream velocity (v).

length_minimum_DO_level=round(time_minimum_dissolved_oxygen*86400*stream_velocity,2) #m, this is a value taken from the briefing.
print('Length of the minimum DO level: ' +str(length_minimum_DO_level)+' m')

#Dissolve oxygen at the time of minimum level (DOt).
dissolved_oxygen_deficit_at_time_minimum_level=((desoxygenation_rate_constant*bod_ultimate_mixing_zone)/(reaeration_rate_constant-desoxygenation_rate_constant))
print('Dissolved oxygen deficit at the time of minimum level: ' +str(dissolved_oxygen_deficit_at_time_minimum_level)+' mg/L')

if dissolved_oxygen_deficit_at_time_minimum_level>maximum_dissolved_oxygen_deficit:
    print('The dissolved oxygen deficit at the time of minimum level is greater than the maximum dissolved oxygen deficit.')
else:
    print('The dissolved oxygen deficit at the time of minimum level is less than the maximum dissolved oxygen deficit therefore it is suitable.')

#This calculation find the length where the dissolved oxygen is the same as the saturation dissolved oxygen.

```

```

↻ Flow receiving waterway: 475467.958 m3/day
Flow discharge waterway: 113785.81 m3/day
Minimum dissolved oxygen: 5.95 mg/L
Maximum dissolved oxygen deficit: 2.55 mg/L
BOD ultimate effluent: 26.34 mg/L
BOD ultimate mixing zone: 7.33 mg/L
Time at which DO will reach minimum level: 1.4355555448886719 days
Length of the minimum DO level: 8682.24 m
Dissolved oxygen deficit at the time of minimum level: 2.5319454091655578 mg/L
The dissolved oxygen deficit at the time of minimum level is less than the maximum dissolved oxygen deficit therefore it is suitable.

```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

```

```

def solve_for_time(target_deficit, desoxygenation_rate_constant, reaeration_rate_constant,
                  bod_ultimate_mixing_zone, initial_dissolved_oxygen_deficit):
    """
    Solves the Streeter-Phelps equation for time when dissolved oxygen deficit equals target value.

    The equation is:
    
$$D(t) = (K_1 L_0) / (K_2 - K_1) [e^{-(K_1 t)} - e^{-(K_2 t)}] + D_0 e^{-(K_2 t)} = \text{target\_deficit}$$


    where:
    D(t) = dissolved oxygen deficit at time t
    K1 = desoxygenation_rate_constant
    K2 = reaeration_rate_constant
    L0 = bod_ultimate_mixing_zone
    D0 = initial_dissolved_oxygen_deficit

    Args:
    target_deficit: Target dissolved oxygen deficit value
    desoxygenation_rate_constant: Deoxygenation rate constant (/day)
    reaeration_rate_constant: Reaeration rate constant (/day)
    bod_ultimate_mixing_zone: Ultimate BOD in mixing zone (mg/L)
    initial_dissolved_oxygen_deficit: Initial dissolved oxygen deficit (mg/L)

    Returns:
    Time(s) at which DO deficit equals target value
    """
    # Define the Streeter-Phelps equation as a function to find roots
    def deficit_equation(t):
        # First term (BOD effect)
        bod_effect = (desoxygenation_rate_constant * bod_ultimate_mixing_zone) / (reaeration_rate_constant - desoxygenation_rate_constant)
        bod_effect *= (np.exp(-desoxygenation_rate_constant * t) - np.exp(-reaeration_rate_constant * t))

        # Second term (initial deficit effect)
        initial_effect = initial_dissolved_oxygen_deficit * np.exp(-reaeration_rate_constant * t)

        # Complete equation (deficit at time t minus target deficit)
        return bod_effect + initial_effect - target_deficit

    # Find critical point (maximum deficit) first
    # This is important because deficit curve typically increases then decreases

```

```

t_values = np.linspace(0, 20, 1000)
deficits = np.zeros_like(t_values)

for i, t in enumerate(t_values):
    deficits[i] = deficit_equation(t) + target_deficit # Add target to get actual deficit

max_idx = np.argmax(deficits)
max_deficit = deficits[max_idx]
critical_time = t_values[max_idx]

print(f"Maximum deficit = {max_deficit:.4f} mg/L at t = {critical_time:.4f} days")

# Check if target deficit is achievable
if target_deficit > max_deficit:
    print(f"Target deficit ({target_deficit}) is higher than maximum deficit ({max_deficit:.4f})")
    print("No solution exists")
    return None

# Initialize results list
results = []

# Check if initial deficit equals target
if abs(initial_dissolved_oxygen_deficit - target_deficit) < 1e-6:
    results.append(0) # Time is 0 when initial equals target
    print(f"Initial deficit already equals target ({target_deficit})")

# Find time when deficit equals target (before critical point)
if initial_dissolved_oxygen_deficit < target_deficit < max_deficit:
    # Use numerical solver with initial guess halfway to critical point
    t_sol1 = fsolve(deficit_equation, critical_time/2)[0]

    # Verify solution
    if t_sol1 > 0 and abs(deficit_equation(t_sol1)) < 1e-6:
        results.append(t_sol1)
        print(f"Deficit reaches {target_deficit} at t = {t_sol1:.4f} days (rising limb)")

# Find time when deficit equals target (after critical point)
if target_deficit < max_deficit:
    # Use numerical solver with initial guess past critical point
    t_sol2 = fsolve(deficit_equation, critical_time + 2)[0]

    # Verify solution
    if t_sol2 > critical_time and abs(deficit_equation(t_sol2)) < 1e-6:
        results.append(t_sol2)
        print(f"Deficit reaches {target_deficit} at t = {t_sol2:.4f} days (falling limb)")

# Plot the results
plt.figure(figsize=(12, 8))

# Plot deficit curve
plt.plot(t_values, deficits, 'b-', linewidth=3, label='DO Deficit')

# Mark critical point
plt.plot([critical_time], [max_deficit], 'ro', markersize=10)
plt.annotate(f'Maximum: {max_deficit:.2f} mg/L',
            xy=(critical_time, max_deficit),
            xytext=(critical_time+1, max_deficit+0.1),
            arrowprops=dict(facecolor='red', shrink=0.05),
            fontsize=12)

# Mark target deficit
plt.axhline(y=target_deficit, color='g', linestyle='--', alpha=0.7,
            label=f'Target Deficit = {target_deficit}')

# Mark solutions
for t_sol in results:
    plt.plot([t_sol], [target_deficit], 'go', markersize=10)
    plt.annotate(f't = {t_sol:.2f}',
                xy=(t_sol, target_deficit),
                xytext=(t_sol, target_deficit - 0.15),
                ha='center', fontsize=12, fontweight='bold')
    plt.vlines(x=t_sol, ymin=0, ymax=target_deficit, color='g', linestyle=':', alpha=0.5)

# Plot settings
plt.title('Dissolved Oxygen Deficit - Streeter-Phelps Model', fontsize=16, fontweight='bold')
plt.xlabel('Time (days)', fontsize=14)
plt.ylabel('DO Deficit (mg/L)', fontsize=14)

```

```

plt.grid(True, alpha=0.3)
plt.legend(loc='best', fontsize=12)
plt.ylim(bottom=0)
plt.xlim(0, min(max(results)*2 if results else 10, 20))

# Add parameter information
param_text = (f'Parameters:\n'
              f'K1 = {desoxygenation_rate_constant} /day\n'
              f'K2 = {reaeration_rate_constant} /day\n'
              f'Lo = {bod_ultimate_mixing_zone} mg/L\n'
              f'Do = {initial_disolved_oxygen_deficit} mg/L')

plt.figtext(0.02, 0.02, param_text, fontsize=12,
           bbox=dict(facecolor='white', alpha=0.8, boxstyle='round,pad=0.5'))
y0, y1 = plt.ylim()      # get current (bottom, top)
plt.ylim(y1, y0)         # swap them
plt.tight_layout()
plt.show()

return results

# Example usage with the specified parameters
if __name__ == "__main__":

    # Target deficit to solve for
    target_deficit = 0.8 # Same as initial

    print(f"Solving for time when deficit = {target_deficit} mg/L:")
    result_times = solve_for_time(target_deficit, desoxygenation_rate_constant,
                                  reaeration_rate_constant, bod_ultimate_mixing_zone,
                                  initial_disolved_oxygen_deficit)

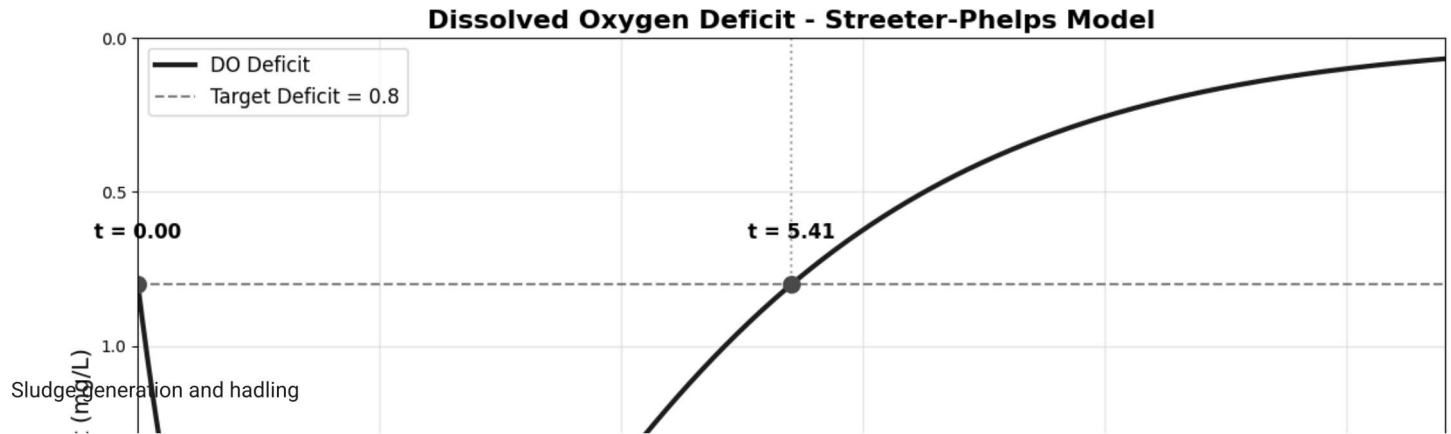
    # Print formatted results
    if result_times:
        if len(result_times) == 1:
            print(f"\nTime at which DO deficit reaches {target_deficit} mg/L: {result_times[0]:.6f} days")
        else:
            print(f"\nDO deficit reaches {target_deficit} mg/L at multiple times:")
            for i, t in enumerate(result_times):
                print(f"  Solution {i+1}: t = {t:.6f} days")

            lenght_same_DO_level=round(result_times[1]*86400*stream_velocity,2) #m, this is a value taken from the briefing.
            print('Length when it is reached the same DO level: ' +str(lenght_same_DO_level)+' m')

    else:
        print(f"\nNo solution found for target deficit = {target_deficit} mg/L")

```

→ Solving for time when deficit = 0.8 mg/L:
 Maximum deficit = 2.5319 mg/L at $t = 1.4414$ days
 Initial deficit already equals target (0.8)
 Deficit reaches 0.8 at $t = 5.4099$ days (falling limb)



```
#BOD_required_to_effluent g/m3, Xe
SRT=sludge_age #days, at Q average
tss_vss_primary_effluent_percentage=85 %%volatile, We suppose this value since we have a domestic and comercial wastewater.
#Therefore, it is not expected a large amount of non-biodegradable solids.
vss_primary_effluent_percentage=30 %% non-biodegradable. Same reason as above.
primary_sludge_solids_content= 7 %% This value is taken from table 18-8 Metcalf & Eddy. It ranges from 2-8% for primary sludge.
secondary_sludge_solids_content= 2 %% This value is taken from table 18-9 Metcalf & Eddy. It ranges from 0.4-2.5% for primary sludge.
specific_gravity_both_sludges=1.03 #dimensionless, this is the specific gravity of both sludges. Table 5.21 Metcalf & Eddy.
specific_weight_water=1000 #kg/m3\n",
```

```
peak_flow_rate=Q_new_peak #m3/day
average_flow_rate=Q_total_projected_average #m3/day
```

```
tss_peak_raw=tss_concentration #mg/l\n",
tss_average_raw=tss_concentration #mg/l\n",
```

```
tss_average_effluent=tss_concentration-0.5114*tss_concentration #mg/l #This value is taken from primary clarifier design.
tss_peak_effluent=tss_concentration-0.5882*tss_concentration #mg/l #This value is taken from primary clarifier design.
```

```
#Primary sludge volume
```

```
primary_sludge_production_average=round(average_flow_rate*(tss_average_raw-tss_average_effluent)/1000,2)
print ('Average flow primary sludge in kg is: '+str(primary_sludge_production_average))
```

✓ Table 1. Useful equations.

Number	Equation	Description
1	$SLR = \frac{Q}{A}$	SLR : Surface Loading Rate Q : inflow A : surface area
2	$t = \frac{Volume}{Q} = \frac{H}{SLR}$	t : detention time Volume : Volume Q : inflow H : water depth in the tank SLR : Surface Loading Rate
3	$Q_{perV-notch} = \frac{8}{15} C_d \sqrt{2g} \tan \frac{\theta}{2} h^{\frac{5}{2}}$	$Q_{perV-notch}$: flow over v-notch C_d : discharge coefficient g : acceleration due to gravity θ : angle of the v-notch h : maximum height of water over weirs
4	$Q_{perV-notch} = \frac{Q}{N}$	$Q_{perV-notch}$: flow over v-notch Q : inflow N : number of v-notches
5	$v_h = \frac{Q}{WH} = \frac{L}{t}$	v_h : horizontal velocity Q : inflow W : tank width H : tank depth t : detention time L : tank length
6	$v_H = \sqrt{\frac{8k(s-1)gd}{f}}$	v_H : scour velocity k : scoured material constant s : spec. gravity of particles g : acceleration due to gravity d : particle diameters f : Darcy-Weisbach friction factor
7	$y_c = \left[\frac{(qL)^2}{4b^2g} \right]^{\frac{1}{3}}$	y_c : critical depth q : discharge per unit launder length L : the length of the launder b : the width of the launder g : acceleration due to gravity
8	$H = \left(y_c^2 + \frac{2q^2x^2}{gb^2y_c} \right)^{\frac{1}{2}}$	H : total head y_c : critical depth q : discharge per unit launder length x : distance (L/2 for a circular basin) g : acceleration due to gravity b : the width of the launder
9	$R = \frac{t}{a + bt}$	R : efficiency in % t : residence time in hours a : parameter b : parameter

Standard Parameters (Table P.C.1)

Constituent	a	b
BOD ₅	0.018	0.020
SS	0.0075	0.014

```
# Design Guidelines Checklist

import pandas as pd
import numpy as np

check_list = [
    ['Detention time, t', '1.5 - 2.5 hr', '-'],
    ['Surface Loading Rate, SLR (ADW)', '30 - 50 m³/m²/day', '-'],
    ['Surface Loading Rate, SLR (PHR)', '60 - 120 m³/m²/day', '-'],
    ['Clarifier Diameter', '3 - 60 m', '-'],
    ['Sidewater depth, H', '3.0 - 5.0 m', '-'],
    ['Weir Loading Rate, WLR', '125 - 500 m³/m/day', '-']
]

headers = ["Parameter", "Typical Value", "Notes"]

# Create a DataFrame with data checklist
df = pd.DataFrame(check_list, columns=headers)
# During the design process the items contained within the DataFrame will be marked as "OK"

'''Functions'''

# Weir length for circular clarifiers (Lw)
def weir_length_cirtank(n_side, tank_diameter, dist_cent_edge):
    """
    Calculates the weir length for circular clarifiers.

    Args:
        n_side (int): Number of sides of the launder (1 or 2).
```

```

    tank_diameter (float): Diameter of the circular tank in meters.
    dist_cent_edge (float): Distance from edge of clarifier to centerline of launder (m)

Returns:
    float: The weir length in meters, or None if n_side is invalid.
"""
if not isinstance(tank_diameter, (int, float)) or not isinstance(dist_cent_edge, (int, float)) or not isinstance(n_side, (int, float)):
    print("Error: All arguments must be a number.")
    return None

if n_side == 1:
    launder_length = np.pi * tank_diameter # Launder Length "m"
    weir_length = launder_length # Weir Length "m"
    return weir_length

elif n_side == 2:
    while True:
        dist_str = input("Distance from edge of clarifier to centerline of launder (m): ")
        # User must provide the centerline distance from edge
        try:
            dist_launder_centerline_edge = float(dist_str)
            break # Exit the loop if input is valid
        except ValueError:
            print("Invalid input. Please enter a valid number for the distance.")
    launder_length = np.pi * (tank_diameter - 2 * dist_cent_edge) # Launder Length "m"
    weir_length = n_side * launder_length # Weir Length "m"
    return weir_length
else:
    print("The sides of the launder must be defined as 1 or 2")
    return None

# Maximum water height over the weirs (bottom of the notch) (h).
def max_height_water_weirs (Q_notch, Cd, gravity, theta):
    """
    Calculates the maximum water height over the weirs.

    Args:
        Q_notch (float): Flow rate per V-notch.
        Cd (float): Discharge coefficient depending on the weir geometry and flow conditions, typically 0.58.
        Gravity (float): Gravity acceleration = 9.81 m/s², usually.
        theta (float): Angle of the V-notch in degrees.

    Returns:
        h (float): Head of water above the bottom of the notch in meters, or None if any argument is invalid.
    """
    # Convert theta degrees to radians
    theta_rad = np.deg2rad(theta)
    h = (Q_notch/(8/15 * Cd * np.sqrt( 2 * gravity ) * np.tan(theta_rad/2) ) )**(2/5)
    return h

# Critical water height at launder discharge point (Yc)
def critical_Yc (q_WLR, launder_length, launder_width, gravity):
    """
    Calculates the maximum water height over the weirs.

    Args:
        q_WLR (float): Discharge per unit launder length (m³/m/s).
        launder_length (float): Length of the launder, for circular tanks x = launder_length/2 (m).
        launder_width (float): Width of the launder (m).
        Gravity (float): Gravity acceleration = 9.81 m/s², usually.

    Returns:
        Yc (float): Critical water height at launder discharge point in meters, or None if any argument is invalid.
    """
    if not isinstance(q_WLR, (int, float)) or not isinstance(launder_length, (int, float)) or not isinstance(launder_width, (int, float)) or not isinstance(gravity, (int, float)):
        print("Error: the input arguments must be a number.")
        return None

    # Determine the critical water height (Yc) at discharge point.
    Yc = (((q_WLR * launder_length/2)**2) / (4 * (launder_width**2) * gravity))**(1/3)
    return Yc

# Maximum depth of water into the launder (H)
def max_water_launder_H (q_WLR, x_water, launder_width, gravity, crit_Yc):
    """

```

Calculates the maximum water height over the weirs.

Args:

q_WLR (float): Discharge per unit launder length (m³/m/s).
 x_water (float): farthest distance water travels (m).
 launder_width (float): Width of the launder (m).
 Gravity (float): Gravity acceleration = 9.81 m/s², usually.
 Yc (float): Critical water height at launder discharge point (m).

Returns:

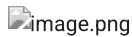
H (float): Maximum depth of water into the launder in meters, or None if any argument is invalid.

"""

```
if not isinstance(q_WLR, (int, float)) or not isinstance(x_water, (int, float)) or not isinstance(laundry_width, (int, float)) or not i:
    print("Error: the input arguments must be a number.")
    return None
```

Determine the maximum depth of water into the launder.

```
H = (crit_Yc**2 + ((2* (q_WLR**2) * (x_water)**2) / (gravity * (launder_width**2) * crit_Yc)))**(1/2)
return H
```



'''First Trial'''
 """

The analysis presented below is based on the following assumptions:

- The design flow rate is based on the projected average dry weather flow rate (Q_ADW) and the projected maximum hourly flow rate (Q_PHR).

"""

0. Primary information:

```
Q_total_current_average = 74706.08 # Current Average Dry Weather flow rate (m³/day)
Q_total_projected_maximum = 193226 # (Peak) Maximum hourly flow rate (m³/day)
```

```
Q_ADW = Q_total_current_average/24 # Current Average Dry Weather flow rate (m³/hr)
Q_PHR = Q_total_projected_maximum/24 # (Peak) Maximum hourly flow rate (m³/hr)
```

```
print(f"Projected Average Dry Weather flow rate (Q_ADW): {Q_ADW:.2f} m³/hr")
print(f"(Peak) Maximum hourly flow rate (Q_PHR): {Q_PHR:.2f} m³/hr\n")
```

```
# BOD5 concentration in the influent (mg/L)
BOD5_influent = 370 # BOD5 concentration in the influent (mg/L)
```

```
# BOD5 concentration in the effluent (mg/L)
BOD5_effluent = 263.9 # BOD5 concentration in the effluent (mg/L)
```

```
# BOD5 removal efficiency
R_BOD = (BOD5_influent - BOD5_effluent) / BOD5_influent *100 # BOD5 removal efficiency (%)
print(f"BOD5 removal efficiency: {R_BOD:.2f}%")
```

1. Determine the time of residence (t) according to the efficiency of BOD removal and determine
 # the efficiency of the the suspended solids (SS) removal from detention time (t_in).

'''From Table P.C.1 "Standard parameter" the time of residence for the SS removal and BOD removal is as follows:'''

```
a_BOD = 0.018
b_BOD = 0.020
```

```
t_in = R_BOD*a_BOD/(1-R_BOD*b_BOD) # Time of residence (hours)
print(f"Time of residence (t): {t_in:.2f} hours")
```

To achieve the same sedimentation performance as achieved at 20°C is necessary to increase the time of residence (t) by a multiplier factor
 # which is a function of the temperature of the water. The multiplier factor is given by the following equation:

```
temperature = 10 # Temperature in degrees Celsius
multiplier_factor = 1.82 * np.exp(-0.03 * temperature) # Multiplier factor
print(f"Multiplier factor for temperature {temperature:.0f}°C: {multiplier_factor:.2f}")
```

```
# Calculate the adjusted time of residence (t) using the security factor for lower temperatures
t_adjusted = t_in * multiplier_factor # Adjusted time of residence (hours)
#print(f"Adjusted time of residence (t) at {t_adjusted:.0f}°C of water temperature: {t_adjusted:.2f} hours")
```

Determine the suspended solids (SS) removal efficiency from the detention time (t)
 # Given paramaters for SS removal efficiency.

```
a_SS = 0.0075
b_SS = 0.014
```



```

# Calculate the suspended solids (SS) removal efficiency using the detention time (t)
#R_SS = t_adjusted/(a_SS + b_SS * t_adjusted) # SS removal efficiency (%)
#print(f"Suspended solids (SS) removal efficiency: {R_SS:.2f}%")

print(f"Adjusted time of residence (t) at {temperature:.0f}°C of water temperature: {t_adjusted:.2f} hours")

↳ Projected Average Dry Weather flow rate (Q_ADW): 3112.75 m³/hr
(Peak) Maximum hourly flow rate (Q_PHR): 8051.08 m³/hr

BOD5 removal efficiency: 28.68%
Time of residence (t): 1.21 hours
Multiplier factor for temperature 10°C: 1.35
Adjusted time of residence (t) at 10°C of water temperature: 1.63 hours

# The time of residence is equal to 1.86 hours.

# 2. The total volume of the clarifier tanks is determined by the detention time (t) and the peak hourly
# flow rate (Q_PHR) using the following equation (Equation 1 from Table P.C.2):

V_total = Q_PHR * t_adjusted # Total volume given by the time of residence (t "h") over the peak flow conditions (Q_PHR "m³/hr")
print(f"Total volume of clarifier tanks is: {V_total:.2f} m³")

# 3. It's checked that the detention time is not less than 1.5 hours or greater than 2.5 hours (see guidelines Table P.C.2) for average dry

# It's obvious that the detention time to be determined is not less than 1.5 hours due to the fact that the volume of the clarifier tanks is
# with t = 1.82 hours and knowing that the average dry flow rate (Q_ADW) is lower than (Q_PHR). Hence, the detention time by average dry wei

t_ADW = V_total / Q_ADW # Detention time for average dry weather flow rate (hours)
print(f"Detention time for average dry weather flow rate: {t_ADW:.2f} hours")

↳ Total volume of clarifier tanks is: 13137.67 m³
Detention time for average dry weather flow rate: 4.22 hours

# The detention time for average dry weather flow rate is equal to 4.04 hours, which is greater than 2.5 hours.
# Thus, using the maximum detention time for Q_ADW allowed by the guidelines, the volume of the clarifier
# tanks is recalculated using the maximum detention time of 2.5 hours:

t_ADW_max = 2.5 # Maximum detention time for average dry weather flow rate (hours)
V_ADW_max = Q_ADW * t_ADW_max # Volume of clarifier tanks using maximum detention time (h) for Q_ADW (m³/h) conditons (m³).
print(f"Volume of clarifier tanks (using maximum detention time for ADW): {V_ADW_max:.2f} m³")

# 4. The excess volume of the clarifier tanks is calculated as follows:

V_excess = V_total - V_ADW_max # Excess volume of clarifier tanks (m³)
# This volume will be determined as the volume for the equalization tank, which is used to balance the flow rate and the volume of the clar:
print(f"Excess volume of clarifier tanks (equalization tank volume): {V_excess:.2f} m³\n")

↳ Volume of clarifier tanks (using maximum detention time for ADW): 7781.88 m³
Excess volume of clarifier tanks (equalization tank volume): 5355.78 m³

'''In this point just the time of residence or detention time (t) meets the design guidelines (Table. P.C.2)'''

# Let's change the status in the last column of the check list regarding "detention time"
df.iloc[0, 2] = 'OK'
print(df)
print('\n')

# 5. Having the new volume of the clarifier tanks, a new peak hourly flow rate is calculated using the new volume of
# the clarifier tanks and the maximum detention time for peak hourly flow condition, meeting the minimum
# removal efficiency for SS and BOD removal requirements.

# The new peak hourly flow rate is calculated as follows:

Q_PHR_new = V_ADW_max / t_adjusted # New peak hourly flow rate (m³/hr)
print(f"New peak hourly flow rate (Q_PHR*): {Q_PHR_new:.2f} m³/hr")

print(f"Average dry weather flow rate (Q_ADW): {Q_ADW:.2f} m³/hr")
print("\n")

# The detention time for average dry weather flow rate is equal to 2.5 hours, which is the maximum detention time allowed by the guidelines
# Additionally, the detention time for the peak hourly flow rate is less than to 2.5 hours (1.82 hours).

"""Then, the detention times both for average and peak flow rates meet the design guidelines after the equalization tank"""

t_ADW_ = t_ADW_max # Detention time for average dry weather flow rate (hours) at 10°C

```

```
t_PHR_ = t_adjusted # Detention time for peak hourly flow rate (hours) at 10°C
print(f"Detention time for average dry weather flow: {t_ADW_:.2f} hours")
print(f"Detention time for peak hourly flow rate: {t_PHR_:.2f} hours")
print("\n")

'''The removal efficiency for TSS and BOD removal for both detention times (t_ADW and t_PHR) are presented below:'''
'''These removal efficiencies are guaranteed during the primary sedimentation process (after the equalization tank)'''

# The removal efficiency for TSS and BOD for detention time (t_PHR) over the peak hourly flow rate (Q_PHR) was determined above.

# The real removal efficiency corresponds to 20°C, which is the temperature of the water in the clarifier tanks.
# Hence, the detention time must be corrected to determine the real removal efficiency for TSS and BOD.
# The correction is done just dividing the detention time by the multiplier factor (1.82) for the temperature of 10°C.

print("For Average Dry Weather Flow Rate (ADW):")
t_ADW_20 = t_ADW_ / multiplier_factor # Detention time for average dry weather flow rate (hours) at 20°C
R_BOD_ADW = t_ADW_20 / (a_BOD + b_BOD * t_ADW_20) # BOD5 removal efficiency (%)
print(f"BOD5 removal efficiency for detention time (t_ADW_ = {t_ADW_:.2f} h at {temperature:.0f}°C) is: {R_BOD_ADW:.2f}%")
R_SS_ADW = t_ADW_20 / (a_SS + b_SS * t_ADW_20) # SS removal efficiency (%)
print(f"TSS removal efficiency for detention time (t_ADW_ = {t_ADW_:.2f} h at {temperature:.0f}°C) is: {R_SS_ADW:.2f}%\n")

print("For Peak Hourly Flow Rate (PHR*):")
# These removal efficiencies are known (previously calculated)
t_PHR_20 = t_PHR_ / multiplier_factor # Detention time for peak hourly flow rate (hours) at 20°C
R_BOD_PHR = t_PHR_20 / (a_BOD + b_BOD * t_PHR_20) # BOD5 removal efficiency (%)
print(f"BOD5 removal efficiency for detention time (t_PHR_ = {t_PHR_:.2f} h at {temperature:.0f}°C) is: {R_BOD_PHR:.2f}%")
R_SS_PHR = t_PHR_20 / (a_SS + b_SS * t_PHR_20) # SS removal efficiency (%)
print(f"TSS removal efficiency for detention time (t_PHR_ = {t_PHR_:.2f} h at {temperature:.0f}°C) is: {R_SS_PHR:.2f}%")
```

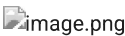
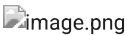
	Parameter	Typical Value	Notes
0	Detention time, t	1.5 - 2.5 hr	OK
1	Surface Loading Rate, SLR (ADW)	30 - 50 m³/m²/day	-
2	Surface Loading Rate, SLR (PHR)	60 - 120 m³/m²/day	-
3	Clarifier Diameter	3 - 60 m	-
4	Sidewater depth, H	3.0 - 5.0 m	-
5	Weir Loading Rate, WLR	125 - 500 m³/m/day	-

New peak hourly flow rate (Q_PHR*): 4768.93 m³/hr
Average dry weather flow rate (Q_ADW): 3112.75 m³/hr

Detention time for average dry weather flow: 2.50 hours
Detention time for peak hourly flow rate: 1.63 hours

For Average Dry Weather Flow Rate (ADW):
BOD5 removal efficiency for detention time (t_ADW_ = 2.50 h at 10°C) is: 33.66%
TSS removal efficiency for detention time (t_ADW_ = 2.50 h at 10°C) is: 55.42%

For Peak Hourly Flow Rate (PHR*):
BOD5 removal efficiency for detention time (t_PHR_ = 1.63 h at 10°C) is: 28.68%
TSS removal efficiency for detention time (t_PHR_ = 1.63 h at 10°C) is: 49.51%



```
# 6. The Surface Loading Rate (SLR) is calculated for both cases, average dry flow (ADW) and peak flow conditions (PHR)

# It's determined the surface area from the volume of the clarifier tank

# In this scenario, it is assumed to work with one clarifier tank:

# Sidewater depth (m) (Table 21-1), it will have a freeboard of 0.5m.
H_clarifier = 5 # Total tank depth = 5.0 + 0.5 = 5.5m
print(f"Sidewater depth: {H_clarifier:.2f} m")

V_tanks = V_ADW_max # Volume of the clarifier tank (m³)
print(f"Volume of clarifier tank: {V_tanks:.2f} m³")

# The area of the clarifier tank is calculated as follows:
A_clarifiers = V_tanks / H_clarifier
```

```

print(f'Area of clarifier tank: {A_clarifiers:.1f} m²')

# The diameter of the clarifier tank is calculated as follows:
D_clarifiers = np.sqrt(4 * A_clarifiers / np.pi) # Diameter of the clarifier tanks (m)
print(f'Diameter of clarifier tank: {D_clarifiers:.1f} m\n')

'''The diameter of the clarifier tanks is equal to 46.3m (below of Dmax = 60m), which is within the design requirements (Table 5-20)'''
'''It means that for this trial only will be needed one clarifier tank'''

# From Equation 1 in Table P.C.2

# The SLR for PHR is:
SLR_PHR = (Q_PHR_new / A_clarifiers) * 24 # SLR for PHR (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for peak flow conditions is: {SLR_PHR:.1f} m³/m²/day')
# The SLR for ADW is:
SLR_ADW = (Q_ADW / A_clarifiers) * 24 # SLR for ADW (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for average dry flow is: {SLR_ADW:.1f} m³/m²/day\n')

# Let's change the status in the last column of the checklist.
df.iloc[1, 2] = 'OK'
df.iloc[2, 2] = 'OK'
df.iloc[3, 2] = 'OK'
df.iloc[4, 2] = 'OK'
print(df)
print('\n')

"""One clarifier tank is used for the design, both SLR for AWD and PHR meet the design requirements (Table 21-1)"""

"""To check the weir loading rate (WLR) for the clarifier tank, it is necessary assume the critical condition
for the operation of the clarifier tanks. To analyze the critical condition, it is necessary to design the primary
treatment system composed by minimum two clarifier tanks. The critical condition occurs when at least one clarifier
tank is out of service. (reliability, redundancy, maintenance needs)"""

➡ Sidewater depth: 5.00 m
Volume of clarifier tank: 7781.88 m³
Area of clarifier tank: 1556.4 m²
Diameter of clarifier tank: 44.5 m

The Surface Loading Rate of the clarifier tank for peak flow conditions is: 73.5 m³/m²/day
The Surface Loading Rate of the clarifier tank for average dry flow is: 48.0 m³/m²/day



|   | Parameter                       | Typical Value      | Notes |
|---|---------------------------------|--------------------|-------|
| 0 | Detention time, t               | 1.5 - 2.5 hr       | OK    |
| 1 | Surface Loading Rate, SLR (ADW) | 30 - 50 m³/m²/day  | OK    |
| 2 | Surface Loading Rate, SLR (PHR) | 60 - 120 m³/m²/day | OK    |
| 3 | Clarifier Diameter              | 3 - 60 m           | OK    |
| 4 | Sidewater depth, H              | 3.0 - 5.0 m        | OK    |
| 5 | Weir Loading Rate, WLR          | 125 - 500 m³/m/day | -     |



'To check the weir loading rate (WLR) for the clarifier tank, it is necessary assume the critical condition\nfor the operation of the
clarifier tanks. To analyze the critical condition, it is necessary to design the primary\ntreatment system composed by minimum two
clarifier tanks. The critical condition occurs when at least one clarifier\ntank is out of service. (reliability, redundancy,
maintenance needs)'

"""Second Trial"""

print("Second Trial: Adding another clarifier tank\n")

# The design guidelines checklist is re-reseted:

df.iloc[0, 2] = '-'
df.iloc[1, 2] = '-'
df.iloc[2, 2] = '-'
df.iloc[3, 2] = '-'
df.iloc[4, 2] = '-'
print(df)
print('\n')

# 8. The number of clarifier tanks is determined:
n_clarifiers = 2 # Number of clarifier tanks

# The volume of the clarifier tank is calculated as follows:
V_per_clarifier = V_tanks / n_clarifiers # Volume of the clarifier tank (m³)
print(f'Volume of clarifier tank: {V_per_clarifier:.1f} m³')

# The area of the clarifier tank is calculated as follows:

```

```

A_per_clarifier = V_per_clarifier / H_clarifier # Area of the clarifier tank (m²)
print(f'Area of clarifier tank: {A_per_clarifier:.1f} m²')

# The diameter of the clarifier tank is calculated as follows:
D_per_clarifier = np.sqrt(4 * A_per_clarifier / np.pi) # Diameter of the clarifier tanks (m)
print(f'Diameter of clarifier tank: {D_per_clarifier:.1f} m\n')

'''The diameter of the clarifier tanks is equal to 32.7m (below of Dmax = 100m), which is within the design requirements (Table 5-20 and Tal
'''It means that the dimensions of the clarifier tanks are OK for the design.'''

"""The SLR condition is checked, but it's obvious that SLR meets the design requirements (Table 21-1) for both cases,"""
# 9. The SLR for PHR is:
SLR_PHR = (Q_PHR_new / (A_per_clarifier * n_clarifiers)) * 24 # SLR for PHR (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for peak flow conditions is: {SLR_PHR:.1f} m³/m²/day')
# The SLR for ADW is:
SLR_ADW = (Q_ADW / (A_per_clarifier * n_clarifiers)) * 24 # SLR for ADW (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for average dry flow is: {SLR_ADW:.1f} m³/m²/day\n')

# Let's change the status in the last column of the checklist.
df.iloc[0, 2] = 'OK'
df.iloc[1, 2] = 'OK'
df.iloc[2, 2] = 'OK'
df.iloc[3, 2] = 'OK'
df.iloc[4, 2] = 'OK'
print(df)
print('\n')

```

↺ Second Trial: Adding another clarifier tank

	Parameter	Typical Value	Notes
0	Detention time, t	1.5 - 2.5 hr	-
1	Surface Loading Rate, SLR (ADW)	30 - 50 m³/m²/day	-
2	Surface Loading Rate, SLR (PHR)	60 - 120 m³/m²/day	-
3	Clarifier Diameter	3 - 60 m	-
4	Sidewater depth, H	3.0 - 5.0 m	-
5	Weir Loading Rate, WLR	125 - 500 m³/m/day	-

Volume of clarifier tank: 3890.9 m³
Area of clarifier tank: 778.2 m²
Diameter of clarifier tank: 31.5 m

The Surface Loading Rate of the clarifier tank for peak flow conditions is: 73.5 m³/m²/day
The Surface Loading Rate of the clarifier tank for average dry flow is: 48.0 m³/m²/day

	Parameter	Typical Value	Notes
0	Detention time, t	1.5 - 2.5 hr	OK
1	Surface Loading Rate, SLR (ADW)	30 - 50 m³/m²/day	OK
2	Surface Loading Rate, SLR (PHR)	60 - 120 m³/m²/day	OK
3	Clarifier Diameter	3 - 60 m	OK
4	Sidewater depth, H	3.0 - 5.0 m	OK
5	Weir Loading Rate, WLR	125 - 500 m³/m/day	-

10. Weir Loading Rate (WLR) of the clarifier tanks

It's defined the number of clarifier tanks at critical condition as follows:

```

n_clarifiers_critical = n_clarifiers - 1 # Critical condition
print(f'Number of clarifier tanks at critical condition: {n_clarifiers_critical}\n')

```

First, the weir length (Lw) for circular tanks is determined using the function described at beginning of this notebook:

'''It's assumed that the clarifier tank launder is 2-sided and it has a distance from its centerline to the edge of the clarifier tank equal to 1.0m.'''

This dimension assumed will be used to calculate the weir length (Lw) for circular tanks.

```

sides_launder = 2 # 2-sided launder
dist_cent_edgetank = 1 # Distance from clarifier edge to centerline of the launder

```

```

# Applying the function defined at the beginning of this notebook to determine the weir length
weir_length = weir_length_circ tank(sides_launder, D_per_clarifier, dist_cent_edgetank) # 2-sided clarifier, tank_diameter = D_clarifier, c

```

```

# Then, the WLR is calculated from the next equation:
WLR = (Q_PHR_new * 24) / weir_length

```

```

# In the critical condition, WLR is determined as follows:
WLR_crit = WLR / n_clarifiers_critical # Weir Loading Rate (m³/m²/day)
print(f'The Weir Loading Rate of the clarifier tank is: {WLR_crit:.1f} m³/m/day\n')

"""The WLR calculated (618.0) m³/m/day) exceed the upper limit defined by the design requirements (500 m³/m²/day) (Table 21-1)"""

➡ Number of clarifier tanks at critical condition: 1

The Weir Loading Rate of the clarifier tank is: 618.0 m³/m/day

'The WLR calculated (618.0) m³/m/day) exceed the upper limit defined by the design requirements (500 m³/m²/day) (Table 21-1)'

'''Third Trial'''

print("Third Trial: Adding another clarifier tank to reduce WLR\n")

# The design guidelines checklist is re-reseted:

df.iloc[0, 2] = '-'
df.iloc[1, 2] = '-'
df.iloc[2, 2] = '-'
df.iloc[3, 2] = '-'
df.iloc[4, 2] = '-'
print(df)
print('\n')

# 11. Thus, another tank must be added to decrease the WLR for whole system

n_clarifiers = n_clarifiers + 1 # Number of clarifier tanks
print(f"Number of the clarifiers in this trial: {n_clarifiers:.0f} \n")

# The volume of the clarifier tank is calculated as follows:
V_per_clarifier = V_tanks / n_clarifiers # Volume of the clarifier tank (m³)
print(f'Volume of clarifier tank: {V_per_clarifier:.1f} m³')

# The area of the clarifier tank is calculated as follows:
A_per_clarifier = V_per_clarifier / H_clarifier # Area of the clarifier tank (m²)
print(f'Area of clarifier tank: {A_per_clarifier:.1f} m²')

# The diameter of the clarifier tank is calculated as follows:
D_per_clarifier = np.sqrt(4 * A_per_clarifier / np.pi) # Diameter of the clarifier tanks (m)
print(f'Diameter of clarifier tank: {D_per_clarifier:.1f} m\n')

'''The diameter of the clarifier tanks is equal to 25.7m (below of Dmax = 60m or 45m), which is within the design requirements (Table 5-20 :
'It means that the dimensions of the clarifier tanks are OK for the design.'''

"""The SLR condition is checked, but it's obvious that SLR meets the design requirements (Table 21-1) for both cases,"""

# 12. The SLR for PHR is:
SLR_PHR = (Q_PHR_new / (A_per_clarifier * n_clarifiers)) * 24 # SLR for PHR (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for peak flow conditions is: {SLR_PHR:.1f} m³/m²/day')
# The SLR for ADW is:
SLR_ADW = (Q_ADW / (A_per_clarifier * n_clarifiers)) * 24 # SLR for ADW (m³/m²/day)
print(f'The Surface Loading Rate of the clarifier tank for average dry flow is: {SLR_ADW:.1f} m³/m²/day\n')

# Let's change the status in the last column of the checklist.
df.iloc[0, 2] = 'OK'
df.iloc[1, 2] = 'OK'
df.iloc[2, 2] = 'OK'
df.iloc[3, 2] = 'OK'
df.iloc[4, 2] = 'OK'
print(df)
print('\n')

➡ Third Trial: Adding another clarifier tank to reduce WLR



|   | Parameter                       | Typical Value      | Notes |
|---|---------------------------------|--------------------|-------|
| 0 | Detention time, t               | 1.5 - 2.5 hr       | -     |
| 1 | Surface Loading Rate, SLR (ADW) | 30 - 50 m³/m²/day  | -     |
| 2 | Surface Loading Rate, SLR (PHR) | 60 - 120 m³/m²/day | -     |
| 3 | Clarifier Diameter              | 3 - 60 m           | -     |
| 4 | Sidewater depth, H              | 3.0 - 5.0 m        | -     |
| 5 | Weir Loading Rate, WLR          | 125 - 500 m³/m/day | -     |



Number of the clarifiers in this trial: 3

Volume of clarifier tank: 2594.0 m³

```

Area of clarifier tank: 518.8 m²
Diameter of clarifier tank: 25.7 m

The Surface Loading Rate of the clarifier tank for peak flow conditions is: 73.5 m³/m²/day
The Surface Loading Rate of the clarifier tank for average dry flow is: 48.0 m³/m²/day

	Parameter	Typical Value	Notes
0	Detention time, t	1.5 - 2.5 hr	OK
1	Surface Loading Rate, SLR (ADW)	30 - 50 m ³ /m ² /day	OK
2	Surface Loading Rate, SLR (PHR)	60 - 120 m ³ /m ² /day	OK
3	Clarifier Diameter	3 - 60 m	OK
4	Sidewater depth, H	3.0 - 5.0 m	OK
5	Weir Loading Rate, WLR	125 - 500 m ³ /m/day	-

13. Weir Loading Rate (WLR) of the clarifier tanks (for 3 clarifier tanks)

Again, it is defined the number of clarifier tanks at critical condition as follows:

```
n_clarifiers_critical = n_clarifiers - 1 # Critical condition
print(f'Number of clarifier tanks at critical condition: {n_clarifiers_critical}\n')
```

First, the weir length (Lw) for circular tanks is determined using the function described at beginning of this notebook:

```
'''It's assumed that the clarifier tank launder is 2-sided and it has a distance from its centerline to
the edge of the clarifier tank equal to 1.0m.'''
```

This dimension assumed will be used to calculate the weir length (Lw) for circular tanks.

```
sides_launder = 2 # 2-sided launder
dist_cent_edgetank = 1 # Distance from clarifier edge to centerline of the launder
```

```
# Applying the function defined at the beginning of this notebook to determine the weir length
weir_length = weir_length_circ tank(sides_launder, D_per_clarifier, dist_cent_edgetank) # 2-sided clarifier, tank_diameter = D_clarifier, c
```

Then, the WLR is calculated from the next equation:

```
WLR = (Q_PHR_new * 24) / weir_length
```

In the critical condition, WLR is determined as follows:

```
WLR_crit = WLR / n_clarifiers_critical # Weir Loading Rate (m3/m2/day)
print(f'The Weir Loading Rate of the clarifier tank is: {WLR_crit:.1f} m3/m/day\n')
```

```
"""The WLR calculated (384.3 m3/m/day) does not exceed the upper limit defined by the design requirements (500 m3/m2/day) (Table 21-1)"""
'''The design guidelines checklist is updated'''
```

```
df.iloc[5, 2] = 'OK'
print(df)
print('\n')
```

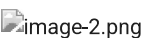
```
"""All the design requirements are met for the clarifier tanks, and the design is OK."""
```

➡ Number of clarifier tanks at critical condition: 2

The Weir Loading Rate of the clarifier tank is: 384.3 m³/m/day

	Parameter	Typical Value	Notes
0	Detention time, t	1.5 - 2.5 hr	OK
1	Surface Loading Rate, SLR (ADW)	30 - 50 m ³ /m ² /day	OK
2	Surface Loading Rate, SLR (PHR)	60 - 120 m ³ /m ² /day	OK
3	Clarifier Diameter	3 - 60 m	OK
4	Sidewater depth, H	3.0 - 5.0 m	OK
5	Weir Loading Rate, WLR	125 - 500 m ³ /m/day	OK

'All the design requirements are met for the clarifier tanks, and the design is OK.'

```
'''Design of V-Notches'''
```

```
print("Design of V-Notches\n")
```

The design of the V-Notches is based on the following assumptions:

1. The number of V-Notches is determined based on centre-centre spacing between them

Then, is assumed that the centre to centre spacing is 300mm.

```

S_v_notch = 0.3 # Centre to centre spacing between V-Notches (m)

# The number of V-Notches is calculated as follows:
N_notches = np.ceil(weir_length / S_v_notch)
print(f'The number of V-Notches per clarifier is: {N_notches:.0f}\n')

# 2. Calculation of the flow rate per V-Notch

# It's defined the total amount of V-Notches for all clarifiers as follows:
N_total_notches = N_notches * n_clarifiers # Clarifier tanks 3
print(f'The total number of V-Notches for all clarifiers is: {N_total_notches:.0f} \n')

# The flow per notch is determined based on the following equation (Equation 4 from Table P.C.2):
# It's based on Q_PHR_new, which is the new peak hourly flow rate (Q_PHR*) calculated previously.
Q_Vnotch = (Q_PHR_new / 3600) / N_total_notches
print(f'The flow per each V-Notch is: {Q_Vnotch:.6f} m³/s\n')

# 4. The maximum height of water over weirs (h) is determined based on Equation 3, solve for "h":

Cd = 0.58 # Predetermined
theta_V = 90 # V-notch angle
g = 9.81 # gravity acceleration (m/s²)

h_max_height_water_over_weirs = max_height_water_weirs(Q_Vnotch, Cd, g, theta_V)
print(f'The maximum height of water over weirs is: {h_max_height_water_over_weirs:.3f} m')
print(f'The maximum height of water over weirs is: {(1000*h_max_height_water_over_weirs):.0f} mm\n')

""The height of water over the weirs is between 13mm and 63mm, which is within the design requirements by Metcalf & Eddy""

# 5. The water depth is determined based on the safety factor (15%).

SF_water_notch = 0.15
y = (1 + SF_water_notch) * h_max_height_water_over_weirs # Water depth over weirs (m)

print(f'The water depth over weirs (safety factor) is: {y:.3f} m')
print(f'The water depth over weirs (safety factor) is: {(1000*y):.0f} mm\n')

# 6. The width of the V-notch at the top is two times the water depth (y)

width_notch = 2 * y # Width of the V-notch at the top (m)
print(f'The width of the V-notch at the top is: {width_notch:.3f} m')
print(f'The width of the V-notch at the top is: {(1000*width_notch):.0f} mm\n')

➡ Design of V-Notches

The number of V-Notches per clarifier is: 497

The total number of V-Notches for all clarifiers is: 1491

The flow per each V-Notch is: 0.000888 m³/s

The maximum height of water over weirs is: 0.053 m
The maximum height of water over weirs is: 53 mm

The water depth over weirs (safety factor) is: 0.061 m
The water depth over weirs (safety factor) is: 61 mm

The width of the V-notch at the top is: 0.122 m
The width of the V-notch at the top is: 122 mm

# 7. The discharge per unit length of launder is determined as follows:

q_wlr = WLR_crit / 86400 * sides_launder # It's defined by the 2-sided launder (fed from both sides)
print(f'The discharge (q) per unit length of the launder is: {q_wlr:.4f} m³/m/s\n')

# 8. The critical height of water at launder discharge point (Yc) is determined based on the Equation 5 from Table P.C.2
# The clarifiers only have one discharge point.
n_disch_points = 1

# The perimeter of the clarifier is calculated:
# The perimiteer of the 2-sided launder clarifier tank is equal to the weir length (Lw) divided by sides_launder.
perimeter_clarifier = weir_length / sides_launder # Perimeter of the clarifier (m)
print(f'The perimeter of the clarifier is: {perimeter_clarifier:.1f} m')

```

```

# The farthest distance that water travels into the launder is calculated as follows:
x = perimeter_clarifier / (n_disch_points * 2)
print(f'The farthest distance that water travels into the launder is: {x:.2f} m\n')

# Width of the launder, previously defined as 0.5m.
width_launder = 0.5      # Width of the launder (m)

# To determine the Yc for 2-sided launder circular clarifiers is used the next function based on Equation 6.
Yc = critical_Yc(q_wlr, weir_length, width_launder, 9.81)
print(f'The critical height of water at launder discharge point (Yc) is: {Yc:.2f} m\n')

# 9. The maximum depth of water in the launder (H_water_launder) is calculated by the function defined at the beginning of this notebook.
# The maximum depth of water in the launder is determined based on the Equation 8 from Table P.C.2
"""Here is applied the x value (farthest distance)"""
H_water_launder = max_water_launder_H(q_wlr, x, width_launder, 9.81, Yc)
print(f'The maximum depth of water into the launder (H) is: {H_water_launder:.2f} m\n')

# 10. Multiplying H_water_launder by the depth safety factor (50%), defined at the beginning of this notebook, the depth of water in the laun
SF_H_depth = 0.5
H_water_launder_f = (1 + SF_H_depth) * H_water_launder
print(f'The depth of water into the launder applying the safety factor, is: {H_water_launder_f:.2f} m\n')

🔍 The discharge (q) per unit length of the launder is: 0.0089 m³/m/s

The perimeter of the clarifier is: 74.5 m
The farthest distance that water travels into the launder is: 37.23 m

The critical height of water at launder discharge point (Yc) is: 0.35 m

The maximum depth of water into the launder (H) is: 0.61 m

The depth of water into the launder applying the safety factor, is: 0.92 m

```


✓ Infiltration

This section is to calculate the infiltration

```
#relevant libraries import
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

#From https://www.abs.gov.au/census/find-census-data/quickstats/2021/SAL22315 the data of
#Melburne subburbs population and area was obtained

#creation of data frame based on population data
df = pd.read_csv('./melbourne_density.csv')

total_melbourne_population=df["Population"].mean()
total_melbourne_area=df["Area (km2)"].mean()

print("The total melbourne population is: " + str(total_melbourne_population))
print("The total melbourne area is: " + str(total_melbourne_area))

PE_domestic=PE_domestic #variable defined in previous python calculation notebook
print("The suburb under design population is: " + str(PE_domestic))
print(str(PE_domestic))
print(str(PE_commercial))

#estimation of under study suburb area based on area per population ratio

estimated_area=(total_melbourne_area/total_melbourne_population)*PE_domestic

print("The estimated area of the understudy suburb is: " + str(estimated_area) + 'km2')

#converting the estimated area into ha

estimated_area=estimated_area*100

print('The estimated area of the understudy suburb is: ' + str(estimated_area) + 'ha')

#from Metcalf and Eddy the infiltration for sewer can be estimated as 0.2 to 28 m3/ha/d
#lower value is taken as it is a new sewer

infiltration_ratio=0.2

infiltration=estimated_area*infiltration_ratio

print('The estimated infiltration is: ' + str(infiltration) +'m3/d')

#exporting data frame to latex
latex_table = df.to_latex(index=False,longtable=True, caption='Melbourne suburbs with area and population', label='tab:mel_subs')
with open("../document/melbourne_density.tex", "w") as f:
    f.write(latex_table)
```

```
↗ The total melbourne population is: 11199.702380952382
The total melbourne area is: 18.649833333333337
The suburb under design population is: 340000
340000
28461
The estimated area of the understudy suburb is: 566.1706996890863km2
The estimated area of the understudy suburb is: 56617.06996890863ha
The estimated infiltration is: 11323.413993781727m3/d
```


APPENDIX 12 TYPICAL DESIGN PARAMETERS FOR ACTIVATES SLUDGE PROCESSES

Table 21: Typical design parameters for activated sludge processes [Metcalf and Eddy, 2014]

Process name	Reactor type	SRT (d)	F/M (kg BOD/kg MLVSS-d)	Volumetric loading		MLSS (mg/L)	Total τ (h)
				lb BOD/1000 ft ³ /d	kg BOD/m ³ /d		
High-rate aeration	CMAS or plug flow	0.5–2	1.5–2.0	75–150	1.2–2.4	500–1500	1–2
Contact stabilization	CMAS or plug flow	5–10	0.2–0.6	60–75	1.0–1.3	1000–3000 ^a	0.5–1 ^a
						6000–10,000 ^b	2–4 ^b
High-purity oxygen	Staged	1–4	0.5–1.0	80–200	1.3–3.2	2000–4000	1–3
Conventional plug flow	Plug flow	3–15	0.2–0.4	20–40	0.3–0.7	1000–3000	4–8
Step feed	Plug flow or staged	3–15	0.2–0.4	40–60	0.7–1.0	1500–4000	3–5
Complete mix	CMAS	3–15	0.2–0.6	20–100	0.3–1.6	1500–4000	3–6
Extended aeration	CMAS or plug flow	20–40	0.04–0.1	5–15	0.1–0.3	2000–4000	20–30
Oxidation ditch	CMAS + plug flow	15–30	0.04–0.1	5–15	0.1–0.3	3000–5000	15–30
Batch decant (ICEAS, CAAS)	Plug flow	12–30	0.04–0.1	5–15	0.1–0.3	2000–5000	20–40
Sequencing batch reactor	Batch	15–30	0.04–0.1	5–15	0.1–0.3	2000–5000	15–40
Counter current aeration (CCAS™)	Plug flow	15–30	0.04–0.1	5–10	0.1–0.3	2000–4000	15–40

^a MLSS and detention time in contact basin.^b MLSS and detention time in stabilization basin.